

Computer Science College Notes

Natan Asrat

Linkedin: <https://www.linkedin.com/in/natan-asrat>

Online view: [Computer Science College Notes](#)

Last updated: Wednesday, June 11, 2025

Table of Contents

[Computer Programming](#)
[Database Systems](#)
[Object Oriented Programming](#)
[Computer Networking](#)
[System and Network Administration](#)
[Data Structures and Algorithms Analysis](#)
[Computer Security](#)
[Automata and Complexity Theory](#)
[Compiler Design](#)
[Design and Analysis of Algorithms](#)
[Artificial Intelligence](#)
[Web Programming](#)
[Operating Systems](#)
[Advanced Database Systems](#)
[Computer Organization and Assembly Language](#)

Computer Programming

- Process of writing, testing, debugging, and maintaining source code of computer programs.
- Problem: gap between existing and desired/proposed situation.
- Problem solving techniques:
 - Analyze problem

- Outline problem requirements: UI? Data? Output?
- Design algorithm
- Properties of algorithms:
 - Correct: always desired output
 - Unambiguous: clearly defined steps
 - Precise: simple and efficient
 - Efficient: measured in time and space
 - Finiteness: terminates
- Tools to document programs:
 - Flow charts: represent processes with symbols
 - rectangle: process
 - skewed rectangle: input/output
 - diamond: decision (if/else)
 - circle: connector
 - rectangle with side stripes: subroutine call
 - rounded rectangle: terminal (start/stop)
 - lines
 - Structured chart: logical functions
 - pseudocode
- Programming language: defined by syntax and semantics
 - Low level languages: machine-dependent
 - High level languages: machine-independent
- Programming paradigms: programmer's view of code execution
 - Procedural: list of operations to reach desired state
 - Structured (in Procedural): break program structure to pieces (no GOTO statements)

- OOP: specify data structure and operations to be applied on those data structures
- C++ elements
 - Keywords: unique meaning in C++, cant be reused for other purpose
 - Identifiers: name of function or data
 - Literals: constant values i.e. numbers or strings (10.5)
 - Comments: descriptive text
 - Data types, Variables, Constants
 - Console communication (cin, cout)
 - Operators: Assignment:
 - Assignment Operators (=)
 - Arithmetic Operators (+, -, *, /, %)
 - Relational operators (==,!=, <, <=, >, >=)
 - Logical operators (! Negation, &&, ||)
 - Increment/decrement Operators
 - Ternary /Conditional operator (? :)
 - Control statements: flow control to determine what gets executed
 - Declaration statements: define variables
 - Assignment statements: algebraic computations
 - Branching statements: alternate paths
 - Loop statements: repeated computations
 - Scope of variables
 - Local variable: inside a block, has precedence if same name with global variable
 - Global variable: available from any part
 - Pointer: holds memory address of another variable

- Reference operator &: address of
- Dereference operator *: value pointed by
- File Management (fstream library)
 - File: used to store data in storage devices permanently
 - **ofstream**: Stream class to write on file
 - **ifstream**: Stream class to read from files
 - **fstream**: Stream class to both read and write from/to files.
 - File modes:
 - ios::in - opens and reads if file is found
 - ios::out - opens and writes, if found overwrite else new file is created
 - ios::app - opens and appends if found, else new
 - ios::binary - open if found for binary inp/output
 - ios::ate - open and position at end of file, used with ios::in, ios::out and ios::binary for appending
 - ios::trunc - open and truncate/delete all content if found
 - ios::nocreate: dont create new file if not found
 - Combine with pipe (|) after file name
 - example: fstream writefemp;
writefemp.open("Employee.txt",ios::app|ios::out)

Database Systems

- Data: known facts that can be recorded with implicit meaning
- Database: collection of related data

- Database Management System (DBMS): tool for creating and managing databases.
- A file system lacks features of a database:
 - Data redundancy and inconsistency,
 - Difficulty in accessing data,
 - Data isolation,
 - Integrity problems,
 - Atomicity problems,
 - Concurrent access anomalies.
- Database Approach Characteristics:
 - Self-describing nature: complete definition or description of the database structure and constraints
 - Data abstraction and Insulation between programs and data
 - Support of multiple views of the data: view is not stored
 - subset of the database or
 - contain virtual data derived from the database
 - Sharing of data and multiuser transaction processing
- Data model: collection of concepts that can be used to describe the structure of a database
 - High-level or conceptual: ER model, OO model
 - Low-level or physical: details on how data is stored, for specialists
 - Representational (or implementation): for end user understanding but can be implemented directly
- Database schema: description of a database, doesn't change frequently, specified during database design.
 - Three schema Architecture

- Internal level has an internal schema: physical storage structure, complete details
- Conceptual level has a conceptual schema: describe whole database for users; entities, data types, relationships, user operations and constraints...
- External or view level: part of the database that a particular user group is interested in
- Data Independence: change the schema at one level
 - Logical data independence: change the conceptual schema without having to change external schemas
 - Physical data independence: change the internal schema without having to change the conceptual schema
- Database modeling:
 - Phases - database development life cycle (DDLC)
 - Requirement Analysis: most important step, what is needed?
 - Conceptual Database Design
 - Logical database design (Data model mapping) - tables, indexes, views, transaction, access privileges
 - Physical Design - data files, table spaces
 - ERD (Entity Relationship Diagram)
 - Entity and Attribute
 - Entity may be an object with a physical existence, has attributes (properties that describe it)
 - Attribute types:
 - Composite vs Simple (Atomic): composite can be divided to subparts while simple cant.
 - Single-valued vs Multivalued: single number vs array of numbers

- Derived: age determined from birth date
- Key: one or more distinct attributes
 - Super key: collectively identifies an entity
 - Candidate key: minimal super key
 - Primary key: one of candidate keys for uniquely identifying entity set
- Relationship: association between entity types
 - Unary: one entity (i.e. employee manages employee)
 - Binary: two entities (i.e. student takes course)
 - Cardinality:
 - One to one (1-1): each entity take part only once in the relationship
 - One to many (1-M): one entity set can take part only once in the relationship set and entities in other entity set can take part more than once in the relationship set
 - Many to many (M-M): all entity sets can take part more than once in the relationship
- Relational Constraints
 - Domain Constraints: every cell in table must contain atomic values (smallest indivisible units)
 - Key Constraints: should be unique
 - Referential Integrity Constraints: between two relations or tables and used to maintain the consistency
- Enhanced Entity Relationship (EER) Model: improvements or enhancements to the existing ER to reduce complexity
 - includes the concepts of:
 - subclass and superclass,

- categories (UNION types),
 - attribute and relationship inheritance
- Relational Model Concepts in DBMS
 - Attribute: Each column in a Table
 - Tables: has two properties rows and columns. Rows represent records and columns represent attributes.
 - Tuple: a single row of a table containing a single record
 - Column: set of values for a specific attribute
 - Relation Schema: name of the relation with its attributes
- Properties of Relations
 - Name of the relation is distinct from all other relations.
 - Each relation cell contains exactly one atomic (single) value.
 - Each attribute contains a distinct name.
 - Each tuple has no duplicate value.
 - Order of tuple can have a different sequence.
- Functional Dependency: constraint between two attributes or sets of attributes of entities
 - denoted by $X \rightarrow Y$: values of Y component depend on, or are determined by the values of X component.
 - Types:
 - Partial Dependency: an attribute which is not a member of the key is dependent on some part of the key (composite/candidate key)
 - Full Dependency: functionally dependent on that attribute and not on any of its proper subset
 - Transitive Dependency: If A functionally governs B, AND If B functionally governs C THEN A functionally governs C

- Database Anomalies: flaws in databases which occurs because of poor database design
 - Insertion Anomaly: entire primary key is not known, database cannot insert a new record properly
 - Deletion Anomaly: a record is deleted that results in the loss of other related data
 - Update Anomaly: a change to one attribute value causes the database to either contain inconsistent data or causes multiple records to need changing.
- Normalization: organizing the data in database to avoid data redundancy and anomalies:
 - Normal forms: maintenance rules that are used to eliminate or reduce redundancy:
 - First normal form (1NF): all underlying domains of attributes contain only atomic (simple or indivisible) values
 - Second normal form (2NF): it is in 1NF and remove any partially dependent attributes and place them in another relation (only full dependency with key is allowed)
 - i.e. Emp-Teams(EmpId, Name, BDate, Gender, TeamId, Project, TeamName)
 - decomposition
 - EmpId → {Name, BDate, Gender}
 - TeamId → {Project, TeamName}
 - 2nf:
 - Employees(EmpId, Name, BDate, Gender)
 - Teams(TeamId, Project, TeamName)
 - Emp-Teams(EmpId, TeamId)
 - Third Normal Form (3NF): it is in 2NF and fully functionally dependent on every key of R and non-transitively dependent on every key of R..

- i.e. zip code and addresses of employee to separate table with foreign key from employee to address
- Attribute Data Types:
 - CHAR - fixed-length character
 - VARCHAR() - variable-length characters
 - DATE – actual date
 - BLOB – binary large object (good for graphics images, audio, sound clips, etc.)
 - NUMERIC - positive/negative numbers
 - Exact
 - SMALLINT → 16 bit
 - INT → 32 bit
 - LONG → 64 bit
 - DECIMAL → 128 bit
 - Approximate
 - FLOAT (single precision) → 32 bit with $\approx$ 6-9 digit precision...
 - REAL (variation of double) → 32 bit with $\approx$ 15 digit precision...
 - DOUBLE (double precision) → 64 bit with $\approx$ 32 bit precision...
- Data/File Storage
 - Primary storage: store a data which can be operated directly by the CPU; RAM and cache
 - Secondary storage: cannot be processed directly by the CPU, must first be copied into primary storage; hard disk
 - Tertiary storage: removable media, offline storage for archiving databases; CD-ROMs, DVDs
- File Organizations: how the file records are physically placed on the disk

- Heap file (unordered file): no particular order, appending new records at the end of the file
- Sorted file (sequential file): ordered by the value of a particular field (sorting key)
- Hashed file: hash determines record's placement on disk, hash is applied on a field (hash key)
- Secondary organization (auxiliary access structures):
- Indexes
 - Dense Index: created for every search key value (for each records) in each block; index contains search key value and a pointer to the actual record... faster, more size
 - Sparse Index: created only for some of the data records.
 - Single Level Ordered Indexes: assumed that a file already exists with some primary organization; index access structure is usually defined on a single field of a file, called an indexing field
 - Primary: specified on the ordering key field of an ordered file of records. every record has a unique value for that field.
 - Primary index is an ordered file whose records are of fixed length with two fields.
 - PK: ordering key field
 - pointer: points to disk block address
 - Clustering: have the same value for the clustering field, if ordering field is not a key field - same values across records - it can be used as clustering key.
 - Clustering index is also an ordered file with two fields.
 - Clustering field of the data file
 - Disk block/cluster pointer

- Secondary: some primary access already exists, secondary index is dense, faster retrieval.
 - Many secondary indexes can be created for the same file each represents an additional means of accessing that file based on some specific field.
- Relational algebra: basic set of operations for the relational model, to specify basic retrieval requests (queries)
 - result of an operation is a new relation
 - basic operations
 - Selection (σ) - Selects a subset of rows from relation.
 - Projection (π) - Deletes unwanted columns from relation.
 - Cross-product ($\times$) - Allows us to combine two relations.
 - Set-difference ($-$) - Tuples in relation 1, but not in relation 2.
 - Union ($\cup$) - Tuples in relation 1 and in relation 2.
 - Intersection ($\cap$) - Only tuples in relation 1 and in relation 2 (only the common tuples).
- SQL (Structured Query Language): statements for data definitions, and updates, both DDL and DML
 - DDL (data definition language)
 - CREATE, DROP, ALTER, TRUNCATE and RENAME
 - DML (data manipulation language)
 - SELECT, PROJECT, JOIN, INSERT, DELETE, and UPDATE

Object Oriented Programming

- **OOP**: allow the programmer to break down the problem into objects. It is a programming language model organized around objects rather than "actions"

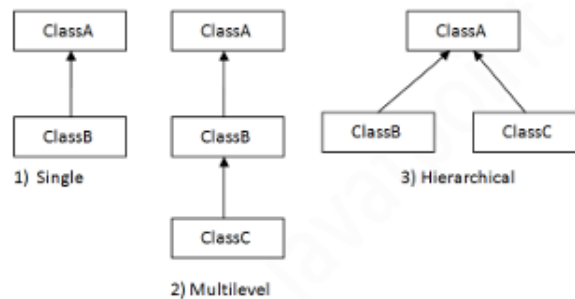
and data rather than "logic"

- **Objects:** self-contained entities consisting of both data and operations together.
- Java:
 - Object oriented
 - Platform independent
 - Simple, Secure, Portable
 - Multithreaded, Robust
- Java platform:
 - Java VM
 - Java API: provides libraries
- Objects: instance of a class, are runtime entities in OO program
- Class: blueprint for an object, definition of a set of objects
- Members: methods and variables defined in a class
- Abstraction: provide only essential information to outside world
- Encapsulation: data inside a class is hidden from everything outside the class (can only be accessed by invoking the methods of the class).
- Inheritance: objects of one class get the properties of objects of another class (reusability)
- Polymorphism: ability to take more than one form, a particular operation may behave differently for different sub-classes of the same class
- Components of java program:
 - Comments:
 - single line `//`,
 - multiline `/* .... */` ,
 - javaDoc `/** ....*/`
 - Token: individual units the compiler recognizes

- Whitespaces
- Variables in Java
 - Local variables: declared in methods, constructors or blocks, access modifier can not be used here
 - Instance variables: declared in a class, but outside a block
 - Class/Static variables: one copy of each class variable per class, regardless of how many objects are created from it
- Data types
 - Primitive (8 types): byte, short, int, long, float, double, char & Boolean
 - Reference: Arrays, classes, and interfaces, String
- Type casting
 - Widening, (Implicit) - Automatic Casting: conversion from one type to another which doesn't lose information
 - Narrowing (Explicit): Java won't do that automatically, do programmatically.
- Boxing and Unboxing
 - Autoboxing: conversion that the Java compiler makes between the primitive types and their corresponding object wrapper classes. i.e. converting an int to an Integer
 - Boxing: pass the value of the primitive data type to the constructor of the Wrapper class
- Java statements
 - Expression statements: perform computations and return values
 - Declaration statements: Introduces a very first variables
 - Control flow statements: regulate order of execution (if/else, loops)
 - Jumping Statements: break, continue
- Types of classes
 - Wrapper Class: eight classes of java.lang package

- Abstract Class: can't be instantiated.
- Final Class: can't be inherited.
- Input-Output Class: contains all the required classes to perform I/O operations (java.io package)
- String Class: Sequence of chars
- Mutable Class: fields that can be changed
- Types by access modifiers
 - Public Classes: Accessible to the world
 - Private Class: Used only inside a block(e.g with in the outer class)
 - Protected Class: Used in parent class, package and other subclasses
 - Default Class: Used only in parent class and package
- Method Overloading: process of having the same name for different methods as long as their parameter list or data types in the parameter list is different
- Method Overriding: same name and type signature as a method in its superclass
- Constructors: Initializes an object immediately upon creation.
 - Once you define your own constructor, the default constructor is no longer used.
 - Have not return type, not even void
 - Have the same name with their class name
- Garbage Collection: ability of deallocation of memory automatically, when no references to an object exist
 - no explicit need to destroy objects as in C++
- Access Specifiers:
 - No Modifier(Default): visible to package, aka package-private
 - Private: Visible to the class only

- Public: Visible to the world
- Protected: Visible to the package and all subclasses
- Static methods: can access only static data, and static methods.
- Final Variables: prevents the content of that variable being modified
- Final method: prevents from overriding the method
- Types of inheritance:



- Simple/single inheritance: a class extends another one class only
- Multi-Level inheritance: inherit from a derived class
- Hierarchical inheritance: inherited by many sub classes
- Constructors are not members, so they are not inherited by subclasses
- Super
 - Used to access parent class instance members
 - Used to access parent class constructor.
- Rules of method overriding in Java
 - similar argument list
 - exactly matching data types
 - same method name
 - access modifier can not be more restrictive of parent
- Dynamic Method Dispatch: mechanism by which a call to an overridden method is resolved at run time, rather than compile time (because it is

determined by the type of object)

- Method Overriding is an example of runtime polymorphism
- Abstract Classes: can't be instantiated, cannot declare abstract constructors, or abstract static methods
- Binding:
 - Late binding: resolves calls to methods dynamically, at run time
 - Early binding: since final methods cannot be overridden, a call to one can be resolved at compile time
- Types of Polymorphism
 - Static Polymorphism: compile-time binding or early binding - i.e. Method overloading
 - Dynamic polymorphism: late binding
- Exception: an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions. It is an indication of a problem. aka runtime error
 - A Java exception is an instance of a class derived from Throwable.
 - Reasons:
 - A user has entered invalid data
 - A file that needs to be opened cannot be found
 - A network connection has been lost
 - JVM run out of memory
 - In general:
 - User error
 - Programmer error
 - Physical resources failed
 - Types:

- Checked exception: occurs at the compile time - compile time exceptions.
 - Unchecked exceptions: occurs at the time of execution, - Runtime Exceptions.
- Exception Vs Errors
 - Errors: serious problem that a reasonable application should not try to catch.
 - Exception: conditions that a reasonable application might try to catch.
- File: an object on a computer that stores data, information, settings, or commands used with a computer program.
- Stream: linked to a physical device by the Java I/O system
 - InputStream: to read data from a source
 - OutputStream: for writing data to a destination.
 - Byte Streams: input and output of 8-bit bytes
 - FileInputStream
 - FileOutputStream
 - Character Streams: useful when we want to process text files
 - FileReader
 - FileWriter
- Java Interface: a blueprint of a class
 - has static constants and abstract methods
 - mechanism to achieve abstraction
 - to support multiple inheritance
 - to achieve loose coupling
- Rules of Interface
 - Variables declared in an interface are public, static & final by default.
 - can implement more than one interfaces in your class

- interface extends another interfaces, can extend multiple interfaces
- has no constructor
- Methods in an interface are all abstract
- Tagging Interface: empty interface, to claim the membership in a particular set i.e. Serializable, EventListener

Computer Networking

- Data: raw facts that are collected
- Information: processed data that enables us to take decisions
- Data Communication: process of exchanging data or information
 - Characteristics
 - Delivery: data delivered to correct destination and user
 - Accuracy: deliver data accurately without any errors
 - Timeliness: audio and video delivered in time without delay (real time transmission)
 - Jitter: variation in packet arrival time
 - Components
 - Message: information to be communicated
 - Sender: any device that is capable of sending the data
 - Receiver: a device that the sender wants to communicate the data
 - Transmission Medium: path by which the message travels
 - Protocol: agreed upon set or rules used by the sender and receiver to communicate data
- Data representation: text, numbers, images, audio, video
- Data flow:

- Simplex (unidirectional): Only one of the devices sends the data and the other one only receives the data
- Half Duplex: both the stations can transmit as well as receive but not at the same time.
- Full duplex mode: both stations can transmit and receive at the same time.
- Computer network: collection of an autonomous computing devices
 - Categories
 - Based on size:
 - LAN
 - MAN
 - WAN
 - Based on medium
 - Wired
 - Wireless
 - Based on architecture
 - Client Server
 - Peer to Peer
 - Based on transmission technology
 - Packet switched
 - Circuit switched networks
 - Based on administration
 - Private
 - Public
- Protocol:
 - Defines:
 - what is to be communicated,

- how it is to be communicated and
- when it is to be communicated
- Elements:
 - Syntax: structure or format of the data
 - Semantics: structure or format of the data, indicates the interpretation of each section
 - Timing: tells the sender about the readiness of the receiver
- Standards: to ensure interconnectivity and interoperability between various networking hardware and software components or vendors
 - De facto Standard: have been traditionally used and mean by fact or by convention, not approved by any organized body but are adopted by widespread use.
 - De jure standard: by law or by regulation.
 - Standard creation committees:
 - International Organization for Standardization(ISO)
 - International Telecommunications Union - Telecommunications Standard (ITU-T) American National Standards Institute (ANSI)
 - Institute of Electrical & Electronics Engineers (IEEE)
 - Electronic Industries Associates (EIA)
 - Standard creation forums:
 - ATM Forum
 - MPLS Forum
 - Frame Relay Forum
 - Standard creation agencies:
 - Federal Communications Committee (FCC)

During transit data is in the form of electromagnetic signals

- Analog data: information that is continuous

- Digital data: information that has discrete states

Signals:

- Analog Signal: infinite values in a range.
 - many levels of intensity over a period of time
 - simple analog signal: is a sine wave that cannot be further decomposed into simpler signals characterized by three parameters:
 - Peak Amplitude:
 - amplitude: absolute value of its intensity at time t
 - proportional to the energy carried by the signal
 - peak amplitude: absolute value of the highest intensity
 - Frequency: number of cycles completed by the wave in one second
 - Period: time taken by the wave to complete one second.
 - Wavelength: distance a signal travels in one period
 - $\text{Wavelength} = \text{Propagation Speed} \times \text{Period}$
 - $\text{Wavelength} = \text{Propagation Speed} \times 1/\text{Frequency}$
 - measured in micrometers
 - varies from one medium to another
 - Phase: position of the waveform with respect to time
 - indicates the forward or backward shift of the waveform from the axis It is measured in degrees or radian
 - Composite Signal: combination of two or more simple sine waves with different frequency, phase and amplitude.
- Digital Signal: limited number of defined
 - represented in the form of voltage levels
 - BIT LENGTH or Bit Interval (T_b): time required to send one bit.
 - measured in seconds.

- BIT RATE: number of bits transmitted in one second.
 - expressed as bits per second (bps).
 - Bit rate = $1 / \text{Bit interval}$
- Baud Rate: rate of Signal Speed, the rate at which the signal changes.
 - A digital signal with two levels 0 and 1 will have the same baud rate and bit rate.

Periodic & Non Periodic Signals

- Periodic: repeat itself after a fixed time period
- Non-Periodic: do not repeat itself after a fixed time period

Transmission Channels:

- Low pass Channel: has the lowest frequency as 0 and highest frequency as some non-zero frequency f_1
- Band pass channel: has the lowest frequency as some non-zero frequency f_1 and highest frequency as some non-zero frequency f_2

Transmission of Digital signal

- Baseband Transmission: transmitted without making any change to it (i.e. Without modulation)
 - the bandwidth of the signal to be transmitted has to be less than the bandwidth of the channel
 - Bandwidth: difference between the highest and lowest frequency
 - Wideband channel: channel whose bandwidth is more than the bandwidth of the signal
 - Narrowband channel: channel whose bandwidth is less than the bandwidth of the signal
- Broadband Transmission: use modulation, i.e. we change the signal to analog signal before transmitting it

- we have a bandpass channel we cannot directly send this signal through the available channel
- the signal is modulated using a carrier frequency
- signal is demodulated and again converted into an digital signal at the other end

Transmission impairments: transmission medium are not perfect, error is introduced in the transmitted data (received signal is not the same as the original signal)

- Causes:
 - Attenuation: loss of energy due to distance
 - resistance of the medium $\Rightarrow$ electrical energy in the signal may convert to heat.
 - amplifiers are used to amplify the signal.
 - Distortion: changes the shape of the signal.
 - Noise: unwanted signal that is mixed or combined with the original signal during transmission.

Transmission Media: anything that can carry information from a source to a destination

- usually free space, metallic cable or fiber optic cable
- Guided: cabling system that guides the data signals along a specific path
 - twisted-pair cable, coaxial cable transport signals in the form of electric signals
 - Twisted-pair cable: two conductors (normally copper), each with its own plastic insulation, twisted together.
 - Unshielded Twisted-Pair cable (UTP)
 - Shielded Twisted-Pair cable (STP)
 - Coaxial cable:- uses copper wire to conduct the signals electronically
 - fiber-optic cable transport signals in the form of light

-
- Unguided: without using a physical conductor, uses wireless electromagnetic signals to send data
 - Radio waves: frequencies between 3 KHz and 1GHz, Omni-directional
 - Micro waves: between 1 and 300 GHz, can be narrowly focused, line-of-sight propagation
 - Infrared: 300 GHz to 400 GHz, short range communication, cannot penetrate walls
- Propagation ways:
 - Ground-wave propagation
 - Sky-wave propagation
 - Line-of-sight propagation

Network Topology: description of the layout of the region or area covered by that network

- Connections
 - Point-to-point: provide a direct link between two devices
 - Multi-point connections: provide a link between three or more devices on a network.
 - Bus (point-to-multipoint) topology
 - Ring topology
 - Star topology
 - Hybrid topology
 - Mesh topology
 - Tree topology

Network Devices

- Repeater: operates only in the physical layer. A repeater receives a signal and, before it becomes too weak or corrupted, regenerates the original bit pattern.

- does not actually connect two LANs; it connects two segments of the same LAN
- Hub: used to connect segments (devices) of a LAN
 - hubs don't filter information;
 - hubs broadcast incoming packets to all computers connected to it
 - used in star or ring topology
 - broadcasting device
 - Passive hub: just a connector.
 - It connects the wires coming from different branches.
 - In a star topology Ethernet LAN, a passive hub is just a point where the signals coming from different stations collide.
 - below the physical layer
 - Active hub: a multipart repeater.
 - connections between stations in a physical star topology.
- Switch: switch filters and forwards data packets across a network.
 - keeps a record of the MAC addresses of the devices attached to it.
 - forwards the packet directly to the recipient device by looking up the MAC address.
 - intelligent and expensive than Hub
 - point to point communication device
- Router: specialized network device, used to interconnect different types of network that uses different protocol.
 - allows the users to connect several LAN and WAN
 - use the routing table to determine how to forward packets.
 - router analyzes a destination IP address of a given packet and compares it with the routing table to decide the packet's next best path
 - shares information with other routers in networking.

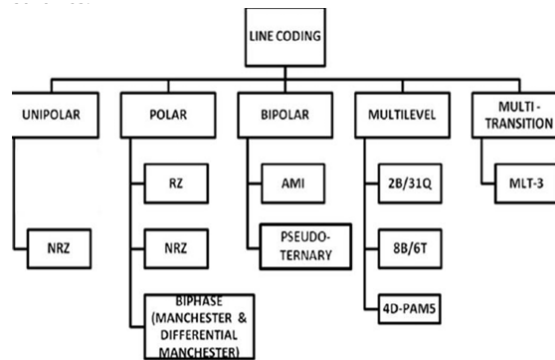
- Wireless router – offers Wi-Fi connectivity
- Broadband routers / Broadband modem – provided by the internet service provider
- Bridge: Connects two LANS having the same protocol
 - (e.g. Ethernet or Token ring).
 - filters content by reading the MAC addresses of source and destination.
 - does not filter broadcast
 - if data is not destined for other network, it is prevented from passing over the bridge
 - slower than repeater due to filtering
- Gateway: connects networks that use different protocols.
 - Transport gateway: connects two computers that use different transport protocols, reformatting packets as need be
 - Application gateway: understands the format and content of the data and translates messages from one format to another,

Errors:

- binary data:
 - Single bit errors: a bit value of 0 changes to bit value 1 or vice versa.
 - more likely to occur in parallel transmission.
 - Burst Errors: multiple bits of the binary value changes.
 - more likely to occur in serial transmission.
- add some extra bits to the original data to detect and correct the errors
 - redundant bits which will be removed by the receiver
 - Detection methods:
 - Parity Check
 - Cyclic Redundancy Check
 - Checksum

Signal Encoding:

- Digital Data to Digital Signal conversion:
 - Line Coding



- Block Coding
- Scrambling: not used
- Analog data to analog signal conversion:
 - Modulation: over long distances
 - Analog modulation: analog audio data is modulated onto a carrier sine wave
 - Amplitude modulation (AM)
 - Frequency modulation (FM)
 - Phase modulation (PM).
 - Digital modulation: to convert digital data to analog signal
 - Amplitude Shift Keying(ASK)
 - Frequency Shift keying (FSK)
 - Phase Shift keying (PSK)
- Analog to Digital Conversion using modulation:
 - Techniques:
 - Pulse Amplitude Modulation (PAM)

- Pulse Code Modulation (PCM)
- Pulse Width Modulation (PWM)

Network Models: operate networks, prominently the OSI reference model and the TCP/ IP Model

- Open Systems Interconnection (OSI):
 - developed by International Organization for Standardization (ISO)
 - developed to allow systems with different platforms (Hardware & software) to communicate with each other
 - Layers:
 - Physical Layer: provides a standardized interface to physical transmission media
 - Representation of bits: transmission of signals from one device to another which involves converting data (1's & 0's)
 - Data rate: defines the data transmission rate
 - Synchronization of bits: sender and receiver have to maintain the same bit rate and also have synchronized clocks
 - Line configuration: point to point link, or a multi-point link
 - Physical Topology
 - Transmission mode: simplex, half duplex or full duplex mode.
 - Data Link Layer: adds reliability to the physical layer by providing error detection and correction mechanisms:
 - Framing:
 - receives the data from Network Layer, divides the stream of bits into fixed size manageable units (Frames).
 - On the receiver side, the data link layer receives the stream of bits from the physical layer and regroups them into frames and sends them to the Network layer.

- Physical Addressing: appends the physical address in the header of the frame
 - Flow control: sender sends the data at a speed at which the receiver can receive it
 - Error Control: imposes error control mechanism to identify lost or damaged frames, duplicate frames and then retransmit them.
 - Access Control: imposes access control mechanism to determine which device has right to send data in an multipoint connection setting.
- Network Layer: makes sure that the data is delivered to the receiver despite multiple intermediate devices
 - sending side:
 - accepts data from the transport layer,
 - divides it into packets,
 - adds addressing information in the header
 - uses logical address commonly known as IP address to recognize devices on the network
 - each packet is independent of the other and:
 - may travel using different routes to reach the receiver
 - may arrive out of turn at the receiver
- Transport Layer: process to process delivery of data and makes sure that it is intact and in order.
 - sending side:
 - receives data from the session layer,
 - divides it into units called segments
 - makes use of port address to identify the data from the sending and receiving process

- transported in a connection oriented or connectionless manner
- responsible for segmentation and reassembly of the message into segments using sequence numbers
- carries out flow control and error control functions; but unlike data link layer these are end to end rather than node to node
- Session Layer (network dialog controller): establishes a session between the communicating devices called dialog and synchronizes their interaction
 - sending side:
 - accepts data from the presentation layer
 - adds checkpoints to it called sync bits
 - receiving end
 - receives data from the transport layer
 - removes the checkpoints
- Presentation Layer: performs translation, encryption and compression
 - communicating devices may be having different platforms.
 - sending side:
 - receives the data from the application layer
 - adds header information related to encryption and compression
- Application Layer: enables the user to communicate its data to the receiver by providing certain services
- TCP/IP model:
 - currently being used on our systems
 - collection of protocols often called a protocol suite
 - existed even before the OSI model was developed
 - Originally had four layers

- Host to Network Layer: combination of protocols at the physical and data link layers
- Network Layer or IP:
 - network layer protocol and is responsible for source to destination transmission of data
 - connection-less & unreliable protocol
 - best effort delivery service: no error checking
 - cannot be considered weak and useless; since it provides only the functionality that is required for transmitting data thereby giving maximum efficiency
 - combination of four protocols:
 - ARP - Address Resolution Protocol: resolve the physical address of a device on a network, where its logical address is known
 - physical address: imprinted on the NIC or LAN card
 - logical address (IP address): the Internet Address used to uniquely & universally identify a device
 - RARP - Reverse Address Resolution Protocol: used by a device on the network to find its own Internet address when it knows its physical address
 - ICMP - Internet Control Message Protocol: inform the sender about datagram problems that occur during transit
 - used by intermediate devices
 - IGMP - Internet Group Message Protocol: allows sending the same message to a group of recipients.
- Transport layer: responsible for transmission of data running on a process of one machine to the correct process running on another machine
 - transport layer contains three protocols:

- TCP – Transmission Control Protocol: reliable connection-oriented
 - divides the data it receives from the upper layer into segments and tags a sequence number
 - receiving end reorders data
 - UDP – User Datagram Protocol: simple protocol used for process to process transmission
 - unreliable, connectionless protocol for applications that do not require flow control or error control
 - adds port address, checksum and length information
 - SCTP – Stream Control Transmission Protocol: relatively new protocol
 - combines the features of TCP and UDP
 - used in applications like voice over Internet and has a much broader range of applications
-
- Application Layer: combination of Session, Presentation & Application Layers
 - defines high level protocols like:
 - File Transfer (FTP),
 - Electronic Mail (SMTP),
 - Virtual Terminal (TELNET),
 - Domain Name Service (DNS), etc.

IP Addressing:

- Packets in the IPv4 format are called datagram
- IP datagram consists of a header part and a text part (payload)
- IPv4:
 - IP addresses: uniquely identifies a device on the Internet.

- Every host and router on the Internet has an IP address,
- encodes its network number and host number.
- An IPv4 address is 32 bits long
- used in the Source address and Destination address fields of IP packets.
- An IP address does not refer to a host but it refers to a network interface
- Address Space: total number of addresses used by the protocol.
 - N bits to define an address $\Rightarrow$ the address space is 2^N
 - 32-bit IPv4 addresses, which means that the address space is 4,294,967,296 (more than 4 billion).
- Notations:
 - Binary notation: as 32 bits. ex. 11000001 10000011 00011011 11111111
 - Dotted decimal notation: easier to read, decimal point (dot) separating the bytes.
 - Each byte (octet) is 8 bits hence each number in dotted-decimal notation is a value ranging from 0 to 255.
 - Ex. 129.11.11.239
- Classful addressing: the address space is divided into five classes: A, B, C, D, and E.
 - An IP address in class A, B, or C is divided into netid and hostid.
- Subnetting: allows a network to be split into several parts for internal use but still act like a single network to the outside world.
 - router needs a subnet mask that indicates the split between network + subnet number and host.
 - Ex. 255.255.252.0/22
 - 22 to indicate that the subnet mask is 22 bits long.

- CIDR: allocate the remaining IP addresses in variable-sized blocks, without regard to the classes.
- NAT (Network Address Translation): scarcity of network addresses in IPv4 led to the development of IPv6. Due to this, we need to use private IP address inside the organization and translate it to public IP address using NAT.
- IP Header: IPV4 has 32 bit header information such as Version, IHL, total length, types of services

Routing Protocols:

- routing algorithm: determines the path for a packet
 - table types:
 - nonadaptive (static)
 - adaptive (dynamic)
 - protocol types:
 - Interior routing protocols
 - Exterior routing protocols
- routing:
 - requires a host or a router to have a routing table which is constructed by the routing algorithm

Switching and Multiplexing:

- switching: mechanism by which data/information sent from source towards destination which are not directly connected
 - Circuit Switching: two communicating stations are connected by a dedicated communication path
 - consists of intermediate nodes in the network and the links that connect these nodes
 - steps:

- Circuit Establishment: end-to-end connection before any transfer of data.
- Data transfer: data may be analog or digital, depending on the nature of the network, generally full-duplex.
- Circuit disconnect: Terminate connection at the end of data transfer.
- Packet Switching: designed to address the shortcomings of circuit switching
 - communication is discrete in form of packets.
 - switching modes:
 - Cut-through
 - Store-and-forward
 - Fragment-free
- Multiplexing
 - whenever the bandwidth of a medium linking two devices is greater than the bandwidth needs of the devices, the link can be shared
 - multiplexing: set of techniques that allows the simultaneous transmission of multiple signals across a single data link
 - Frequency division Multiplexing (FDM)
 - analog technique that can be applied when the bandwidth of a link (in hertz) is greater than the combined bandwidths of the signals to be transmitted.
 - signals generated by each sending device modulate different carrier frequencies.
 - Wavelength-division multiplexing (WDM):
 - uses the high-data-rate capability of fiber-optic cable.
 - optical fiber data rate is higher than the data rate of metallic transmission cable.

- Time division multiplexing
 - at different interleaved time intervals.
 - Each connection occupies a portion of time in the link.
- Medium access control:
 - problem of controlling the access to the medium similar to the rules of speaking in a meeting,
 - two people do not speak at the same time;
 - do not interrupt each other;
 - do not monopolize the discussion;
 - categories of multiple access protocols
 - Random Access Protocols - try your best like taxis do
 - MA - Multiple Access
 - CSMA - Carrier Sense MA
 - CSMA/CD - CSMA with Collision Detection
 - CSMA/CA - CSMA with Collision
 - Avoidance Controlled-Access Protocols - get permission
 - Reservation
 - Polling
 - Token Passing
 - Channelization Protocols - simultaneous use
 - FDMA - Frequency-Division MA
 - TDMA - Time-Division MA
 - CDMA - Code-Division MA

System and Network Administration

Computer system: a collection of entities (hardware and software)

- designed to receive process, manage and present information in a meaningful format.
- Hardware: the physical, tangible computer equipment and devices
 - Functions:
 - input,
 - processing (internal storage, computation and control),
 - output,
 - secondary storage (for data and programs), and
 - communication.
 - Components:
 - Input,
 - Processing,
 - Storage,
 - Output and
 - Communication devices.
- Software, also known as programs or applications, are the intangible components of the computer system.
 - Classification:
 - system software and
 - application software.
- Network: two or more computers connected together in such a way that they can share resources
 - Computer network: a collection of computers or other hardware devices that are connected together, either physically or logically, using special hardware and software, to allow them to exchange information and cooperate
 - Networking: processes involved in:

- designing,
- implementing,
- upgrading,
- managing and
- working with networks and network technologies
- Resources:
 - a file,
 - a folder,
 - a printer,
 - a disk drive, or
 - just about anything else that exists on a computer
- Classification of networks
 - by Geographical Span:
 - Personal Area Network (PAN): spanned across a given table
 - distances between the devices not more than few meters.
 - i.e. Bluetooth
 - belongs to a single person.
 - Local Area Network (LAN):
 - span across a building, or
 - across several buildings within a single organization,
 - using intermediate devices, like switches and/or hubs, to interconnect devices in all floors.
 - Metropolitan Area Network (MAN): span across a whole city interconnecting several buildings and organizations.
 - Wide Area Network (WAN): span across multiple cities, or an entire country, or an entire continent or it may even cover the whole world.

- Internet is one example of WAN.
- by Inter-connectivity (topology):
 - Bus Topology: all devices are connected to a central cable, called the bus or backbone, which is terminated at its ends.
 - Message transmitted along the Bus is visible to all computers connected to the backbone cable.
 - the workstation checks the destination address contained in the message either to process or drop the packet if it matches or not respectively.
 - Advantages:
 - ease of installation
 - less amount of cable requirement.
 - Drawback
 - entire network will be shut down if there is a break in the main cable.
 - Star Topology: each node is connected directly to a central device called a hub or a switch.
 - Data passes through the central device (switch) before continuing to its destination.
 - The central device manages and controls all functions of the network.
 - common with twisted pair cable.
 - RJ-45 Connectors are used to connect the cable to the Network Interface Card (NIC) of each computer.
 - Advantages:

- ease of installation and reconfiguration,
 - robust (ease of fault identification and isolation),
 - link failure only affects device(s) connected to that link,
 - less expensive than mesh.
- Drawbacks:
 - more cable requirements (than bus and ring)
 - single point of failure (if central device fail, the whole system will be down).
- Ring Topology: all devices are connected to one another in the shape of a closed loop,
 - each device is connected directly to two other devices, one on either side of it.
 - Advantages
 - easy to install and reconfigure,
 - less expensive (than mesh),
 - performance is even despite the number of users.
 - Drawbacks:
 - break in the ring (such as a disabled station) can disable the entire network,
 - limitations on media and traffic (limitation on ring length and number of devices).
- Mesh Topology: devices connected with many redundant interconnections between network nodes.
 - Full mesh: every node has a connection to every other node
 - most expensive
 - number of cables grows fast as the number of nodes increases,

- $((n(n-1))/2)$, , where n is the number of nodes
- Advantages:
 - dedicated links between devices,
 - robust (single link failure don't affect entire network),
 - privacy/security (direct communication between communicating devices),
 - ease of fault identification and isolation.
- Drawbacks:
 - installation and reconnection are difficult (large number of cables),
 - huge amount of cables consumes a lot of space, and
 - it is the most expensive of all.
- Hybrid Topology: contains more than one topology
 - inherits merits and demerits of all the incorporating topologies.
- by Administration:
 - Private network: belongs to a single autonomous system and
 - cannot access outside of its physical or logical domain.
 - Public network: can be accessed by anyone inside or outside of an organization
- by Architecture:
 - Client-Server Architecture:
 - one or more systems acting as Server.
 - Other being Client, request the Server to serve requests.
 - Servers take and process request on clients' behalf.
 - Peer-to-Peer (Point-to-point): Two systems can be connected Point-to-Point (back-to-back).
 - both reside on same level and called peers.

- can be hybrid network which involves network architecture of both the above types
- Internet Control Message Protocol (ICMP):
 - management protocol and messaging service provider for IP
 - works at the Network layer and is used by IP for many different services
 - events and messages:
 - Destination Unreachable: router can't send an IP datagram any further,
 - it uses ICMP to send a message back to the sender
 - Buffer Full: router's memory buffer for receiving incoming datagrams is full,
 - it will use ICMP to send out this message until the congestion abates.
 - Hops: Each IP datagram is allotted a certain number of routers, called hops, to pass through.
 - If it reaches its limit of hops before arriving at its destination, the last router to receive that datagram deletes it.
 - The executioner router then uses ICMP to send an obituary message, informing the sending machine of the demise of its datagram.
 - Ping (Packet Internet Groper): uses ICMP echo messages to check the physical and logical connectivity of machines on a network.
 - Traceroute Using ICMP timeouts, is used to discover the path a packet takes as it traverses an Internetwork.
- Process/Application Layer Protocols:
 - Telnet
 - File Transfer Protocol (FTP)
 - Network File System (NFS).

- Simple Mail Transfer Protocol (SMTP)
- POP3 is used to receive mail.
- Simple Network Management Protocol (SNMP)
- Domain Name Service (DNS)
- Dynamic Host Configuration Protocol (DHCP)

System Administration

- Philosophy
 - Network and system administration is a branch of engineering that concerns the operational management of human–computer systems
 - about putting together a network of computers, getting them running and then keeping them running in spite of the activities of users who tend to cause the systems to fail
 - A system administrator: works for users, so that they can use the system to produce work
 - about hardware, software, user support, diagnosis, repair and prevention.
 - need to know a bit of everything
 - skills are technical, administrative and socio psychological
 - System vs Network Administration:
 - System administration: term used traditionally by mainframe and Unix engineers to describe the management of computers whether they are coupled by a network or not
 - Network administration: management of network infrastructure devices (routers and switches)
 - The themes that make up the philosophy of system administration:
 - Automate everything
 - Document everything
 - Communicate as much as possible

- Know your resources
- Know your users
- Know your business
- Security cannot be an afterthought
- Plan ahead
- Expect the unexpected
- Backup and disaster recovery planning
- Patching
- Scope of Network administration:
 - Monitoring the network capacity to ensure that all transmission requirements can be met.
 - Adding capacity to the network by
 - increasing band width
 - interconnecting additional modes, or
 - creating and interconnecting additional networks.
 - Training people to use the network effectively
 - Assisting IT professionals in organizational applications that will make good use of the network's capabilities
 - Backing up the network software and data regularly to protect against failure of network
 - Putting security procedures
 - Making sure the network personnel can respond quickly and effectively in the event of a network operational or security failure
 - Diagnosing and troubleshooting problems on the network
- Goal of Network administration
 - to keep the network running properly and
 - configuring and managing services that are provided over the network

- Challenges of system administration
 - Designing a network which is logical and efficient.
 - Deploying large numbers of machines which can be easily upgraded later.
 - Deciding what services are needed.
 - Planning and implementing adequate security.
 - Providing a comfortable environment for users.
 - Developing ways of fixing errors and problems which occur.
 - Keeping track of and understanding how to use the enormous amount of knowledge which increases every year.
- Meta principles of system administration
 - Principle 1 (Policy is the foundation)
 - System administration begins with a policy – a decision about
 - what we want and what should be, in relation to what we can afford.
 - Policy speaks of:
 - what we wish to accomplish with the system, and
 - what we are willing to tolerate of behavior within it.
 - Principle 2 (Predictability)
 - highest level aim in system administration is to work towards a predictable system
 - what makes system administration difficult is that it involves a kind of 'search' problem. It is the hunt for a stable region in the landscape of all policies
 - In choosing policy, one might easily promote a regime of cascading failure, of increasing unpredictability that degenerates into chaos
 - Avoiding these regimes is what makes system administration difficult

- Principle 3 (Scalability)
 - scalable systems are those that grow in accordance with policy;
 - i.e. they continue to function predictably, even as they increase in size
 - When we speak of predictability, it must always be within a margin of error.

Windows Network Concepts

- Workgroup
 - In networking:
 - a collection of computers on a LAN that share common resources and responsibilities
 - According to Microsoft:
 - peer-to-peer Windows workgroups in homes, schools and small businesses that may share files, printers, or Internet connection
- Windows workgroup: group of standalone computers in a peer-to-peer network.
 - Each computer in the workgroup uses its own local accounts database to authenticate resource access.
 - no common authentication process
 - can contain many computers but work best with 15 or fewer computers
 - if number of computers increases,
 - re-organize into multiple networks or
 - set up a client-server network
 - joining a workgroup requires all participants to use a matching name
 - Windows 7, 8 and 10: WORKGROUP
 - Windows XP: MSHOME
 - To access resource:
 - know the name of the workgroup

- the username and password of an account on the remote computer
- Disadvantages of workgroups:
 - user account will need to be created on those machines with resources (same username and password).
 - low security protocol used for authentication between nodes.
 - Desktop computers have a fixed limit of 15 or less connections.
- Domain Controller: support client-server local networks.
 - Specially configured computer (Domain Controller) running a Windows Server operating system serves as a central server for all clients.
 - can handle more computers than workgroups
 - A client PC can belong to either to a workgroup or to a Windows domain, but not both.
- Microsoft HomeGroup: introduced in windows 7
 - simplify the management of workgroups for administrators particularly homeowners
 - HomeGroup security settings can be managed through one shared login instead of manually setting up shared user accounts
 - Joining a Homegroup does not remove a PC from its Windows WORKGROUP
 - PCs older than Windows 7 (like XP and vista), cannot be members of HomeGroups
- LDAP & Windows Active Directory
 - Lightweight Directory Access Protocol (LDAP)
 - open, vendor-neutral, industry standard application protocol for accessing and maintaining distributed directory information services over an Internet Protocol (IP) network
 - sharing of information about users, systems, networks, services, and applications throughout the network

- Protocol:

1. A client starts an LDAP session by connecting to an LDAP server (Directory System Agent - DSA)
2. client sends an operation request to the server,
 - a. StartTLS: use TLS for a secure connection
 - b. Bind – authenticate and specify LDAP protocol version
 - c. Search – search for and/or retrieve directory entries
 - d. Compare – test if a named entry contains a given attribute value
 - e. Add a new entry
 - f. Delete an entry
 - g. Modify an entry
 - h. Modify Distinguished Name (DN) – move or rename an entry
 - i. Abandon – abort a previous request
 - j. Extended Operation – generic operation used to define other operations
 - k. Unbind – close the connection (not the inverse of Bind)
3. server sends responses
 - client does not need to wait for a response before sending the next request
 - server may send the responses in any order
 - information is transmitted using Basic Encoding Rules (BER)
 - common alternative method of securing LDAP communication is using an SSL tunnel.

- Windows Active Directory

- Active Directory (AD): developed by Microsoft for Windows domain networks.

- AD Domain Controller:
 - authenticates and authorizes all users and computers in a Windows domain type network,
 - assigning and enforcing security policies for all computers, and
 - installing or updating software.
- Services:
 - Domain Services (DS) - best known
 - Lightweight Directory Services (LDS)
 - Certificate Services (CS)
 - Federation Services (FS)
 - Rights Management Services (RMS)
- Logical Structure: consists of a database and corresponding executable code responsible for servicing requests
 - Objects
 - resources (e.g., printers) and
 - security principals (user or computer accounts and groups)
 - Forests, trees, and domains: logical divisions
 - Domain: groups objects
 - Tree: collection of one or more domains in a contiguous namespace,
 - Forest: collection of trees that share a common global catalog, directory schema, logical structure, and directory configuration
 - represents the security boundary within which users, computers, groups, and other objects are accessible
 - Organizational Units: objects held within a domain can be grouped into organizational units
 - provides:

- hierarchy to a domain,
 - ease its administration, and
 - can resemble the organization's structure in managerial or geographical terms
- Microsoft recommends using OUs rather than domains for structure and to simplify the implementation of policies and administration.
- Physical Structure: defined by one or more IP subnets.
 - Replication: synchronizes changes using multi-master replication
 - replicas pull changes from the server where the change was effected
 - Knowledge Consistency Checker (KCC) creates a replication topology of site.
 - Intra-site replication is frequent and automatic as a result of change notification, which triggers peers to begin a pull replication cycle.
 - Inter-site replication intervals are typically less frequent and does not use change notification by default,
- AD Implementation
 - in general has more than one licensed Windows server computer.
 - Certain Microsoft products such as SQL Server and Exchange can interfere with the operation of a domain controller, necessitating isolation of these products on additional Windows servers.
 - Physical hardware costs for the many separate servers can be reduced through the use of virtualization
- AD Trusting: to allow users in one domain to access resources in another

- Trusts inside a forest are automatically created when domains are created
- Terminology
 - One-way trust: One domain allows access to users on another domain, but the other domain does not allow access to users on the first domain.
 - Two-way trust: allow access to users on both domains.
 - Trusted domain: The domain that is trusted; whose users have access to the trusting domain.
 - Transitive trust: A trust that can extend beyond two domains to other trusted domains in the forest.
 - Intransitive trust: A one way trust that does not extend beyond two domains.
 - Explicit trust: A trust that an admin creates. It is not transitive and is one way only.
 - Cross-link trust: explicit trust between domains in different trees or in the same tree when a descendant/ancestor (child/parent) relationship does not exist between the two domains.
 - Shortcut: Joins two domains in different trees, transitive, one- or two-way.
 - Forest trust: Applies to the entire forest. Transitive, one- or two-way.
 - Realm: Can be transitive or nontransitive (intransitive), one- or two-way.
 - External: Connect to other forests or non-AD domains. Nontransitive, one- or two-way.
 - PAM trust: A one-way trust used by Microsoft Identity Manager from a (possibly low-level) production forest to a

(Windows Server 2016 functionality level) 'bastion' forest, which issues time-limited group memberships.

- AD Management solutions
 - Active Directory Administrative Center ($\geq$ Windows Server 2012),
 - Active Directory Users and Computers,
 - Active Directory Domains and Trusts,
 - Active Directory Sites and Services,
 - ADSI Edit,
 - Local Users and Groups,
 - Active Directory Schema snap-ins for Microsoft Management Console (MMC),
 - SysInternals ADEplorer

User Administration

- User account: a collection of settings and information
 - which files and folders you can access,
 - what you can do on your computer,
 - what are your preferences, and
 - what network resources you can access when connected to a network.
 - allows you to authenticate to Windows or any other operating system so that you are granted authorization to use them
 - Attributes:
 - User name: the name you are giving to that account.
 - Password: the password associated with the user account (can also use blank passwords in $\leq$ Windows 7).
 - User group: a collection of user accounts that share the same security rights and permissions.

- A user account must be a member of at least one user group.
- Type: defines their permissions and what they can do in Windows.
 - Administrator: has complete control over the PC.
 - can install anything and make changes that affect all users of that PC.
 - Standard:
 - can only use the software that's already installed by the administrator and
 - change system settings that don't affect other users.
 - Guest: special type of user account that has the name Guest and no password.
 - can only use the software that's already installed by the administrator and
 - cannot make any changes to system settings.
- Multi-user operating systems such as Windows don't allow a user to use them without having a user account.
- File & Folder Permissions
 - Permissions: for assigning access rights to specific user accounts and user groups.
 - defines which user accounts and user groups can access which files and folders, and what they can do with them
 - to learn the permissions of any folder, right click on it and select —Properties.
 - Properties → Security tab → Group or user names section → see all the user accounts and use groups that have permissions to that folder.
 - User account or group can receive one of the permissions:
 - Read: allows the viewing and listing of a file or folder (and subfolders).
 - Write: allows writing to a file or adding files and subfolders to a folder.

- List folder contents: this permission can be assigned only to folders. It permits the viewing and listing of files and subfolders, as well as executing files that are found in that folder.
- Read & execute:
 - File: permits the reading and accessing of a file's contents as well as its execution.
 - Folders: allows the viewing and listing of files and subfolders, as well as the execution of files.
- Modify:
 - Files, allows their reading, writing and deletion.
 - Folders, allows the reading and writing of files and subfolders, plus the deletion of the folder.
- Full control
- Generally, files inherit the permissions of the folder where they are placed, but users can also define specific permissions that are assigned only to a specific file.
- Assigning User Rights
 - efficient way: make the user a member of a group that already has the right.
 - or give the user the rights directly
 - approach used with the Remote Desktop Users group
 - assign user rights through the Local Policies node of Group Policy.
- Roaming profile
 - set of files that contains all settings of a user including per-user configuration files and registry settings.
 - In an Active Directory or NT4 domain you can set that the profile of a user is stored on a server.
 - enables the user to log on to different Windows domain members and use the same settings

- when a user logs in: a copy of the profile is downloaded from the server to the Windows domain member when a user logs into.
- Until the user logs out, all settings are stored and updated in the local copy.
- During the log out, the profile is uploaded to the server.
- Application data and settings
 - Directories you create under c:\users\[your username] i.e. Documents, Downloads... have a roaming profile, the master copy of the profile is stored on a fileserver.
 - When you log in to a Windows computer, the contents of your profile will be synchronized from the fileserver to the local computer.
 - When you log out of the computer, any changes to the profile are then synchronized back to the server.
 - the contents of the profile are stored centrally, whenever you log on to any computer in the department you will have the same application data and settings (e.g., internet browser bookmarks, preferences in Microsoft Office etc.)
 - all data stored within File server is automatically backed up
- Roaming vs local profiles
 - Main advantage to using a local profile is that the profile is accessible even when the computer is disconnected from the network.
 - A major drawback of a local profile is that the user profile data is not being automatically backed up by the server.
 - If you have a large roaming profile, the login and logoff times may take a significant amount of time.
 - Solution:
 - Folder redirection with roaming profiles
 - allows specific folders (such as the Desktop and Documents folder) to be permanently stored on the server.

- eliminates the need for the redirected folder to be copied as a part of the logon and logoff processes
 - Windows Registry: hierarchical database that stores low-level settings for Windows operating system and for applications that opt to use the registry.
 - The kernel, device drivers, services, Security Accounts Manager, and user interface can all use the registry.
 - Registry Structure
 - Keys and values
 - keys are container objects similar to folders
 - referenced with a syntax similar to Windows' path names,
 - using backslashes to indicate levels of hierarchy
 - may contain values and subkeys
 - values are non-container objects similar to files.
- Routing and NAT
 - Network Address Translation (NAT): translates the IP addresses of computers in a local network to a single IP address
 - This address is often used by the router that connects the computers to the Internet.
 - Computers on the internet only see the IP address of the router.
 - adds an extra level of security, since the router can be configured as a firewall
 - Proxies and Gateways
 - Proxy server: acts as a gateway between you and the Internet
 - Used for:
 - Firewalls
 - Content filters
 - Bypassing content filters

- Caching
- Security
- Sharing Internet connections
- Gateway: a node (router), a key stopping point for data on its way to or from other networks.
 - Possible forms and tasks:
 - Web application firewalls
 - Cloud storage gateways
 - IoT gateways
 - Email security gateways
 - VoIP trunk gateways

Resource Monitoring and Management

a great deal of system administration revolves around resources and their efficient use.

System Performance Monitoring:

- normally done in response to a performance problem
 - Either the system is running too slowly, or
 - programs (and sometimes even the entire system) fail to run at all.
- Monitoring
 - to identify the nature and scope of the resource shortages that are causing the performance problems
 - Monitoring System Capacity:
 - part of an ongoing capacity planning program
 - Capacity planning uses long- term resource monitoring to determine rates of change in the utilization of system resources.
 - Once these rates of change are known it becomes possible to conduct more accurate long- term planning regarding the

procurement of additional resources

- What to Monitor:
 - CPU power, bandwidth, memory, and storage
 - How much free space is available?
 - How many I/O operations on average does it perform each second?
 - How long on average does it take each I/O operation to be completed?
 - How many of those I/O operations are reads? How many are writes?
 - What is the average amount of data read/written with each I/O?
- Monitoring CPU Power
 - determine if CPU utilization reaches 100%, if not - can process more work
- Monitoring Bandwidth: more difficult than the other resources to monitor
 - Statistics:
 - Bytes received/sent
 - Interface counts and rates
 - Transfers per Second
- Monitoring Memory: a wealth of performance statistics can be found
 - Statistics:
 - Page Ins/Page Outs
 - Active/Inactive Pages
 - Free, Shared, Buffered, and Cached Pages
 - Swap Ins/Swap Outs
- Monitoring Storage
 - Levels: possible to have dire problems in one area and no problems whatsoever in the other
 - Monitoring for sufficient disk space

- Monitoring for storage-related performance problems
- Statistics:
 - Free Space
 - File System-Related Statistics
 - Transfers per Second
 - Reads/Writes per Second
- Monitoring Tools
 - As your organization grows, so does the number of servers, devices, and services you depend on.
 - Users will notice performance problems with the software that they use, but those problems rarely arise within that software.
 - Knowing whether a computer has issues is fairly straightforward when the computer is right in front of you.
 - Even the smallest small-office/home-office network has multiple nodes: laptops, desktops, tablets, WiFi access points, Internet gateway, smartphones, file servers and/or media servers, printers, and so on. That means you are in charge of —infrastructure‖ rather than just —equipment.‖ Any component might start misbehaving and could cause issues for the others.
- 1. Bandwidth monitoring tools and NetFlow and sFlow
 - traffic analyzers help you stay aware of the activity, capacity, and health of your network
- 2. Windows Task Manager: information about computer performance and shows detailed information about the programs and processes running on the computer
 - name of running processes,
 - CPU load,
 - commit charge,
 - I/O details,

- logged-in users,
- Windows services,
- network status and how the network works

3. Windows Resource Monitoring: look at the presence and allocation of resources on a computer.

- Monitors:
 - CPU Usage
 - Memory
 - Commit: Amount of virtual memory reserved by the operating system for the process.
 - Working Set: Amount of physical memory currently in use by the process.
 - Shareable: Amount of physical memory in use by the process that can be shared with other processes.
 - Private: Amount of physical memory in use by the process that cannot be used by other processes.
 - Network Activity
 - Bandwidth: maximum data transfer rate of a network
 - most common Internet bottleneck is your connection to your Internet Service Provider
 - Internet speed can also be different from the Internet bandwidth expectations
 - Network bandwidth: fixed, can also be monitored by a network bandwidth monitor
 - can also optimize data so that it consumes less bandwidth from what is allocated
 - Network Printers

- Remote Administration: control either a computer system or a network or an application or all three from a remote location.
 - adopted when it is difficult or impractical to a person to be physically present and do administration on a system's terminal
 - Achieved using:
 - SSH (Secure Shell)
 - OpenSSH (OpenBSD Secure Shell)
 - Telnet
 - rlogin
 - rsh
 - VNC (Virtual Network Computing)
 - FreeNX
 - Wireless Remote Administration
 - Remote Desktop Connection (RDC)
- Redundant Array of Inexpensive (or Independent) Disks (RAID)
 - combines multiple physical disk drive components into one or more logical units for the purposes of data redundancy, performance improvement, or both.
 - in contrast with previous concept of highly reliable mainframe disk drives referred to as Single Large Expensive Disk (SLED)
 - Data is distributed across the drives in one of several ways, referred to as RAID levels
 - Originally, there were five standard levels of RAID, but many variations have evolved, including several nested levels and many non-standard levels (mostly proprietary).
 - standardised by: Storage Networking Industry Association (SNIA)
 - standard: RAID Disk Drive Format (DDF) standard
 - Levels:

- RAID 0:
 - consists of striping, but no mirroring or parity.
 - sum of the capacities of the drives in the set.
 - striping distributes the contents of each file among all drives in the set,
 - failure of any drive causes the entire RAID 0 volume and all files to be lost.
 - throughput of read and write operations to any file is multiplied by the number of
 - reads and writes are done concurrently.
- RAID 1
 - consists of data mirroring, without parity or striping .
 - Data is written identically to two or more drives,
 - produces a mirrored set of drives.
 - any read request can be serviced by any drive in the set.
 - it can be serviced by the drive that accesses the data first (depending on its seek time and rotational latency),
 - Actual read throughput of most RAID 1 implementations is slower than the fastest drive.
 - Write throughput is always slower because every drive must be updated, and the slowest drive limits the write performance.
 - The array continues to operate as long as at least one drive is functioning
- RAID 2 consists of bit-level striping with dedicated Hamming-code parity. All disk spindle rotation is synchronized and data is striped such that each sequential bit is on a different drive. Hamming-code parity is calculated across corresponding bits and stored on at least

one parity drive. This level is of historical significance only; as of 2014 it is not used by any commercially available system.

- RAID 3
 - consists of byte-level striping with dedicated parity.
 - All disk spindle rotation is synchronized and data is striped such that each sequential byte is on a different drive.
 - Although implementations exist, RAID 3 is not commonly used in practice.
- RAID 4
 - consists of block-level striping with dedicated parity.
 - advantage of RAID 4 over RAID 2 and 3 is I/O parallelism:
 - in RAID 2 and 3, a single read I/O operation requires reading the whole group of data drives,
 - while in RAID 4 one I/O read operation does not have to spread across all data drives.
 - more I/O operations can be executed in parallel, improving the performance of small transfers.
- RAID 5
 - consists of block-level striping with distributed parity.
 - Unlike RAID 4, parity information is distributed among the drives, requiring all drives but one to be present to operate.
 - Upon failure of a single drive, subsequent reads can be calculated from the distributed parity such that no data is lost.
 - requires at least three disks.
 - large RAID 5 implementations are susceptible to system failures
 - array rebuild time and the chance of drive failure during rebuild.

- Rebuilding an array requires reading all data from all disks, opening a chance for a second drive failure and the loss of the entire array.
- RAID 6
 - consists of block-level striping with double distributed parity.
 - Double parity provides fault tolerance up to two failed drives.
 - This makes larger RAID groups more practical, especially for high-availability systems,
 - requires a minimum of four disks.
 - The larger the drive capacities and the larger the array size, the more important it becomes to choose RAID 6 instead of RAID 5.
 - RAID 10 also minimizes these problems.

Security

Unix OS: a set of programs that act as a link between the computer and the user.

Kernel: allocates the system resources and coordinates all the details of the computer's internals

Shell: users use it to communicate with the kernel

Linux distribution: operating system that is made up of a collection of software based on Linux kernel

- contains the Linux kernel and supporting libraries and software
- Architecture

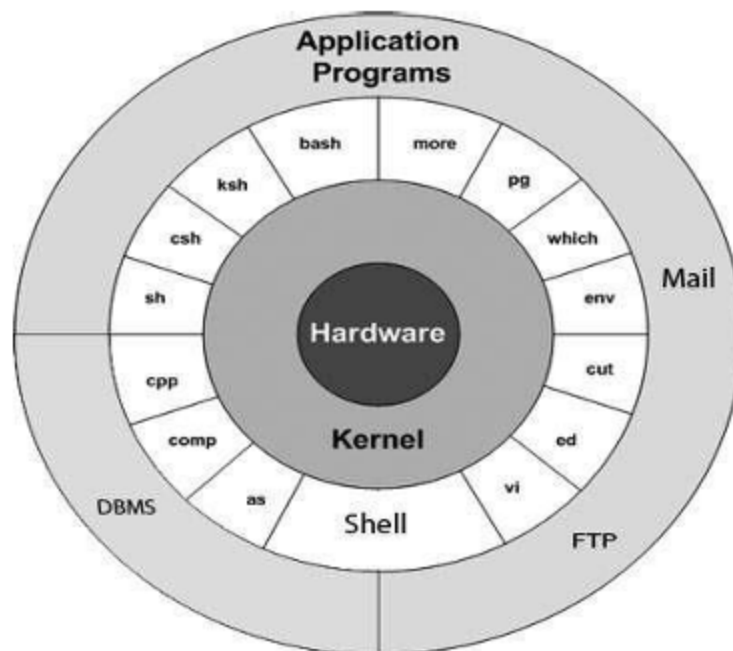


Figure. Block diagram of Unix system

- Properties of Linux
 - Linux Pros
 - A lot of the advantages are due to its root in UNIX:
 - Linux is free
 - Linux is portable to any hardware platform
 - Linux was made to keep on running
 - Linux is secure and versatile
 - Linux is scalable
 - very short debug-times
 - Linux Cons
 - far too many different distributions
 - not very user friendly and confusing for beginners

- Files

Symbol	Meaning
-	Regular file
d	Directory
l	Link
c	Special file
s	Socket
p	Named pipe
b	Block device

- Directory Structure

- /bin:
 - common programs,
 - shared by the system, the system administrator and the users.
- /boot:
 - startup files and the kernel, vmlinuz.
- /dev:
 - references to all the CPU peripheral hardware represented as files
- /etc:
 - most important system configuration files
 - data similar to those in the Control Panel in Windows.
- /home: Home directories of the common users.
- /lib:
 - Library files,

- includes files for all kinds of programs needed by the system and the users.
- /lost+found:
 - every partition has a lost+found
 - files that were saved during failures are here.
- /misc: for miscellaneous purposes.
- /mnt:
 - standard mount point for external file systems,
 - e.g. a CD-ROM or a digital camera.
- /opt: typically contains extra and third party software.
- /proc: virtual file system containing system resources information.
- /root: the administrative user's home directory.
- /sbin: programs for use by the system and the system administrator.
- /tmp :
 - temporary space to be used by the system,
 - its contents will be cleaned upon reboot,
 - don't use this for saving any work!
- /usr: Programs, libraries, documentation etc. for all user-related programs.
- /var:
 - storage for all variable and temporary files created by users,
 - log files,
 - temporary files downloaded from the Internet, or
 - to keep an image of a CD before burning it.

Data Structures and Algorithms Analysis

Computer program is a solution to a problem which contains two basic entities:

- The way data are organized in a computer's memory which referred as Data Structure and
- The sequence of computational steps to solve a problem is said to be an algorithm
- "program = algorithm + data structure"

Classification of data structure:

- Primitive: closest to the machine level instruction
- Non-Primitive: data item are not operated closest to the machine level instruction.
 - Linear: data items stored in sequence
 - Example: Array, stack and Queue,
 - Operations:
 - Traversal – Processing each element in the list
 - Search – Finding the location of the element with a given value.
 - Insertion – Adding a new element to the list.
 - Deletion – Removing an element from the list.
 - Sorting – Arranging the elements in some type of order.
 - Merging – Combining two lists into a single list.
 - Non-Linear: in which there is order of data items
 - Example: Trees and Graphs

Algorithm: finite set of instructions which, if followed, accomplish a particular task.

- Criteria
 - Input: there are zero or more quantities which are externally supplied;
 - Output: at least one quantity is produced;

- Definiteness: each instruction must be clear and unambiguous;
- Finiteness: for all cases the algorithm will terminate after a finite number of steps;
- Algorithm analysis: determining the amount of computing time and storage space required by different algorithms.
 - produce a function $T(n)$ that describes the algorithm in terms of the operations performed in order to measure the complexity of the algorithm.
- Complexity Analysis: isystematic study of the cost of computation,
 - measured either in time units or in operations performed, or in the amount of storage space required.
- Asymptotic analysis: how the running time of an algorithm increases with the size of the input in the limit, as the size of the input increases without bound.
 - Notations:
 - Big-Oh Notation: an upper bound on the growth rate of a function,
 - Big-Omega Notation: lower bound;
 - $f(n) = \text{Big-omega}(g(n))$ - means that the growth rate of $f(n)$ is greater than or equal to $g(n)$.
 - Theta Notation: have the same rate of growth (tight bound)
 - Little-o Notation: $f(n) = o(g(n))$ means $f(n)$ has less growth rate compared to $g(n)$.
 - Little-Omega Notation: a lower bound that is not asymptotically tight.

Searching: looking for a specific element in a list of items or determining that the item is not in the list.

- Linear/Sequential searching
 - Works with unsorted List
 - Loop through the list starting at the first element
 - time complexity $O(n)$.

- Binary Searching
 - Works on ordered/sorted list
 - divide and conquer method:
 - locates mid-point and identify which half the target belongs to since the list is sorted.
 - It performs this recursively till the item found or the list is empty (target is not in the list)
 - time complexity - $O(\log_2 n)$.

Sorting: reordering a list of items in either increasing or decreasing order.

- Insertion Sort: most instinctive type of sorting
 - finds the location for an element and move all others up
 - starts with the left most two items and put them in order
 - add 3rd item to the relatively sorted portion and find its correct location
 - Comparisons $\Rightarrow O(n^2)$; Swaps $\Rightarrow O(n^2)$
- Selection Sort: loop through and select smallest, put on left side, repeat
 - Comparisons $\Rightarrow O(n^2)$; Swaps $\Rightarrow O(n)$
- Bubble Sort: simplest to implement but slowest on very large inputs.
 - loop through and swap adjacent elements if out of order
 - Comparisons $\Rightarrow O(n^2)$; Swaps $\Rightarrow O(n^2)$

Struct: object consisting of multiple data items that may be of different types.

- Single Linked lists: self-referential structure, a chain of structs/Nodes
 - Each node in the list contains data items and pointer to next node
 - Operations:
 - Creating Linked Lists
 - Adding node to the list

```

if (start == NULL) // If list is empty, then new node become start node
    start = temp;
else {
    temp2 = start; // list not empty! Thus temp2 starts from first node
    while (temp2->next != NULL) { // till the current last node found navigates forward
        temp2 = temp2->next; // Move to next link in chain
    }
    temp2->next = temp; // new node (temp) appended at the end of the list
}

```

- Displaying the list of Nodes
- Deleting a node from the list

1. Look at the start pointer. If it is NULL, then the list is empty, so print out a "No nodes to delete" message.
2. Make temp1 point to whatever the start pointer is pointing to.
3. If the next pointer of what temp1 indicates is NULL, then we've found the last node of the list, so jump to step 7.
4. Make another pointer, temp2, point to the current node in the list.
5. Make temp1 point to the next item in the list.
6. Go to step 3.
7. If you get this far, then the temporary pointer, temp1, should point to the last item in the list and the other temporary pointer, temp2, should point to the last-but-one item.
8. Delete the node pointed to by temp1.
9. Mark the next pointer of the node pointed to by temp2 as NULL - it is the new last node.

- Stack: ordered list in which all insertions and deletions are made at one end, called the "top"
 - Operations:
 - Push: To add an element to the stack
 - Pop: To remove an element from the stock
 - Peek: To look at elements in the stack without removing them
 - Implementations:
 - Array implementation of stack

<u>Push(n): Adding item at top</u> <i>top is the current top of stack and n is its maximum size</i> Begin if (top == n) then stack is full; else top := top+1; stack[top] := item; end;	<u>Pop(): removing item from stack top</u> <i>remove top element from the stack and put it in the item</i> Begin: if (top == 0) then stack is empty; else item := stack[top]; top := top-1; end;
--	---

- Linked list implementation:
 - stack is adding items at the end of the chain (Push item)
 - deleting the end node of the list (Pop item)
- Applications:
 - to evaluate arithmetic expressions. (Postfix Expressions)
 - Infix to Postfix Transformation
 - execution of recursive programs
 - designing the compiler in operating system
 - function calls including recursion.
 - reversing a List
 - convert Decimal to Binary
 - backtracking
- Queue: ordered list in which all insertions at one end called REAR and deletions are made at another end called FRONT
 - Operations
 - Enqueue – Inserts an item at the rear
 - overflow error occurs if the queue is full.
 - Dequeue – Removes an item from the front and returns it to the user.
 - underflow error occurs if the queue is empty.
 - Implementations:
 - Simple Array implementation (static memory)

- even though there are free released spaces at the front of the array the queue implementation might generate overflow message.
- Circular array implementation
 - freed spaces are re-used to store data, copy data to left
- Linked list implementation
- Deque (pronounced as Deck): Double Ended Queue in which insertion and deletion can occur at either end.
- Priority Queue:- each data has an associated key provided at insertion time.
- Applications:
 - Print server
 - Disk Driver- queue of disk input/output requests
 - Task scheduler in multiprocessing system- priority queues of processes
 - Telephone calls in a busy environment
 - Simulation of waiting line- e.g. supermarkets checkout, customer service line...
- Tree: set of nodes and edges that connect pairs of nodes that connect pairs of nodes.
 - abstract model of a hierarchical structure.
 - Terminologies:
 - Root: a node without a parent.
 - Internal node: a node with at least one child.
 - External (leaf) node: a node without a child.
 - Ancestors of a node: parent, grandparent, grand-grandparent, etc of a node.
 - Descendants of a node: children, grandchildren, grand-grandchildren etc of a node.

- Depth of a node: number of ancestors or length of the path from the root to the node.
- Height of a tree: depth of the deepest node.
- Subtree: a tree consisting of a node and its descendants.
- Binary tree: a tree in which each node has at most two children called left child and right child.
- Binary search tree (ordered binary tree): a binary tree that may be empty, but if it is not empty it satisfies the following.
 - Every node has a key and no two elements have the same key.
 - The keys in the right subtree are larger than the keys in the root.
 - The keys in the left subtree are smaller than the keys in the root.
 - The left and the right subtrees are also binary search trees.
- Operations
 - Tree structure definition
 - Insertion
 - Tree Traversing
 - Pre order traversal - traversing binary tree in the order of parent, left and right.
 - Inorder traversal - traversing binary tree in the order of left, parent and right.
 - Postorder traversal - traversing binary tree in the order of left, right and parent.
 - Searching
 - Deletion
 - Cases:
 - Case 1: Deleting a leaf node, straightforward operation.
 - delete the leaf node and make the pointer of the deleted node parent NULL (right or left child pointer)

- Case 2: Deleting a node having only one child
- Case 3: Deleting a node having two children
- Case 4: Deleting the root node
- Approaches for Cases 2, 3, 4:
 - Deletion by copying:
 - Copy the node containing the largest element in the left (or the smallest element in the right) to the node containing the element to be deleted
 - Delete the Copied Node
 - Deletion by merging
 - Case 2 (1 child):
 - Promote a child (left or right) of deleted node as a child (left or right) for the parent of the deleted node keeping the definition of BST.
 - Case 3 (2 children):
 - Promote a child (left or right) of deleted node as a child for the parent of the deleted node.
 - The other child of the deleted node is made the left child of the node containing smallest element in the right of the deleted node or the right child of the deleted node is made the right child of the node containing largest element in the left of the deleted node
 - Case 4 (root):
 - the root node pointer is made to point to the right child or left child
 - the left child/right child of the root node is made the left child/right child of the node containing the smallest element/largest in the right/left of the root node respectively.

Advanced Sorting and Searching Algorithms

- Shell Sort: improvement of insertion sort, creates a reasonable order first.
 - time complexity is $O(n^{3/2})$
 - Algorithm:
 - Choose gap g_k between elements to be partly ordered.
 - Generate a sequence (called increment sequence) $g_k, g_{k-1}, \dots, g_2, g_1$ where for each sequence g_i , $A[j] \leq A[j+g_i]$ for $0 \leq j \leq n-1-g_i$ and $k \geq i \geq 1$
- Quick sort: fastest known algorithm, divide and conquer strategy
 - worst case its complexity is $O(n^2)$.
 - expected complexity is $O(n \log n)$.
 - Algorithm
 - Choose a pivot value (mostly the first element is taken as the pivot value)
 - Position the pivot element and partition the list so that:
 - the left part has items less than or equal to the pivot value
 - the right part has items greater than or equal to the pivot value
 - Recursively sort the left part
 - Recursively sort the right part
- Heap Sort: first convert the list in to a heap tree.
 - Heap tree is a binary tree in which each node has a value greater than both its children (if any).
 - process called "adjust to accomplish its task (building a heap tree) whenever a value is larger than its parent.
 - time complexity - $O(n \log n)$.
 - Algorithm:
 - Construct a binary tree

- The root node corresponds to Data[0].
- If we consider the index associated with a particular node to be i , then the left child of this node corresponds to the element with index $2i+1$ and the right child corresponds to the element with index $2i+2$. If any or both of these elements do not exist in the array, then the corresponding child node does not exist either.
- Construct the heap tree from initial binary tree using "adjust" process. Sort by swapping the root value with the lowest, right most value and deleting the lowest, right most value and inserting the deleted value in the array in it proper position.
- Merge sort: divide and conquer strategy
 - time complexity is $O(n \log n)$.
 - Algorithm:
 - Divide the array in to two halves.
 - Recursively sort the first $n/2$ items.
 - Recursively sort the last $n/2$ items.
 - Merge sorted items (using an auxiliary array).

Computer Security

Security: the quality or state of being secure (to be free from danger)

- protection against adversaries
- National security: protects the sovereignty of a state, its assets, its resources, and its people

Information Assurance: assuring information and managing risks related to the use, processing, storage, and transmission of information or data, the systems and processes used for those purposes.

- protection of the integrity, availability, authenticity, non-repudiation and confidentiality of user data.
- physical, technical and administrative controls to accomplish these tasks, not only digital
- restoration of information systems by incorporating protection, detection, and reaction capabilities.
- superset of information security.
- Pillars
 - Confidentiality:
 - Preserving authorized restrictions on access and disclosure.
 - Includes personal privacy and proprietary information protection.
 - Integrity:
 - protecting against improper information modification or damage
 - Availability:
 - ensuring timely and reliable access to and use of information.
 - service isn't denied to authorized users.
 - Authentication:
 - that you are who you say you are.
 - non-repudiation:
 - Prevention of either the sender or receiver denying a transmitted message.
 - Prove the occurrence of a claimed event or action and its originating entities.
- Information Assurance strategy
 - Cyber security awareness and education
 - Strong cryptography
 - Good security-enabled commercial information technology

- An enabling global Security Management Infrastructure; and
- A civil defense infrastructure equipped with an attack sensing and warning capability and coordinated response mechanism

InfoSec, computer security & IA

- IA: measures that ensure availability, integrity, authentication, confidentiality, and non-repudiation.
- Computer Security: (W. Stallings): protection to an automated information system
- InfoSec (Information protection) : protection from unauthorized access, use, disclosure, disruption, modification, perusal, inspection, recording or destruction.
 - Aspects:
 - IT security
 - IA: ensuring that data is not lost when critical issues like natural disasters, system malfunction, physical theft, etc. arise

Enterprise security & Cyber defense

- Enterprise security: building systems to remain dependable in the face of malice, error, or mischance.
- Cyber defense: includes response to actions and critical infrastructure protection and IA for possible networks.
- Business-driven approach to enterprise security: enabling the objective of an organization by controlling operational risk.

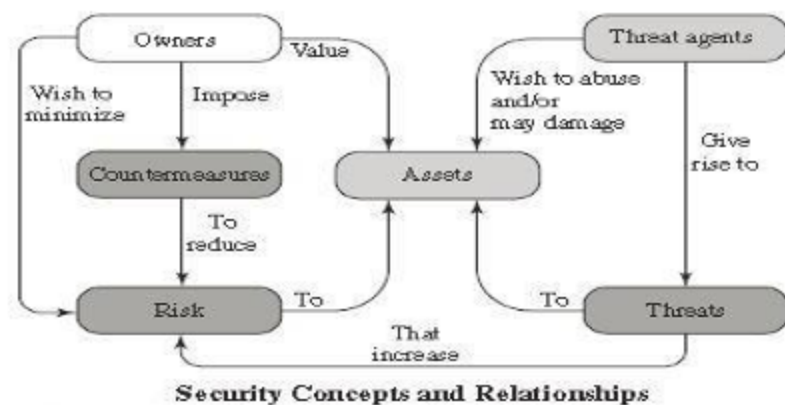
Threats, vulnerabilities, risk, controls (W. Stallings)

- Adversary (threat agent): entity that attacks (threat) a system, e.g. all hackers in the world
- Threat: events that can potentially harm an information system
- Vulnerability: weaknesses or fault in an information system or its components.

- Attack: actual violation of security that derives from an intelligent threat
- Risk: expectation of loss (exploitation a particular vulnerability)
 - $\text{Risk} = \text{Threats} \times \text{Vulnerabilities}$
- Control, safeguard, or countermeasure: An action, device, procedure, or technique that reduces a threat, a vulnerability, or an attack by eliminating or preventing it, by minimizing the harm it can cause, or by discovering and reporting it so that corrective action can be taken.
- Security Policy: set of rules and practices regulating how a system/organization protects sensitive and critical system resources.
- System Resource (Asset):
 - Data contained in a system; or
 - a service provided by a system; or
 - a system performance; or
 - an item of system equipment (hardware, software, or documentation); or
 - a facility that houses system operations and equipment.

Information security models and mechanisms

- Model for Computer Security



- Goals of computer security

- Confidentiality
- Integrity
- Availability... (same as above)
- Security Attack
 - Active attack: attempt to alter/affect system resources or their operation
 - Passive attack: an attempt to learn/make use of information from the system that does not affect system resources
 - Inside attack: Initiated by an entity inside the security perimeter (an insider).
 - Outside attack: Initiated from outside the perimeter.
- Types of Attacks on Data CIA
 - Disclosure/Interception (Attack on data confidentiality): unauthorized party snooping, or getting access to a resource
 - Modification/Deception (Attack on data integrity): unauthorized party modifying a resource,
 - Disruption/Interruption (Attack on data availability): unavailable services/resources, DoS
 - Fabrication: unauthorized party inserts a fake asset/resource, loss of confidentiality, authenticity and integrity of the message
- Types of Attackers
 - Amateurs: Opportunistic attackers (use a password they found), Script kiddies
 - Hackers - nonmalicious
 - Crackers – malicious
 - Career criminals
 - State-supported spies and information warriors
- Countermeasure Methods to Risk
 - Prevent attack: Block attack / Close vulnerability

- Deter attack: discourage potential attackers, Make attack harder,
- Deflect attack: Make another target more attractive than this target
- Detect attack: accept that an attack will occur; determine that an attack is under way, or has occurred, and report it.
- Recover from attack: Stop attack, assess and repair damage or restore the operations of the business
- Controls: Security policies & mechanisms
 - Encryption: primary control
 - Software controls: password checker, virus scanner, IDS (intrusion detection system)
 - Hardware controls: Locks, Smart cards, dongles, hardware keys...
 - Policies and procedures: users' legal and ethical standards; eg. password policy
 - Physical controls:
 - natural and man-made disaster protection;
 - walls, locks, guards, security, cameras, backup copies and archives
- Principles of Computer Security
 - Principle of Easiest Penetration
 - Principle of Adequate Protection
 - Principle of Effectiveness: Controls must be efficient, easy to use, and appropriate.
 - Principle of Weakest Link: Security can be no stronger than its weakest link.
 - Principles of Least Privilege: Don't give more privileges than necessary for the job.
 - Defense in Depth: having more than one layer or type of defense

- Minimization: do not run any software, or applications that aren't required for the job
- Compartmentalization: limits the damage in other compartments when one compartment is compromised.
- Keep Things Simple: Complexity is the worst enemy of security.
- Fail Securely: if security measure has failed, the system is not rendered to an insecure.
- Balancing Security and Access

Computer Threats/Attacks

- Types of Threats/Attacks
 - Physical Attack: Stealing, breaking or damaging of computing devices
 - Malware Attack: A generic term for software that has malicious purpose.
 - Viruses
 - Worms
 - Trojan horses
 - Spy-wares
 - Zombie
 - Botnets
 - Bacterium
 - Logic bomb
 - Backdoor(Trapdoor)
 - Adware
 - Flooders (DoS client)
 - Key loggers
 - Rootkit

- Spam/scam, identity theft, e-payment frauds, etc.
- Hacking /Intrusion/: attempt to intrude or gain unauthorized access, purpose may or may not be for malicious purposes.
- Cracking: is hacking conducted for malicious purposes.
- Denial of Service (DoS) Attack: Blocking access from legitimate users
- Distributed DoS Attack: tricking routers into attacking a target or using Zombie hosts to simultaneously attack a given target with large number of packets.
- Spoofing: MAC cloning, IP spoofing, and email spoofing
- Password attacks: guessing password; social engineering attack, Dictionary attack
- Cryptographic Attack:
 - Frequency Analysis attack: Guess values based on frequency of occurrence
 - Brute-force attack: attacker tries every possible key on a piece of ciphertext.
 - Cryptanalysis: the process of breaking an encrypted code.
 - Ciphertext only Attack,
 - Known plaintext Attack,
 - Chosen plaintext Attack,
 - Chosen ciphertext Attack, and
 - Chosen text Attack
- Class of Attacks:
 - Reconnaissance:
 - hackers first identifies a target to launch an attack,
 - extract maximum information regarding this target,
 - understand its vulnerabilities,

- and then only explore the best ways to exploit it.
- Examples:
 - Credential Theft using Keyloggers and Spyware; Identity
 - Theft using spam e-mail, phishing attack and social engineering attack
- Access: Once the attacker Reconnaissance the target, S/he starts a successful exploitation and continued access to the system. Then, the attacker is in control of the target.
- Denial of Service: Once the attacker having a successful access to the system. S/he starts to deny users from accessing or using the service or system.

Application of Security Mechanisms

Cryptography and hash functions

- Cryptography: science and art of secret, or hidden writing
- Cryptanalysis: science and art of breaking codes
- Cryptology - deals with both cryptography and cryptanalysis
- Terms:
 - Plaintext - the original form of a message
 - Ciphertext - the coded/encrypted form of a message
 - Cipher – an encryption method encompassing the algorithm and key
 - Key - known only by the sender/receiver, value that the cipher, as an algorithm, operates on.
 - Symmetric: same key used for encryption & decryption
 - Key Distribution: requires both parties to share a common secret key
 - Key Hierarchy:
 - Session key: temporary key, for one logical session then discarded

- Master key: used to encrypt session keys, shared by user & key distribution center
- Advantage: Simpler and faster
- Disadvantages: Less secured, key exchange problem, large number of keys are needed
- Categories of symmetric ciphers
 - Stream Ciphers: one bit or one byte at a time
 - Block Ciphers: characters of a fixed size (a block) encrypted at once
 - Playfair,
 - Hill ciphers,
 - modern symmetric ciphers
 - Combination Cipher
- Examples
 - Data Encryption Standard (DES) (56bits key)
 - 64 bit block length
 - Feistel cipher:
 - half of the data block is used to modify the other half of the data block
 - then the halves are swapped.
 - passes through an initial permutation (IP) that rearranges the bits to produce the permuted input.
 - followed by sixteen rounds of the same function: permutation and substitution functions.
 - The left and right halves of the output are swapped to produce the preoutput.
 - The preoutput passed through the inverse of the initial permutation function (IP⁻¹)

- Triple DES (3DES) (168 bits key)
- Advanced Encryption Standard (AES)
 - not a Feistel structure
 - one permutation and three substitution
 - Substitute bytes: S-box to perform a byte-by-byte substitution of the block
 - ShiftRows: A simple permutation
 - MixColumns: A substitution that makes use of arithmetic over GF(28)
 - AddRoundKey: A simple bitwise XOR of the current block with a portion of the expanded key
- Asymmetric: different keys are used for encryption & decryption
 - Secret transmission of key for decryption is not required
 - Every entity can generate a key pair and release its public key
 - Breaking these algorithms
 - does not involve trying every possible key;
 - involves trying to factor the large number (or taking discrete logarithms in a very large finite field)
 - If the number is too small, you have no security. If number is large enough, secured against all computing power.
 - Advantages: more secured, Authentication
 - Disadvantages: relatively complex
 - Public Key Infrastructure (PKI)
 - consists of
 - Certification Authorities (CAs),
 - Registration Authorities (RAs),
 - Certificate holders,

- Clients,
 - Repositories,
 - Cryptographic Algorithms and Protocols.
- management of public key and distribution of digital certificates
- SSL certificates and TLS (newer version)
- HTTPS certificates serve to identify the web site; ensure no-one can eavesdrop
- Examples
 - El Gamal: developed by Taher ElGamal
 - Variable key size (512 or 1024 bits), Less common than others
 - Diffie-Hellman Key Agreement (DH)
 - first practical method for secret share over unsecured channel
 - Rivest, Shamir, Adleman (RSA)
 - Variable Key Size (512, 1024, or 2048 bits), Most popular public key algorithm
 - Principle: no mathematical method to efficiently find the prime factors of large numbers
 - private and public keys are constructed from very large prime numbers
- Digital Signature Algorithm (DSA)
 - Digital Signature procedure:
 - authenticates sender, does not give privacy of data
 - Sender encrypts message with its private key
 - Receiver can decrypt using sender's public key
 - Encipher (encrypt): converting plaintext to ciphertext

- Decipher (decrypt) - recovering plaintext from ciphertext
- Hash functions
 - hash function takes a variable-size message as input and produces a fixed-size output (hash code)
 - no key is used
 - impossible for the contents of the plain text to be recovered
 - used by many operating systems to encrypt passwords
- Cryptography Classification:
 - Classical/Conventional Cryptography:
 - Substitution Cipher: replaces one symbol with another
 - Monoalphabetic Ciphers: one-to-one relationship between symbols in cipher and plain text
 - Additive cypher
 - Multiplicative cypher (with modulo)
 - Affine Ciphers: uses a pair of keys
 - Polyalphabetic Ciphers: one-to-many relationship between a character in the plaintext to a character in the ciphertext
 - Autokey Cipher
 - Playfair Cipher
 - Vigenere Cipher
 - Hill Cipher: key matrix with multiplicative inverse
 - Transposition Cipher: changes the location of the symbols
 - Keyless Transposition Ciphers: permute the characters
 - rail-fence transposition: transposed into several rows
 - Example (2 rail/row → key = 2)
 - Plaintext: THEBIGBANGTHEORY

- Output:

THEBIGBA
NGTHEORY

- columnar transposition: transposed into several columns
 - Example (2 columns → key = 2)
 - Plaintext: SECRET
 - Output

S R
E E
C T
- Keyed Transposition Ciphers: divide the plaintext into groups of predetermined size, called blocks, and then use a key to permute the characters in each block separately
 - Permutation key: key used for encryption and decryption
- Combining the Two Approaches
- Modern Cryptography: Symmetric key cryptography and Public key cryptography
 - Based on Kerckhoff's principle, always assume they, know the encryption algorithm.
 - Resistance of the cipher to attack must be based only on the secrecy of the key and the strength of the algorithm.

Application Security

- Vulnerability: any mistakes or weakness in the system security
 - Vulnerability Assessment: a software testing technique to evaluate the sudden increase of risks
 - Penetration Testing: authorized simulated attack on a computer system performed to evaluate the security of the system
 - Types of vulnerability scanner:

- Host Based:
 - Identifies the issues in the host or the system.
 - carried out by using host-based scanners
- Network Based:
 - will detect the open port, and identify the unknown services running on these ports.
 - carried out by using Network-based Scanners.
- Database Based :
 - identify the security exposure in the database systems to prevent from SQL Injections.
- Vulnerability Testing Methods:
 - Active Testing: tester will actively involve in the process of finding out the new test cases and analyzes the results.
 - Passive Testing: monitoring the result of running SW under test without introducing new test cases or data.
 - Network Testing: measuring the state of network operation over a period of time to find out the problems created by new services.
 - Distributed Testing: applied for testing distributed applications.
 - Penetration Test (pen test):
 - To identify both weaknesses (vulnerabilities) & strengths, to enable a full risk assessment.
 - Determine whether a system is vulnerable.
 - Determine whether defenses were sufficient.
- Secure Shell (SSH)
 - cryptographic protocol for operating NW services securely over an unsecured NW
- Secure Email
 - adds confidentiality and integrity protection to ordinary e-mail

- example: PGB
- Secure Web servers
 - securing the website or web application itself and the network around it.
 - i.e. securing against Cross-Site Scripting (XSS)

Firewall

- program or network devices (e.g. router) which filters access to a protected network from the internet connection
- monitors & controls incoming & outgoing network traffic based on predetermined security rules.
- Categorizes:
 - Network firewalls: traffic b/n two or more networks and run-on NW hardware.
 - Implemented at a specific point in the network path and protects all computers on the internal side of the firewall from all computers on the external side of the fire wall.
 - Host-based firewalls: run on host computers and control network traffic in/out of those machines.
 - Packet Filtering Router: Examines TCP/IP header information of network data packets going in both directions.
 - Application-Level Gateway (application firewall / proxy server): runs special software that acts as a proxy for a service request.
 - Proxy server receives requests for Web pages, accesses the Web server on behalf of the external client, and returns the requested pages to the users.
 - Circuit Level Gateway: Standalone Software that sets up two TCP connections (inside & outside). It determines which connections will be allowed.
- Firewalls implementation

- Hardware firewalls: a stand-alone product or integrated into the router.
- Software firewalls: installed on your computer and will protect your computer from outside
- Combination of both

Physical security

- nothing else about a computer system can be called secure without physical security

Operations security

- weaknesses in an operating system are the most valued amongst crackers.
- Biometric (e.g. voice, face, retina, iris, fingerprint) and physical token-based (swipe cards, pin-generating cards) authentication are sometimes used to augment simple passwords, but the costs and accuracy of the technology limit their adoption.

Access control

- many systems define which packets may and which packets may not pass based on the sender's IP address (router access control lists)
- access enforcement mechanism:
 - authorizes requests from multiple subjects (e.g. users, processes, etc.)
 - to perform operations (e.g., read, write, etc.)
 - on objects (e.g., files, sockets, etc.).
- Network access control (NAC): managing access to a network.
 - NAC authenticates users logging into the network
 - determines what data they can access and actions they can perform
 - examines the health of the user's computer or mobile device (the endpoints).
 - operating system provides an access enforcement mechanism.
 - Concepts of access control:

- a protection system that defines the access control specification and
- a reference monitor that is the system's access enforcement mechanism that enforces this specification.

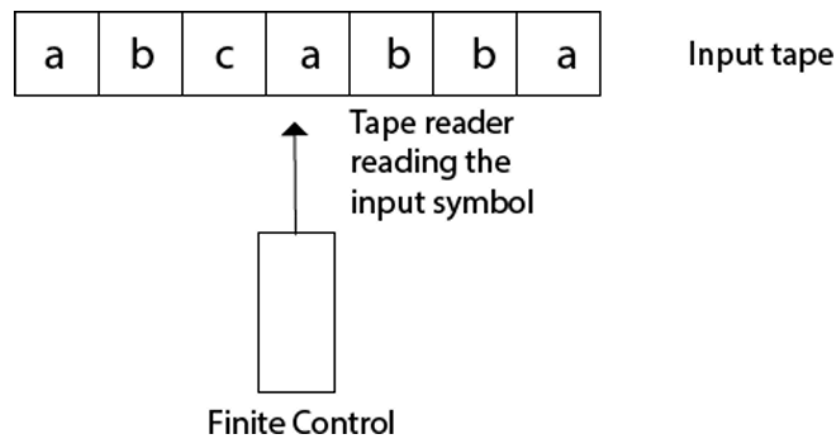
Plan, Design and manage security, security policies, business continuity plans, disaster recovery plans, and social and legal issues of information security
Design secure systems for real world information system services

Automata and Complexity Theory

Theory of Automata

- theoretical branch of computer science and mathematics
- study of abstract machines and the computation problems that can be solved using these machines
- the abstract machine is called the automata
- automaton consists of states and transitions.
 - State is represented by circles, and
 - Transition is represented by arrows
- Automata is the kind of machine that takes some string as input and this input goes through a finite number of states and may enter the final state
 - Symbols: an entity or individual objects, which can be any letter, alphabet or any picture
 - Alphabet: finite set of non-empty symbols. It is denoted by Σ (sigma)
 - String: finite collection of symbols from the alphabet. The string is denoted by w .
 - Language: collection of appropriate string.
 - can be Finite or Infinite.
 - Grammar: defines a set of rules, and with the use of these rules, valid sentences in a language are generated

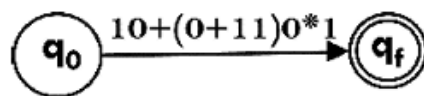
- popular way to derive a grammar recursively is phrase-structure grammar
- Finite Automata:
 - used to recognize patterns
 - takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs
 - at the time of transition, the automata can either move to the next state or stay in the same state
 - have two states, Accept state or Reject state
- Formal Definition of FA
 - collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$
 - Q : finite set of states
 - Σ : finite set of the input symbol
 - q_0 : initial state
 - F : final state
 - δ : Transition function
- Finite Automata Model: represented by input tape and finite control



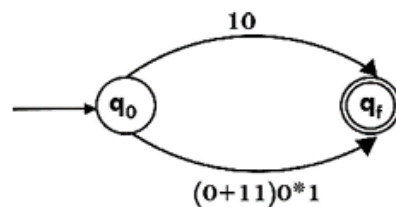
- Input tape: linear tape with some number of cells. input symbols placed in each cell.
- Finite control: decides the next state on receiving particular input from input tape.
 - tape reader reads the cells one by one from left to right,
 - at a time only one input symbol is read
- Types of Automata:
 - DFA(deterministic finite automata)
 - machine goes to one state only for a particular input character.
 - does not accept the null move
 - NFA(non-deterministic finite automata)
 - transmit any number of states for a particular input
 - can accept the null move
 - Every DFA is NFA, but NFA is not DFA
 - There can be multiple final states in both NFA and DFA
 - DFA is used in Lexical Analysis in Compiler
 - NFA is more of a theoretical concept
- Regular Expression
 - description of language accepted by finite automata
 - languages accepted by some regular expression are referred to as Regular languages
 - x^* means zero or more occurrence of x .
 - x^+ means one or more occurrence of x
- Operations on Regular Language
 - Union
 - Intersection

- Kleen closure: L^* = Zero or more occurrence of language L
- Conversion of RE to FA
 1. Design a transition diagram for given regular expression, using NFA with ϵ moves.
 2. Convert this NFA with ϵ to NFA without ϵ .
 3. Convert the obtained NFA to equivalent DFA.

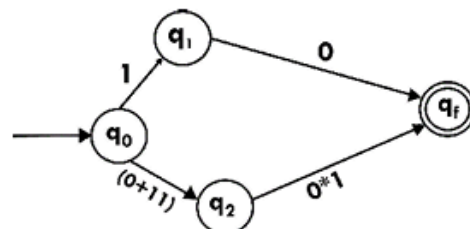
Step 1:



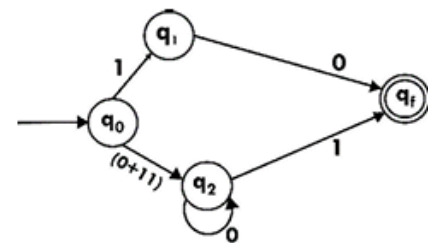
Step 2:



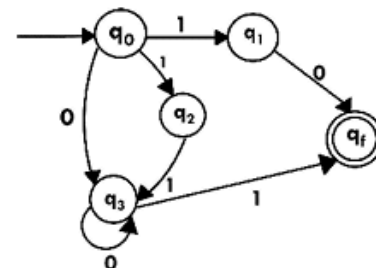
Step 3:



Step 4:



Step 5:

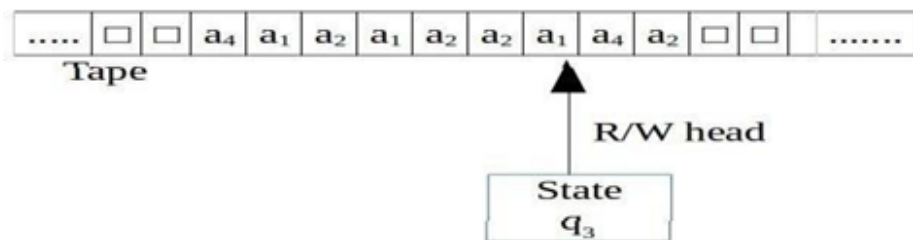


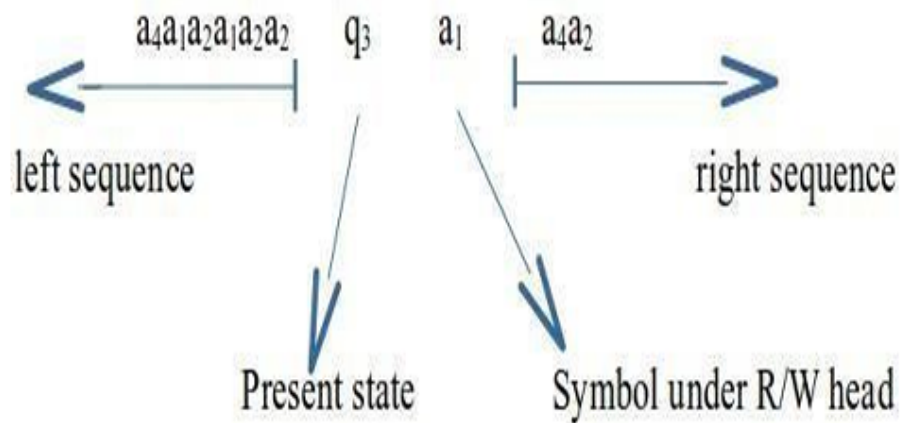
- Regular Languages
 - language that can be described with regular expressions
 - Regular Languages (LREG) is the set of all regular languages
 - language is regular language if and only if some finite state machine recognizes it
 - languages accepted by all DFAs form the family of regular languages

- Pumping Lemma:
 - For any regular language L , there exists an integer n , such that
 - for all $x \in L$ with $|x| \geq n$, there exists $u, v, w \in \Sigma^*$,
 - such that $x = uvw$, and
 - (1) $|uv| \leq n$
 - (2) $|v| \geq 1$
 - (3) for all $i \geq 0$: $u^i v^i w \in L$
 - this means that if a string v is 'pumped', i.e., if v is inserted any number of times, the resultant string still remains in L
 - if a language is regular, it always satisfies pumping lemma
- Context free languages
 - Context-free grammar: set of recursive rules used to generate patterns of strings
 - can be modelled as parse trees
 - nodes of the tree represent the symbols and the edges represent the use of production rules
 - leaves of the tree are the end result (terminal symbols) that make up the string the grammar is generating
 - described by a four-element tuple (V, Σ, R, S)
 - V is a finite set of variables (which are non-terminal)
 - Σ is a finite set (disjoint from V) of terminal symbols
 - R is a set of production rules
 - S (which is in V) which is a start symbol
 - sentential form: any string derivable from the start symbol
 - sentence is a sentential form consisting only of terminals
 - derivation tree: graphical representation of production rules in a CFG

- Simplification of grammar: reduction of grammar by removing useless symbols
 - Reduction of CFG,
 - Removal of Unit Productions,
 - Removal of Null Productions
- Pushdown automata
 - nondeterministic finite state machines augmented with additional memory (stack)
 - computational models (theoretical computer-like machines)
 - can do more than a finite state machine,
 - but less than a Turing machine
 - formally defined as a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, Z, F)$
 - useful when thinking about parser design and any area where context-free grammars are used
- there are two ways of proving that a language is context-free
 - provide the context-free grammar
 - provide a pushdown automaton for the language
- Turing machines:
 - first described by Alan Turing,
 - simple abstract computational devices intended to help investigate the extent and limitations of what can be computed
 - a model of general purpose computer
 - can do everything that a real computer can do
 - uses an infinite tape as its unlimited memory
 - has a tape head that can read and write symbols and move around on the tape
 - defined as a 7-tuple, namely $(Q, \Sigma, \Gamma, \delta, q_0, b, F)$

- Q is a finite nonempty set of states
- Σ is a nonempty set of input symbols and is subset of Γ and $b \notin \Sigma$
- Γ is a finite nonempty set of tape symbols
- $b \in \Gamma$ is the blank symbol
- δ is the transition function, mapping (q, x) onto (q', y, D) where
 - D denotes the direction of movement of R/W head:
 - $D = L$ or R (left or right)
 - δ may not be defined for some elements of $Q \times \Gamma$
- $q_0 \in Q$ is the initial state, and
- $F \subseteq Q$ is the set of final states
- TMs are deterministic: for each state there is only one unique Transition on each symbol
- Partial Transition Function: no transition for all input symbol
- Representation of Turing Machine
 - Instantaneous descriptions
 - defined in terms of the entire input string and the current state
 - written as $\alpha\beta\gamma$, where
 - β is the present state of M , and
 - input string is split as $\alpha\gamma$, the first symbol of γ is the current symbol under the R/W head





- Transition table
 - definition of transition function δ in the form of a table
 - If $\delta(q, a) = (\gamma, \alpha, \beta)$, we write $\alpha\beta\gamma$ under the α -column and in the q -row
 - $\alpha\beta\gamma$
 - α is written in the current cell
 - β gives the movement of the head (L or R)
 - γ denotes the new state
 - initial state is marked with $\rightarrow$ and the final state is marked with a circle

Present State	Tape Symbol		
	$\square$	0	1
$\rightarrow q_1$	1 L q_2	0 R q_1	-
q_2	$\square$ R q_3	0 L q_2	1 L q_2
q_3	-	$\square$ R q_4	$\square$ R q_5
q_4	0 R q_5	0 R q_4	1 R q_4
$\odot q_5$	0 L q_2		

- Transition diagram (transition graph)

- states are represented by vertices
- Directed edges are used to represent transition of states.
 - for directed edge from q_i to q_j with label (α, β, γ) , it means
 - $\delta(q_i, \alpha) = (q_j, \beta, \gamma)$
- possible outcomes of executing a Turing machine over a given input
 - Halt and accept the input
 - Halt and reject the input
 - Never halt (infinite loop)
- Decidable language (recursive language): there is a TM (decider) which accepts and halts on every input
- Recursive enumerable languages: TM halts sometimes & may not halt sometimes
 - not Recursively Enumerable that language is not accepted by any Turing Machine

Computational complexity

- Time Complexity: approximate number of operations required to solve a problem of size n
- Space Complexity: approximate memory required to solve a problem of size n
- Big-O notation: most commonly used notation for specifying asymptotic complexity
 - classes
 - Constant Time Complexity $O(1)$
 - Logarithmic time : $\log n$
 - Linear time $O(n)$
 - Quasilinear Time $O(n \log n)$
 - Quadratic Time $O(n^2)$
 - Exponential time $O(2^n)$

- Polynomial and Non Polynomial Class

Polynomial	Non Polynomial
P problems are set of problems which can be solved in polynomial time by deterministic algorithms.	NP problems are the problems which can be solved in non-deterministic polynomial time.
The problem belongs to class P if it's easy to find a solution for the problem.	The problem belongs to NP, if it's easy to check a solution that may have been very tedious to find
P Problems can be solved and verified in polynomial time. P problems are subset of NP problems.	Solution to NP problems cannot be obtained in polynomial time
P problems are subset of NP problems	NP problems are superset of P problems

Compiler Design

Compiler

- a program that reads a program written in one language and translates it into an equivalent program in another language
- converts human readable instructions to computer readable instructions one time
- Applications:
 - Parsers for HTML in web browser
 - Machine code generation for high level languages
 - Software testing
 - Program optimization
 - Malicious code detection
 - Design of new computer architectures

- Cousins
 - Preprocessor:
 - produces input for compiler
 - file inclusion, language extension, etc.
 - Assembler
 - assembly language into machine code
 - output of an assembler is called an object file
 - Linker
 - links and merges various object files to make an executable file.
 - determine the memory location where these codes will be loaded
 - Loader
 - loading executable files into memory and execute them.
 - calculates size of a program (instructions and data) and creates memory space for it.
 - initializes various registers to initiate execution.
 - Cross-Compiler
 - compiler that runs on one platform and generates executable code for another platform.
 - Source-to-source Compiler
 - compiler that translates source code of one programming language to another
- Phases
 - Analysis: Machine Independent, Language Dependent
 - Lexical / Linear Analysis (scanning)
 - Scans the source code as a stream of characters
 - Represent lexemes in the form of tokens as: <token-name, attribute-value>

- Token: smallest meaningful element that a compiler understands.
 - Identifiers, Keywords, Literals, Operators and Special symbols.
- Blanks, new lines, comments will be removed from the source program.
- Syntax / Hierarchical Analysis – Parsing
 - Tokens are grouped hierarchically into nested collections with collective meaning.
 - The result is generally a parse tree.
 - Most syntactic errors in the source program are caught in this phase.
 - Syntactic rules of the source language are given via a Grammar.
- Semantic Analysis
 - make sure that the components of the program fit together meaningfully.
 - checks for semantic errors in the source program (e.g. type mismatch)
 - stores type information in the symbol table or the syntax tree.
 - Types of variables, function parameters, array dimensions, etc.
- Intermediate Code Generation
 - Easy to produce and easy to translate to machine code
- Synthesis: Machine Dependent, Language independent
 - Code Optimization
 - Changes the Intermediate Code by removing inefficiencies
 - Code Generation
 - Converts intermediate code to machine code.
 - handle all aspects of machine architecture
 - Storage allocation decisions are made

- Register allocation and assignment

Interpreter

- converts human instructions to machine instructions each time the program is run

Lexical Analysis

- first phase of a compiler
- input: high level language program
- output: sequence of tokens
- Token: string of characters which logically belong together
 - Classes:
 - Identifiers: names chosen by the programmer
 - Keywords: names already in the programming language
 - Separators: punctuation characters
 - Operators: symbols that operate on arguments and produce results
 - Literals: numeric, textual literals
- Pattern: rule which describes a token
- Lexeme: sequence of characters matched by a pattern to form the token

Syntax Analysis

- input: streams of tokens from lexical analyzer
- output: parse tree
- Parsers or syntax analyzers are generated for a particular grammar
 - CFG are used for syntax specification
- Derivations:
 - Leftmost Derivation

- Apply a production only to the leftmost variable (non terminal) at every step
- Example grammar: $S \rightarrow aAS \mid a \mid SS$, $A \rightarrow SbA \mid ba$
- $S \Rightarrow a\underline{AS} \Rightarrow a\underline{SbAS}$
- $a\underline{SbAS} \Rightarrow a\underline{abAS}$
- $aab\underline{AS} \Rightarrow aab\underline{baS} \Rightarrow aabbaa$
- Rightmost Derivation
 - Apply production to the rightmost variable at every step
 - Example grammar: $S \rightarrow aAS \mid a \mid SS$, $A \rightarrow SbA \mid ba$
 - $S \Rightarrow aA\underline{S} \Rightarrow aA\underline{a}$
 - $aA\underline{a} \Rightarrow a\underline{SbAa}$
 - $aSb\underline{Aa} \Rightarrow aSb\underline{baa} \dots$
- Parsing: constructing parse tree for a sentence generated by a given grammar.
 - Top Down
 - Starts from the start symbol and transform it to the input
 - LL(1) Grammar (Left-to-right Left-most-derivation 1-lookahead)
 - Bottom Up
 - Starts with the input symbols and tries to construct the parse tree up to the start symbol.
 - LR(k) Parsing (Left-to-right Right-derivation k-lookahead)
 - Types
 - LR(0)
 - SLR(1)
 - LALR(1)
 - CLR(1)

Semantic Analysis

- Syntax Directed Translation: Attaching actions to the grammar rules
 - Actions are executed during the compilation, not during the generation of the compiler
- Syntax Directed Definitions
 - generalization of a context free grammar
 - CFG with attributes and rules

<u>Production</u>	<u>Semantic Rules</u>
$L \rightarrow E \text{ return}$	$\text{print}(E.\text{val})$
$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$
$E \rightarrow T$	$E.\text{val} = T.\text{val}$
$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$
$T \rightarrow F$	$T.\text{val} = F.\text{val}$
$T \rightarrow (E)$	$F.\text{val} = E.\text{val}$
$F \rightarrow \text{digit}$	$F.\text{val} = \text{digit.lexval}$

- Functions
 - $\text{mknode}(\text{op}, \text{left}, \text{right})$: Creates an operator node with label op & two pointers
 - $\text{mkleaf}(\text{id}, \text{entry}), \text{mkleaf}(\text{num}, \text{val})$: entry is pointer for identifier i.e. variable 'a', val is an actual value like 4

Type Checking

- Types: describes the values to be computed
- Type Errors: improper operations
- Type Safety: absence of type errors
- Type Checking: semantic checks to enforce the type safety

- Static – done during compilation
 - incompatible operands
 - Flow Control Check
 - Uniqueness Check
 - Name Related Check
- Dynamic – done during run-time
- Type Checking Expressions:
 - $E \rightarrow E1 \text{ mod } E2 \{ E.type = \text{if } E1.type = \text{int and } E2.type = \text{int then int else type_error} \}$

Intermediate Code Generation

- Three Address Code
 - sequence of statements of the form
 - $X = Y \text{ op } Z$
 - X,Y and Z are names, constants or compiler generated temporaries
 - Op is operator (arithmetic, logical)
 - LHS is the target (X in ' $X = Y \text{ op } Z$ ')
 - RHS has at most two sources and one operator (' $Y \text{ op } Z$ ' in ' $X = Y \text{ op } Z$ ')
- Implementations

Implementations of 3-Address Code

3-address code

```

1.  $t_1 = b * c$ 
2.  $t_2 = a + t_1$ 
3.  $t_3 = b * c$ 
4.  $t_4 = d / t_3$ 
5.  $t_5 = t_2 - t_4$ 

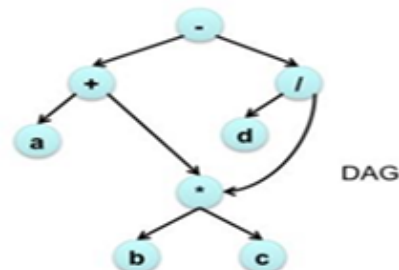
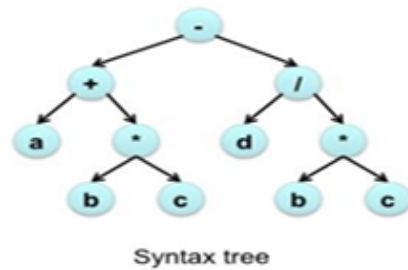
```

Quadruples

op	arg ₁	arg ₂	result
*	b	c	t1
+	a	t1	t2
*	b	c	t3
/	d	t3	t4
-	t2	t4	t5

Triples

	op	arg ₁	arg ₂
0	*	b	c
1	+	a	(0)
2	*	b	c
3	/	d	(2)
4	-	(1)	(3)



- Quadruples: instruction is divided into four fields
 - Operator, arg1, arg2, and result
- Triples: three fields
 - Operator, arg1 and arg2
- DAG and Tree: similar to triples
- Indirect Triples: pointers instead of position to store results

Code Optimization

- Remove redundant code without changing the meaning of program
- Techniques
 - Common sub-expression elimination: Repeated appearance computed previously
 - Strength reduction: Replacement of expensive expressions with simple ones
 - Code movement: Moving a block of code outside a loop

- Dead code elimination: Eliminate statements that are either never executed or unreachable
- Register Allocation
 - variables that die after being used only once can be reassigned for other expressions
 - Example:
 - a is only used in e's expression, e is only used in f's... name both a and e to same variable(register)

a = c + d

e = a + b

f = e - 1

r1 = r2 + r3

r1 = r1 + r4

r1 = r1 - 1

- Peephole Optimization: Transforming to optimal sequence of instructions
- Common Techniques:
 - Elimination of redundant loads and stores
 - $r2 = r1 + 5 \Rightarrow r3 = r1 + 5$
 $l = r2$
 $r3 = l$
 - Constant folding
 - $r2 = 3 * 2 \Rightarrow r2 = 6$
 - Constant Propagation
 - $r1 = 3 \Rightarrow r2 = 3 * 2 \Rightarrow r2 = 6$
 $r2 = r1 * 2$

- Elimination of useless instructions
 - $r1 = r1 + 0$
 - $r1 = r1 * 1$

Design and Analysis of Algorithms

Properties of an algorithm:

- Finiteness: must complete after a finite number of steps.
- Definiteness: each step must be clearly defined, having one and only one interpretation.
- Sequence:
 - each step has unique preceding and succeeding steps.
 - first (start) and last (halt) step must be clearly noted.
- Feasibility: must be possible to perform each instruction.
- Correctness: must compute correct answer for all possible legal inputs.
- Language Independence: must not depend on any one programming language.
- Completeness: must solve the problem completely.
- Effectiveness: must be possible to perform each step exactly and in a finite amount of time.
- Efficiency: must solve with the least amount of computational resources.
- Generality: should be valid on all possible inputs
- Input/Output: must be a specified number of input values, and one or more result values.

Analysis: provides a general idea of how long an algorithm will take for a given set

Components of space complexity:

- Instruction space: space needed to store the compiled version of the program instructions.
 - Factors:
 - Compiler used
 - Compiler options at time of compilation.
 - Target computer
- Data space: space needed to store all constant and variable values.
 - Components:
 - Constants and simple variables.
 - Dynamically allocated objects such as arrays and class instances.
- Environmental stack space: space used to save information needed to resume execution of partially completed methods (functions).
 - method func1 invokes or calls method func2, save a pointer where func1 will be executed when func2 terminates.
 - Each time a function or method is called or invoked, the following data are stored on the environment stack:
 - Return address
 - The values of all local variables and formal parameters in the method or function being invoked or called.

Algorithm Analysis: produce a function $T(n)$ that describes the complexity of the algorithm

- Order of Magnitude Analysis: Analysis of the function $T(n)$ to determine the general complexity category to which it belongs
- No generally accepted set of rules
 - assume an arbitrary time unit
 - time 1 operations
 - Assignment Operation
 - Single Input/output Operation

- Single Boolean Operations
- Single Arithmetic Operations
- Function Return
- if/switch:
 - time for the condition evaluation plus
 - maximum of the running times for the individual clauses in the if/switch cases
- loops:
 - running time of statements in the loop * number of iterations

Measures of Times

- Average Case: on an "average" set of inputs.
- Worst Case: on the worst possible set of inputs.
- Best Case: on the smallest possible set of inputs.

Design and analysis of Divide and Conquer Algorithms

- Base case scenario OR you divide the problem, run algorithms C & D on the subproblems and return the combined result of C & D
 - The complexity: (a - number of subproblems, n/b - input size of subproblems D & C - algorithms that take care of subproblems)

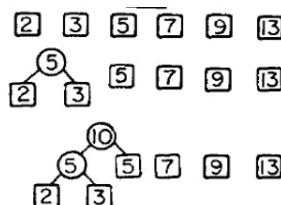
$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq C \\ aT(n/b) + D(n) + C(n), & \text{otherwise} \end{cases}$$

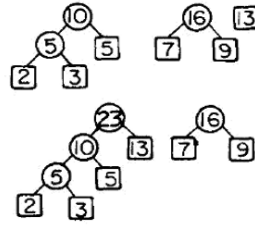
Greedy Model: most straightforward designed technique, 'short sighted', and 'non-recoverable'

- ideal only for problems which have 'optimal substructure'.
- best suited for simple problems
- mostly (but not always) fail to find the globally optimal solution

- Variations:
 - Pure greedy algorithms
 - Orthogonal greedy algorithms
 - Relaxed greedy algorithms
- Knapsack Problem
 - given n objects and knapsack or bag with capacity M
 - objects have weights and profits
 - we have to fill the bag with the help of N objects and the resulting profit has to be maximum
 - Steps:
 - sort objects by profit/weight ratio in descending order
 - select highest p/w ratio objects if their weights are less than the bag's available weight and decrement available weight by the selected object's weight and repeat
- Job Sequencing with Deadlines
 - given number of jobs (with profit and deadlines)
 - find sequence of jobs completed within deadlines yielding maximum profit
 - waiting time and the processing time should be less than or equal to the deadline of the job
- Minimum Cost Spanning Tree
 - A sub-graph $t=(V,E)$ of the G is a Spanning tree of G iff t is a tree
 - generate a graph $G = (V,E)$ where
 - We have to visit all the nodes
 - does not form a cycle
 - any connected graph with N vertices must have at least $N-1$ edges
 - Algorithms
 - Prim's Algorithm: compare's edge's weights before including them

- start from an arbitrary vertex (root)
- select the smallest edge that connects to the already chosen vertices
- Kruskal's Algorithm:
 - Sort all edges in increasing order of weight.
 - Pick the smallest edge and add it to the MST, as long as it doesn't form a cycle
 - result is a forest of trees that grows until all the trees merge in a single tree
- Optimal Merge Patterns: merging two sorted files into one sorted file
 - Pair wise merging:
 - have pointers on both linked lists first elements, pointers: p1 on LL1, p2 on LL2
 - have another pointer curr on the smallest of the two first items of the linked lists
 - advance p1 if it is smaller or advance p2 otherwise
 - compare p1 and p2's values and connect the next of curr to the smallest, advance the smallest from p1 and p2, advance curr to the next (smallest chosen), repeat
 - For multiple files:
 - Merge two smallest available files together
 - On the next merge, use the merged file with other available smallest files or just use the other smallest available files





- Huffman encoding

- https://www.youtube.com/watch?v=co4_ahEDCho
- reduce number of bits required using new encoding instead of ASCII for only the characters you have used... i.e. instead of 7 bits, you can have 3 bits if you use less than 8 characters, and 2 bits if you have less than 4 characters in the set of characters for your message.
 - Binary 'a': **01100001** , Binary 'b': **01100010**, Binary 'c': **01100011**
 - Your encoding for 'a': 00, your 'b': 01, your 'c': 10
- Huffman's approach:
 - rearrange the characters by frequency of occurrence
 - use optimal merge to construct the binary tree like the above image
 - the left edges will be labeled as 0 and right edges as 1
 - character (leaf) will have the encoding same as the labels in path from the root - shorter path will have less number of bits (inconsistent - sometimes 2 bits sometimes 3 bits per character ...)
 - use the tree to decode by traversing, write character when reached leaf, and start from root for the rest and repeat

- Single Source Shortest Path Algorithm (Dijkstra's)

- Single-source shortest path:
 - length of a path is now defined to be the sum of the weights of the edges on that path
 - determine the shortest path from v_0 to all the remaining vertices of G

- Dijkstra's Algorithm:
 - <https://www.youtube.com/watch?v=XB4MlexjvY0>
 - set distance for directly connected nodes to the source vertex (length of path)
 - set other node's distance to infinity
 - choose the smallest distance as current node and calculate the distance for the connected nodes by adding the distance of current node to the length of the edges connecting them.
 - if the calculated distance is less than the current distance, reassign (relaxation) - initially infinity so any distance will do on first time, on other visits it may or may not be reassigned, repeat above step only choosing unselected (smallest distance) nodes.

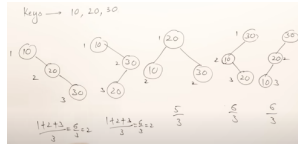
Dynamic programming

- like the divide-and-conquer method, solves problems by combining the solutions to subproblems
- unlike divide-and-conquer, which solves the subproblems top-down, a dynamic programming is a bottom-up technique
 - Start with the smallest sub problems and combine solutions
- stage-wise search suitable for optimization problems solved with a sequence of decisions
- early partial decision solutions are reused by keeping a table of known results of subproblems
- Principle of Optimality: the optimal solution to a problem is a combination of optimal solutions to some of its sub problems
- Multistage Graph: $k \geq 2$ disjoint sets of graphs (no edge in the same set, but edges connect nodes from one set to another); k = number of sets
 - problem: find minimum cost path from source node to destination node

- cost matrix: stores the distance from any vertex to the destination
- solution:
 - $\text{cost}(i, j)$ - cost of node at stage i and vertex j
 - $\text{cst}(i, j)$ or $\text{c}(i, j)$ - cost (length of path if direct) between vertex i and vertex j
 - forward method
 - <https://www.youtube.com/watch?v=9iE9Mj4m8jk>
 - start from last node and calculate the distance from each node connected to the last node (the previous stage), save the cost for each node and which path it took (path to destination node is the only path here)
 - to calculate for the nodes in the previous stage from there, calculate the minimum cost from each node to destination = length of edge to node in next stage + cost of node on the next stage (already calculated cost so reuse that value)... choose from the edges of that node in your current stage to find minimum cost for that node, and calculate for the other nodes in that stage
 - go back one more stage and repeat until reached the source node
 - backward method
 - <https://www.youtube.com/watch?v=JHRuqckZxWk>
 - start from source node and set its cost to 0, calculate the next stage's nodes cost which is the length of path from source the nodes
 - on the next stage after that, calculate cost of nodes by getting the minimum cost from their edges to the previous nodes (i.e. if node N_4 has two edges to nodes N_2 and N_3 , the minimum is $\min(\text{cst}(2, 4) + \text{cost}(1, 2), \text{cst}(3, 4) + \text{cost}(1, 3))$; or length of path to previous node + cost of that previous node (the minimum of all edges).

- go forward one more stage and calculate the next nodes until you reach the destination node
- All Pair Shortest Path
 - <https://www.youtube.com/watch?v=oNI0rf2P9gE>
 - calculate distance from each node to other node
 - Dijkstra only finds shortest path between two nodes
 - first build a table/matrix A0 where rows and columns are nodes and cells are the direct length from node in the row to node in the column, if no direct edge set the cell to infinity
 - now build another table/matrix A1 where the node 1 (1st row; also 1st column) is used as an intermediary node, so 1st row and 1st column will not be changed,
 - calculate the remaining cells by comparing the cell value in A0 to what the cell value would be if node 1 is intermediary and take the minimum
 - i.e. for row 2, column 3 $\Rightarrow (2, 3)$, assume the cell A0(2, 3) has value 4 and since we are using node intermediary we will add $A0(2, 1) + A0(1, 3)$... assume $A0(2, 1) = 1$, $A0(1, 3) = 2$; $A0(2, 1) + A0(1, 3) = 1 + 2 = 3 < A0(2, 3)$ so we choose the intermediary since its smaller (the path from 2 to 3 through 1 is shorter than the direct path from 2 to 3)
 - when building the second table having second node as intermediary use A1 as reference instead of A0
 - General formula:
 - $A^k[i, j] = \min(A^{k-1}[i, k], A^{k-1}[k, j])$
- Optimal Binary Search Trees
 - <https://www.youtube.com/watch?v=vLS-zRCHo-Y>
 - in an optimal tree, all the subtrees must also be optimal with respect to the keys that they contain

- for a sorted array of keys, there could be different variations of binary search trees built
- the optimal binary search tree is one with least cost or less number of comparisons to make when searching if an element exists in a tree (if searching for a greater value you choose right branch, else you look in left branch)
- the search terminates when reached the leaf without finding the element so the deeper the binary search tree the more comparisons are made and the cost will be more
 - if the tree is optimized into having less depth and more branches, the comparison/cost will be less since the other branch will not be checked if one branch is chosen according to the comparison to the parent node



◦ 0/1 Knapsack problem

- 0/1 : either carry whole object or dont, not divisible
- maximize profit, weight of carried objects $\leq$ weight capacity of bag
- Time complexity: $O(n * W)$.
 - n is the number of items and
 - W is the weight of the knapsack
- Steps
 - <https://www.youtube.com/watch?v=nLmhmB6NzcM>
 - Make a table with $n+1$ rows and $W+1$ columns (n - number of items, W - capacity of bag)

- first row and first column are 0 (first row - no item, first column - no weight used)
- in the second row cells (first item) write the profit(cell value) as 0 if weight of item is more than column (weight of knapsack), else write profit of item in the cells where the item's weight is equal to or less than the column
- in the next rows, fill the above row's values where the weight of item is more than the column, on first equal encounter fill the item's weight, on next column for the row, check if it is possible to combine any of the above rows' items and if so write the combined profit
- After filling the table, find the maximum value of profit, check if the above rows have the maximum value in any of the columns, pick the top most row with that maximum value and include that item in knapsack, i.e. 8 (maximum), item 4
- Deduct the profit of picked item from the maximum profit and look for the top most row with that value (i.e. item 4's profit = 6, deduct: $8-6=2$, look for 2 in the rows; item 2 & 3 have value 2 in their rows, pick 2; deduct; $2-2=0$... finished with item 4 and 2)
- General formula:
 - $A[i, w] = \max \{ A[i-1, w], A[i-1, w-W[i]] + P[i] \}$
 - i - item row; w - knapsack weight column;
 - $W[i]$ - weight of item, $P[i]$ - profit of item
 - second part in max: $A[i-1, w-W[i]] + P[i]$ could be invalid if $w-W[i]$ is negative or $W[i] > w$ (weight of item is more than knapsack so it won't fit), in that case you are using the above row's value (left part of max) $A[i-1, w]$
 - $fn(M) = \max \{ fn-1(x), fn-1(M-Wn)+Pn \}$

$m=8$
 $n=4$
 $P=\{1, 2, 5, 6\}$
 $\omega=\{2, 3, 4, 5\}$

		ω									
		V	0	1	2	3	4	5	6	7	8
P_i	ω_i	0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

$$V[i, \omega] = \max \{ V[i-1, \omega], V[i-1, \omega - \omega_i] + P_i \}$$

$$V[4, 8] = \max \{ V[3, 8], V[3, 8-5] + 6 \}$$

$$7, 2+6$$

○ Reliability Design

- <https://www.youtube.com/watch?v=uJOMqBwENB8>
- problem: in a system, some devices may fail and you will need extra just in case, reliability of each device is likelihood it will continue working, reliability of system is product of all it's component's reliability... task is to increase reliability by keeping multiple copies of a device with cost in mind and finding an optimum solution with highest system reliability
- solution
 - set a budget, i.e. 105, and calculate how much it takes to buy 1 of each device... calculate the remaining cash and with it you can buy more devices
 - upper bound for a device is how many you can buy with the remaining cash after you bought 1 of each device (includes 1 that you bought first then add the floor of remaining cash/cost of device]

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

- create set $S^0 = \{(1, 0)\}$ which means when no device is bought and cost is 0, reliability is 1
- create next set for first device
 - when buying only one copy of device 1 $S_1^1 = \{(0.9, 30)\}$
 - when buying two copies of device 1 $S_2^1 = \{(0.99, 60)\}$
 - $S^1 = \{(0.9, 30), (0.99, 60)\}$ - combined the two sets for device 1
 - superscript - device # ; subscript - # of copies
 - reliability for multiple copies: $1 - (1 - r)^k$ where k is # of copies, r is reliability of a device

$$\begin{aligned}
 1 - (1 - r_1)^2 &= 1 - (1 - 0.9)^2 \\
 &= 1 - (0.1)^2 \\
 &= 1 - 0.01 \\
 &= 0.99
 \end{aligned}$$

- on the next set, multiply the reliability of a device (# of copies changes this) with the previous reliabilities
 - when buying only one copy of device 2
 - device 1 of 1 copy $S_1^2 = \{(0.8 * 0.9, 15 + 30) = (0.72, 45)\}$
 - device 1 of 2 copies $S_1^2 = \{(0.8 * 0.99, 15 + 60) = (0.79, 75)\}$
 - $S_1^2 = \{(0.72, 45), (0.79, 75)\}$ - combined
 - when buying two copies of device 2
 - device 1 of 1 copy $S_2^2 = \{(0.96 * 0.9, 30 + 30) = (0.86, 60)\}$
 - device 1 of 2 copies $S_2^2 = \{(0.96 * 0.9, 30 + 30) = (0.86, 90)\}$
 - discard second set since for cost 90 currently, we cant buy device 3 for 20 and still have a budget of 105
 - $S^2 = \{(0.86, 60)\}$
- remove pairs in the set where the next item has more reliability with less cost
 - i.e. $S = \{(0.72, 45), (0.79, 75), (0.86, 60), (0.89, 75)\}$

- remove second element since the next element has more reliability with less cost
- $S = \{(0.72, 45), (0.86, 60), (0.89, 75)\}$
- continue until the set for the last item is complete and choose the highest reliability
- trace back the origins for that set element to find optimum count of devices

Handwritten notes showing the dynamic programming solution for the knapsack problem. The table lists items with their IDs, costs, reliabilities, and utilities. The sets S^0, S^1, S^2, S^3 are built up step by step, showing the inclusion and exclusion of items based on their cost and reliability. The final result is $C=105$.

D_i	C_i	r_i	u_i
D_1	30	0.9	2
D_2	15	0.8	3
D_3	20	0.5	3

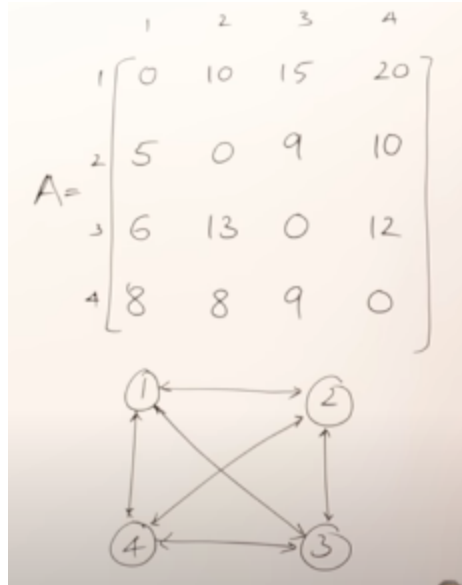
$C=105$

Handwritten sets and calculations:

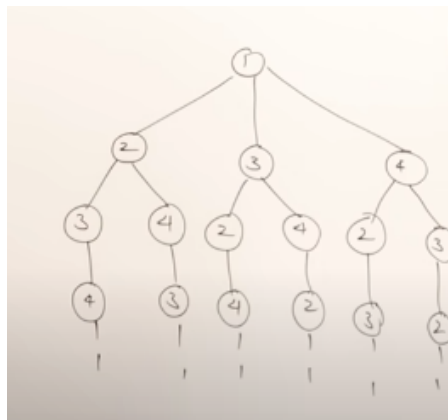
- $S^0 = \{(1, 0)\}$
- $D_1=1$
 - $S_1^1 = \{(0.9, 30)\}$
 - $S_2^1 = \{(0.99, 60)\}$
 - $S^1 = \{(0.9, 30), (0.99, 60)\}$
- $D_2=2$
 - $S_1^2 = \{(0.72, 45), (0.792, 75)\}$
 - $S_2^2 = \{(0.864, 60), (0.9, 90)\}$
 - $S_3^2 = \{(0.8928, 75), (0.9, 105)\}$
 - $S^2 = \{(0.72, 45), (0.792, 75), (0.864, 60), (0.8928, 75)\}$
- $D_3=2$
 - $S_1^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$
 - $S_2^3 = \{(0.54, 85), (0.648, 100), (0.6, 115)\}$
 - $S_3^3 = \{(0.63, 105), (0.72, 120), (0.75, 135)\}$
 - $S^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95), (0.54, 85), (0.6, 115), (0.648, 100)\}$

◦ Travelling Salesman Problem

- <https://www.youtube.com/watch?v=XaXsJJh-Q5Y>
- directed graph with edge cost c_{ij} is defined, find a tour of minimum cost



- tour of G is a directed cycle that include every vertex in V = shortest path that starts and ends at the same vertex
- Brute force
 - draw a tree where the root is the start node
 - connect children nodes of all remaining available nodes except starting node
 - continue with 3rd level where children of a 2nd level node are from available nodes except root and 2nd level node itself, continue downward in same fashion

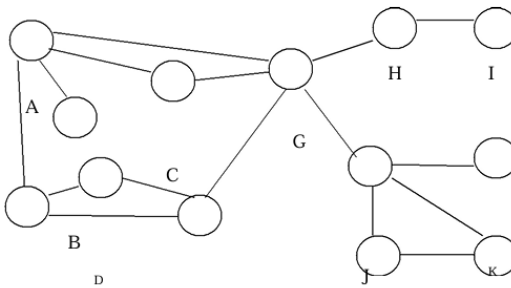


- $g(2, \{3\})$ means being on node 2 where remaining node is 3 (in bottom-up it means you have visited 3 and you are now at 2)
- $g(2, \{3, 4\})$ means 3 and 4 are remaining (in bottom-up you visited either 3 and then 4, or it means you visited 4 and then 3 but you are now in node 2)
 - $4 \rightarrow 3 \rightarrow 2$ scenario
 - $3 \rightarrow 4 \rightarrow 2$ scenario
 - choose minimum
 - $g(2, \{3, 4\}) = \min(c_{2,3} + g(2, \{4\}), c_{2,4} + g(2, \{3\}))$ - the values inside have already been calculated, multiply by either $c_{2,3}$ or $c_{2,4}$ and get the min
- finally calculate $g(1, \{2, 3, 4\}) = \min(c_{1,2} + g(2, \{3, 4\}), c_{1,3} + g(3, \{2, 4\}), c_{1,4} + g(4, \{2, 3\}))$

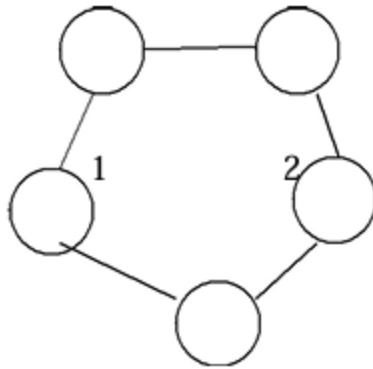
Traversal Techniques

- Complete graph: undirected graph with
 - n vertices
 - exactly $n(n-1)/2$ edges
- Reachability problem: determine whether there exist a path in the graph which starts at vertex 'v' and ends at vertex 'u'
- Search methods:
 - <https://www.youtube.com/watch?v=pcKY4hjDrxk>
 - Breadth First: uses queue of unexplored nodes and array of visited nodes
 - start at vertex v and mark it as having been reached
 - vertex is 'explored' when the algorithm has visited all vertices adjacent from it
 - all unvisited vertices adjacent from v are visited next

- there are new unexplored vertices (children or next level)
- continues until no unexplored vertex is left
- time complexity becomes $O(n^2)$ if adjacency list and adjacency matrix is used, else $O(n + e)$ - n is number of vertices, e is number of edges
- space complexity - $O(n)$
- Depth First: uses a stack (explicit stack or recursion) for unexplored nodes and an array of visited nodes
 - exploration of a vertex v is suspended as soon as a new vertex(child) is reached
 - exploration of the new vertex(child) u begins
 - search terminates when all reached vertices have been fully explored
 - same complexity as BFS
- Bi-connected:
 - Articulation Point: a vertex that if deleted, would break the graph into two or more pieces (Connected components)
 - i.e. A,G,H,J



- Bi-connected graph: has no articulation points; if any vertex is deleted the graph remains connected



- Bi-connected component: subgraph that is bi-connected

Backtracking

- <https://www.youtube.com/watch?v=DKCbsiDBN6c>
- one of the most general algorithm design techniques
- many searching and optimization problems satisfying some constraints can be solved with backtracking
- the problem is to find a vector, which maximizes or minimizes a criterion function $P(x_1, \dots, x_n)$
- Constraints:
 - Explicit constraints: rules that restrict each X_i to take values only from a given set; i.e. $X_i > 0$
 - Implicit constraint: determines which of the vectors actually satisfy the criterion functions
- Examples
 - N-Queens Problem: place n -queens in an N by N matrix in such a way that no two queens attack each (no two queens should be in the same row, column, diagonal)
 - https://www.youtube.com/watch?v=xFv_Hl4B83A
 - Sum of Subsets: find all the combination of numbers from a set of positive numbers that sum up to a number N

- <https://www.youtube.com/watch?v=kyLxTdsT8ws>
- Graph Coloring: color a graph G in such a way that no two adjacent nodes have the same color with a limited number of m (count) available colors
 - Problems:
 - m -coloring optimization: coloring a graph using the minimum number of coloring.
 - m -coloring decision(K -coloring): whether graph is colorable using at most k -colors
 - <https://www.youtube.com/watch?v=052VkKhlaQ4>
- Hamiltonian cycle: a round trip path along ' n ' edges of graph G which visits every vertex once and returns to its starting position
 - <https://www.youtube.com/watch?v=dQr4wZCiJJ4>
- Branch and Bound
 - https://www.youtube.com/watch?v=3RBNPc0_Q6g
 - for solving combinatorial optimization problems
 - Branching: Dividing the problem into smaller subproblems (like a decision tree).
 - Bounding: Estimating the best possible solution for each subproblem.
 - Pruning: Skipping branches that can't lead to a better solution than the current best.
- Knapsack Problem using Backtracking: similar to the (0/1) knapsack
 - https://www.youtube.com/watch?v=yV1d-b_NeK8
 - The Solution space is the same as that for the sum of subset's problem.
 - Bounding functions are needed to help kill some live nodes without expanding them
- Travelling Salesman Problem: minimization problem, shortest possible route that visits each city once and returns?

- solved using Branch and bound by generating state space tree by calculating cost and upper bound

Artificial Intelligence

AI: study of the mental faculties through the use of computational models

- concerned with the design of intelligence in an artificial device
- Intelligence tasks involving higher mental process; the computational part of the ability to achieve goals
 - creativity, solving problems, pattern recognition, classification, classification,
 - learning, induction, deduction, building analogies,
 - optimization language processing, knowledge and many more.
- Interpretations of Intelligence:
 1. Think human like: AI is about designing systems that are as intelligent as humans; emulate the human thought process; cognitive science approach to AI.
 - Cognitive science approach
 2. Act humanly: 'imitation game', interrogator would not know if it was interrogating an AI or a human (Turing test)
 - Turing test approach
 3. Think rationally: soundness and completeness of the inference mechanism of it's Logic (how the system arrives at a conclusion; reasoning behind its selection of actions)
 - Law of thought approach
 4. Act rationally: AI as the study of rational agents (how the system acts and performs, and not so much about the reasoning process)

- Rational agent approach
- Intelligent behavior: tasks and applications
 - Perception: image recognition, computer vision
 - Reasoning
 - Learning
 - Natural language processing, speech processing
 - Solving problems
 - Robotics
 - Thinking
 - Acting - in the complex environment
 - Knowledge – applying successfully in new situation
- Approaches to AI
 - Strong AI: build self-aware machines that can truly reason and solve problems
 - indistinguishable from that of a human being.
 - Weak AI: cannot truly reason and solve problems, but can act as if it were intelligent.
 - suitably programmed machines can simulate human cognition.
 - Applied AI: produce commercially viable smart system (i.e. recognize faces)
 - already enjoyed considerable success.
 - Cognitive AI: test theories about how the mind works on computers
 - solve abstract problems.
- AI Techniques
 - Act intelligent techniques
 - Describe and match

- Constraint satisfaction
- Generate and Test
- Goal reduction
- Tree search
- Rule based system
- Biology inspired
 - Neural Network
 - Reinforcement learning
 - Genetic Algorithms

Intelligent Agents

- AI system: composed of an agent and its environment.
 - Agent: can perceive it's environment (sensors) and act upon that environment (effectors)
 - Autonomous agent: decides autonomously which action to take in the current situation to maximize progress towards its goals
 - Agent = Architecture + Agent Program
 - Architecture: the machinery that an agent executes on
 - Agent Program: an implementation of an agent function
 - Autonomy: extent of an agent's behavior is determined by its own experience, rather than knowledge of designer
 - No autonomy: ignores environment/data
 - Complete autonomy: must act randomly/no program
 - Ideal: design agents to have some autonomy
 - Possibly become more autonomous with experience
 - environment may contain other agents.

- most famous artificial environment: the Turing Test environment
- Rationality: status of being reasonable, sensible, and having good sense of judgment
 - concerned with expected actions and results depending upon what the agent has perceived
 - Ideal Rational Agent: agent capable of doing expected actions to maximize its performance measure, on the basis of
 - Its percept sequences
 - Its built-in knowledge bases
 - rationality depends on:
 - performance measures, (determine the degree of success)
 - agent's Percept Sequence till now.
 - agent's posterior knowledge about the environment.
 - actions that the agent can carry out.
 - A rational agent always performs right action (right action is which makes agent most successful in the given percept sequence)
- Omniscience: state of possessing unlimited knowledge about all things possible
 - knows the actual effects of its actions
 - is impossible in real world
- Performance Measure of Agent: criteria determining how successful an agent is
 - subjective measure
 - in terms of:
 - speed or efficiency
 - accuracy or the quality of the solutions achieved
 - power usage, money, ...

- Behavior of Agent: action performed after any given sequence of percepts.
- Percept: perceptual inputs (complete set of inputs) at a given instance;
- Percept Sequence: history of all that the agent has perceived till date.
- Agent Function: map from the precept sequence to an action.
- Action: operation involving an effector
- Properties of Task environments:
 - Discrete / Continuous: whether there are a limited number of distinct, clearly defined, states of the environment or not
 - i.e. chess - discrete; continuous - driving
 - Observable / Partially Observable: whether it is possible to determine the complete state of the environment at each time point from the percepts (observable) or not (partially observable)
 - Static / Dynamic: whether environment changes while an agent is acting(dynamic) or not (static)
 - Single agent / Multiple agents
 - Accessible / Inaccessible – whether the agent's sensory apparatus can have access to the complete state of the environment or not.
 - Deterministic / Non-deterministic – If the next state of the environment is completely determined by the current state and the actions of the agent, then the environment is deterministic; otherwise it is non-deterministic.
 - Episodic / Non-episodic
 - episodic environment: each episode consists of the agent perceiving and then acting.
 - quality of its action depends just on the episode itself.
 - subsequent episodes do not depend on the actions in the previous episodes.
 - much simpler because the agent does not need to think ahead
- Elements to take account when designing an agent (PEAS):

- Performance: measure the success of an agents behavior
- Environment: where an agent operates
- Actuator: agent acts with in its environment
- Sensor: the agent senses its environment
- Agent Types
 - Simple reflex agent: simplest agents,
 - decisions on the basis of the current percept and ignore the rest of the percept history
 - these agents only succeed in the fully observable environment
 - works on Condition-action rule: maps the current state to action
 - very limited intelligence, not adaptive to changes in the environment
 - i.e. Room Cleaner agent
 - Model-based reflex agent
 - can work in a partially observable environment, and track the situation
 - factors:
 - Model: knowledge about "how things happen in the world,"
 - Internal State: a representation of the current state based on percept history
 - updating the agent state requires information about:
 - how the world evolves
 - how the agent's action affects the world
 - Goal-based agent: agent needs to know its goal which describes desirable situations
 - expand the capabilities of the model-based agent by having the "goal"
 - may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not (searching and planning)
 - Utility-based agent

- similar to the goal-based agent but provides utility measurement
- utility measurement provides a measure of success at a given state
- agent acts based not only goals but also the best way to achieve the goal
- useful when there are multiple possible alternatives and has to choose best action
- utility function maps each state to a real number (efficiency)
- Learning agent
 - learning allows an agent to operate in initially unknown environments, required for true autonomy
 - agent can learn from its past experiences; starts to act with basic knowledge, then it's able to act and adapt automatically
 - Conceptual components:
 - Learning element: for making improvements by learning from environment
 - Critic Learning element: takes feedback from critic (how well the agent is doing with respect to a fixed performance standard)
 - Performance element: for selecting external action
 - Problem generator: for suggesting actions that will lead to new and informative experiences.
 - learning element modifies the performance element

Problem Solving (Goal Based) Agents

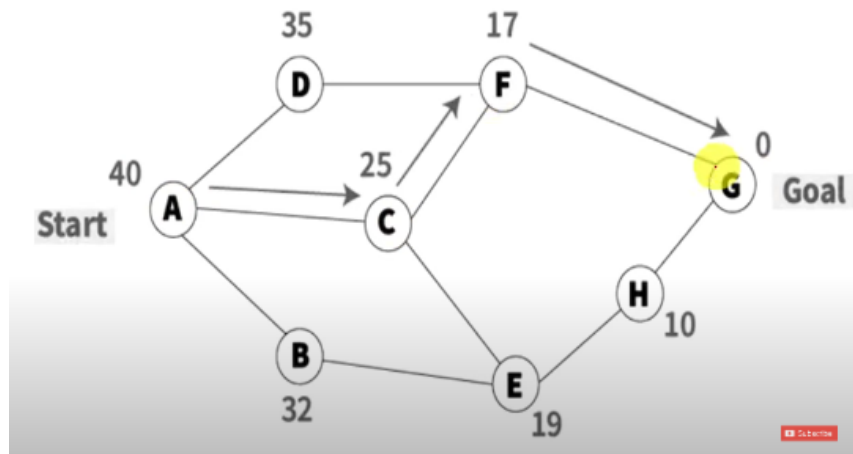
- Problem Solving by Searching
 - Searching: step by step procedure to solve a search-problem in a given search space.
 - Factors:

- Search Space: represents a set of possible solutions, which a system may have.
- Start State: a state from where agent begins the search.
- Goal test: function observing current state & if the goal state is achieved or not.
- Search tree: representation of search problem; root corresponds to the initial state.
- Actions: description of all the available actions to the agent.
- Transition model: description of what each action d
- Path Cost: It is a function which assigns a numeric cost to each path.
Solution: It is an action sequence which leads from the start node to the goal node.
Optimal Solution: If a solution has the lowest cost among all solutions.
- Search Strategies:
 - Uninformed Search Strategies (level order traversal): does not contain any domain knowledge such as closeness and the location of the goal
 - operates in a brute-force way
 - applies a way in which search tree is searched without any information about the search space (blind search)
 - examines each node of the tree until it achieves the goal node
 - DFS
 - requires very less memory
 - may go to the infinite loop
 - Complete within finite state space, non-optimal
 - Time Complexity - $O(n^m)$: m = maximum depth; Space complexity - $O(n)$
 - BFS

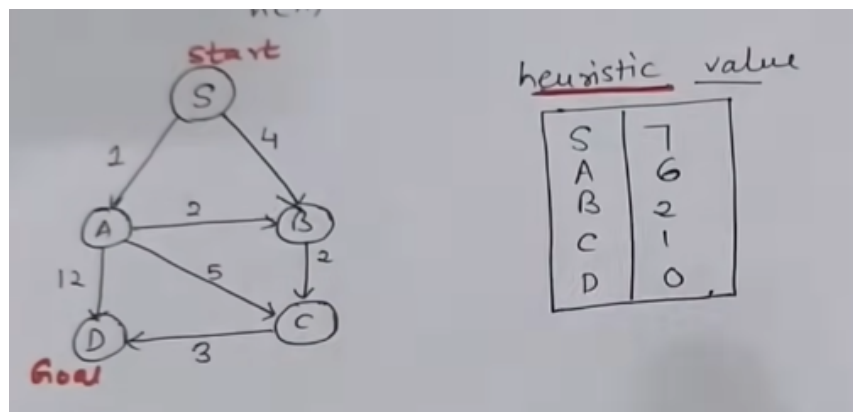
- will provide a solution if any solution exists
- provides the minimal solution which requires the least number of steps if there is more than one solution
- requires lots of memory since each level of the tree must be saved into memory
- needs lots of time if the solution is far away from the root node
- Complete since BFS will find the solution, optimal
- b - branching factor - **maximum number of successors (children)** any node can have (2 for binary tree)
- Time Complexity - $O(b^d)$: d = depth of shallowest solution, b = node at every state; Space complexity - $O(b^d)$
- Depth limited search
 - similar to DFS with a predetermined limit
 - Complete if the solution is above the depth-limit (else incomplete), may not be optimal for more than one solution
 - Time complexity - $O(b^l)$; Space complexity - $O(b * l)$
- Uniform cost search
 - for traversing a weighted tree or graph
 - find a path to the goal node which has the lowest cumulative cost
 - equivalent to BFS algorithm if the path cost of all edges is the same
 - Complete, optimal because at every state the path with the least cost is chosen
 - Time complexity:
 - b - branching factor - **maximum number of successors (children)** any node can have (2 for binary tree)

- **C*** is the cost of the optimal solution (i.e. least cost to reach goal = 10)
- ε is the minimum positive cost of any action (i.e. smallest costing edge = 2)
- **maximum number of steps**(i.e. edge traversals) to reach a cost of C*
 - $\lceil C^*/\varepsilon \rceil$
- node level: $\lceil C^*/\varepsilon \rceil + 1$ including root node
- $O(b^0 + b^1 + b^2 + \dots + b^{C^*/\varepsilon}) = O(b^{1+\lceil C^*/\varepsilon \rceil})$.
 - for initial node - $b^0 = 1$ node
 - for second level - $b^1 = b = 2$ for binary tree (2 nodes)
 - for third level - $b^2 = 2^2 = 4$ for binary tree (4 nodes) ... until maximum depth traversal $\lceil C^*/\varepsilon \rceil$
- Space complexity: same as Time complexity
- Iterative Deeping DFS
 - combination of DFS and BFS algorithms
 - finds out the best depth limit and does it by gradually increasing the limit until a goal is found
 - combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency
 - main drawback of IDDFS is that it repeats all the work of the previous phase
 - Complete if branching factor is finite, optimal if path cost is non-decreasing from depth
 - Time Complexity - $O(b^d)$: d = depth of shallowest solution, b = node at every state (branching factor); Space complexity - $O(b * d)$

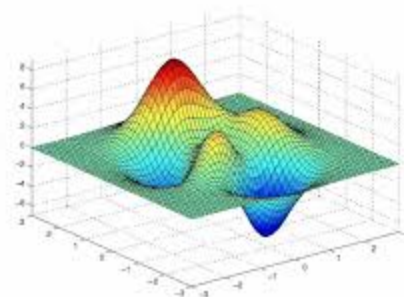
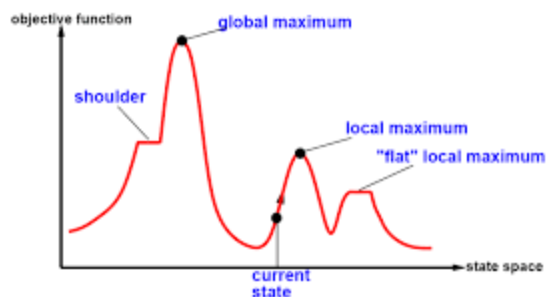
- Bidirectional search
 - runs two simultaneous searches, one starts the search from an initial vertex and other starts from goal vertex
 - search stops when these two graphs intersect each other
 - can use search techniques such as BFS, DFS, DLS, etc...
 - fast, requires less memory
 - implementation is difficult, goal state should be known in advance
 - Same complexity as the used technique, i.e. same as BFS if BFS was used bidirectionally
- Informed Search Strategies
 - Best first search
 - Greedy Best first search: always selects the path which appears best at that moment
 - <https://www.youtube.com/watch?v=GVvN0ikNekw>
 - combination of BFS and DFS
 - at each step, choose the most promising node (expand the node which is closest to the goal node estimated by heuristic function)
 - $h(n)$ = estimated cost from node n to the goal
 - more efficient than BFS and DFS algorithms
 - can behave as an unguided DFS in the worst-case scenario, and can get stuck in a loop as DFS
 - incomplete even if the given state space is finite, not optimal
 - same complexity as BFS in worst case



- A * search / A star search: most commonly known form of best-first search
 - <https://www.youtube.com/watch?v=Eiu1Cb-veCc>
 - combined features of UCS and greedy best-first search
 - similar to UCS except instead of $g(n)$, it uses $g(n)+h(n)$ = fitness number;
 - best algorithm than other search algorithms
 - does not always produce the shortest path as it mostly based on heuristics
 - complete, optimal
 - same complexity as BFS



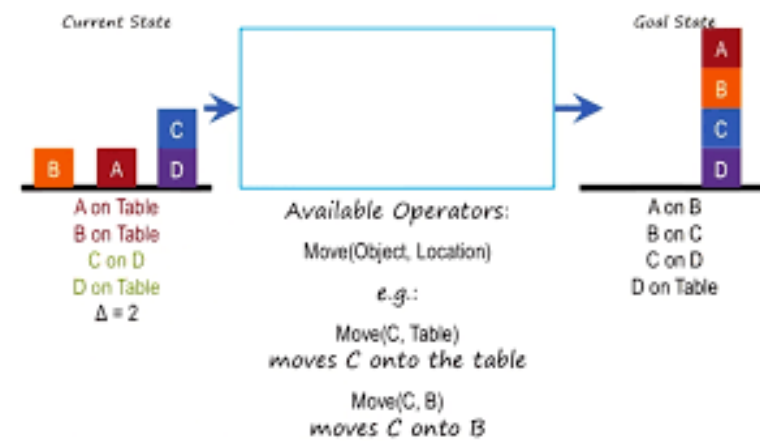
- Local Search Strategies: path to the goal is irrelevant; the goal state itself is the solution; widely used for very big problems; good but not optimal solutions; very memory efficient (1 or a few states)
 - Hill climbing algorithm (greedy local search):
 - moves in the direction of increasing value to find the peak (best solution);
 - terminates where no neighbor has a higher value;
 - used when a good heuristic is available;
 - only keeps a single current state (no need of search tree)
 - no backtracking and doesn't remember previous states
 - shoulder - It is a plateau region which has an uphill edge
 - state-space landscape: graph between various states and cost



- Types:
 1. Simple hill climbing algorithm: only evaluates the neighbor node state at a time; selects the first one

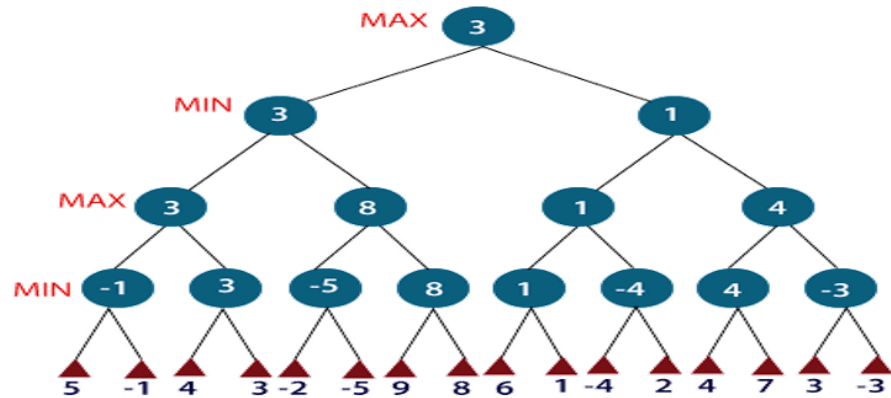
which optimizes current cost

2. Steepest-Ascent algorithm: examines all the neighboring nodes of the current state; select neighbor closest to the goal
 3. Stochastic hill climbing: selects one node at random and decides whether it should be expanded
- Means ends analysis: first solve the major part of a problem and then go back and solve the small problems
 - evaluates the difference between the current state and goal state



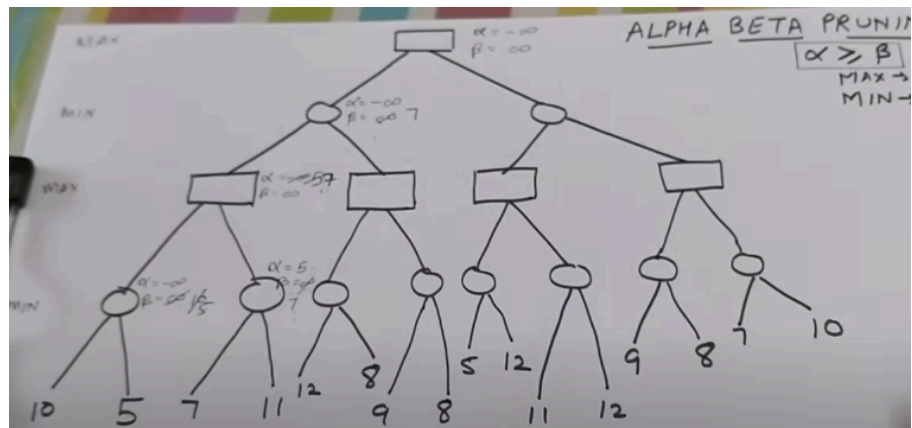
- Adversarial Search Strategies: examine problem when we try to plan ahead of the world and other agents are planning against us
 - <https://www.youtube.com/watch?v=l-hh51ncgDI>
 - two or more players with conflicting goals are trying to explore the same search space for the solution
1. Minimax algorithm: recursive or backtracking algorithm; used in decision making and game theory; DFS algorithm
 - two players play the game, one is MAX and other is MIN
 - MIN: Decrease the chances of MAX to win the game

- MAX: Increases his chances of winning the game
- game is played alternatively between them



Alpha Beta Pruning

- https://www.youtube.com/watch?v=_i-IZcbWkps
- modified version of the minimax algorithm
- Alpha: highest-value choice we have found so far at any point along the path of Maximizer (initial value = $-\infty$)
- Beta: lowest-value we have found so far at any point along the path of Minimizer (initial value = $+\infty$)
- Max player will only update the value of alpha; Min player will only update the value of beta
- main condition for alpha-beta pruning: $\alpha \geq \beta$



Knowledge Representation and Reasoning

- Knowledge based agent
 - agents who have the capability of maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions
 - intelligent agent needs knowledge about the real world for taking decisions and reasoning to act efficiently
 - Logical agent: agents with some representation of complex knowledge about the world or its environment
 - uses the inferences to derive new information from the knowledge combined with new inputs
 - Knowledge: a sentence in a knowledge representation language (formal language)
 - Levels of knowledge
 1. Knowledge level (1st level)
 - specify what the agent knows, and what the agent goals are
 2. Logical level
 - sentences are encoded into different logics
 3. Implementation level
 - agent performs actions as per logical and knowledge level
 - Knowledge Base: set of sentences in a formal language representing facts about the world.
 - central component of a knowledge-based agent
 - collection of sentences
 - Inference: deriving new sentences from old.
 - Inference system: allows us to add a new sentence to the knowledge base

- Inference system rules:
 - Forward chaining (facts to goals, data driven)
 - Backward chaining (goals to facts, goal driven)
- KB agent must be able to do
 - represent states, actions...
 - incorporate new percepts.
 - update the internal representation of the world.
 - deduce the internal representation of the world,
 - deduce appropriate actions.
- Approaches to design KB agents
 - Declarative approach: feed necessary information in an empty knowledge-based system
 - Procedural approach: knowledge stored in empty system in the form of program code
 - a successful agent can be built by combining both
- Knowledge Representation
 - Object: All the facts about objects in our world domain.
 - Events: the actions which occur in our world.
 - Performance: behavior which involves knowledge about how to do things.
 - Meta-knowledge: knowledge about what we know.
 - Facts: truths about the real world and what we represent.
- Techniques of Knowledge Representation
 - Logical Representation: drawing a conclusion based on various conditions
 - Logic Programming: language with some concrete rules which deals with propositions

- consists of precisely defined syntax and semantics
 - Syntax: constructing legal sentences
 - Semantics: rules to interpret the sentence
- Semantic Network Representation: alternative of predicate logic
 - represent knowledge in the form of graphical networks.
 - nodes - objects; arcs - relationship between those objects
 - it is a natural representation of knowledge; simple and easily understandable
 - takes more computational time at runtime to traverse the complete network
 - not possible to build a vast semantic network
- Frame Representation
 - Frame: a record/row for an object
 - Slot: attribute (column)
 - Facet: metadata about the slot (i.e. type)
 - makes the programming easier by grouping the related data
 - comparably flexible and used by many applications in AI
 - inference mechanism is not easy to process
- Production Rules
 - checks for the condition and if the condition exists then production rule fires and corresponding action is carried out
 - production rules are highly modular, so we can easily remove, add or modify an individual rule
 - does not exhibit any learning capabilities

Propositional Logic

- PL: the simplest form of logic where all the statements are made by propositions
 - proposition: a declarative statement which is either true or false
 - tautology: always true
 - contradiction: always false
 - contingency: has both true and false values
 - types:
 - Atomic Proposition: a single proposition symbol.
 - Compound proposition: constructed by combining simpler or atomic propositions, using parenthesis and logical connectives
 - PL consists of an object, relations or function, and logical connectives(operators)
 - Statements which are questions, commands, or opinions are not propositions
 - Logical connectives (mainly five):
 - Negation: $\neg P$
 - Conjunction: $P \wedge Q$
 - Disjunction: $P \vee Q$
 - Implication (if-then rules): $P \rightarrow Q$
 - Biconditional: $P \Leftrightarrow Q$
 - Precedence of connectives:
 - parenthesis $\rightarrow$ negation $\rightarrow$ conjunction $\rightarrow$ dis-junction $\rightarrow$ Implication $\rightarrow$ bi-implication
 - Inference in Propositional Logic:
 - generating the conclusions from evidence and facts
 - Inference rules: the templates for generating valid arguments; applied to derive proofs

- Proof: a sequence of the conclusion that leads to the desired goal
- 1. Modus Ponens: if P and $P \rightarrow Q$ is true, then we can infer that Q will be true
- 2. Modus Tollens: if $P \rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ will also true
- 3. Hypothetical Syllogism: if $P \rightarrow Q$ is true and $Q \rightarrow R$ is true, then $P \rightarrow R$ is true
- 4. Disjunctive Syllogism: if $P \vee Q$ is true, and $\neg P$ is true, then Q will be true
- 5. Addition: if P is true, then $P \vee Q$ will be true.
- 6. Simplification: if $P \wedge Q$ is true, then Q and P will also be true
- 7. Resolution: if $P \vee Q$ and $\neg P \wedge R$ is true, then $Q \vee R$ will also be true

Predicate (First-Order) Logic

- Inference in First-Order Logic: used to deduce new facts or sentences from existing sentences
- Terminologies
 - Substitution: performed on terms and formulas; occurs in all inference systems in FOL.
 - complex in the presence of quantifiers in FOL.
 - $F[a/x]$, so it refers to substitute a constant "a" in place of variable "x"
 - Equality-FOL: also uses equality (two terms refer to the same object) in addition to predicates and terms.
 - $\text{Brother}(\text{John}) = \text{Smith}$
 - Universal Generalization: valid inference rule, states that if premise $P(c)$ is true for any arbitrary element c in the universe of discourse, then $\forall x P(x)$ is true (for all elements)
 - Universal Instantiation: state that we can infer any sentence $P(c)$ by substituting a ground term c (a constant within domain x) from $\forall x P(x)$ for any object in the universe of discourse.

- can be applied multiple times to add new sentences; new KB is logically equivalent to the previous KB.
- IF "Every person like ice-cream" $\Rightarrow \forall x P(x)$ so we can infer that: "John likes ice-cream" $\Rightarrow P(c)$
- Existential introduction(existential generalization): If there is some elements C in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P
 - generalizing from a specific fact to a broad existential claim
 - Priyanka got good marks in English. Therefore, someone got good marks in English.
- Existential Instantiation (Existential Elimination): can be applied only once to replace the existential sentence; the new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.
 - states that one can infer $P(c)$ from the formula given in the form of $\exists x P(x)$ for a new constant symbol c - "There exists an x such that P(x) is true. Let's pick a specific (new) constant c that has property P"
 - i.e. $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$: There exists something that is a crown and is on John's head.
 - you don't know **what** that crown is, but you know **something** exists, so introduce a new name, say **K**, and write: $\text{Crown}(K) \wedge \text{OnHead}(K, \text{John})$ - Skolemization process.
 - K - Skolem constant
- Unification in FOL: process of making two different logical atomic expressions identical by finding a substitution.
 - The UNIFY algorithm is used for unification
 - i.e. two different expressions: $P(x, y), P(a, f(z))$
 - substitute x with a, and y with f(z) in the first expression, and it will be represented as a/x and $f(z)/y$
 - substitution set will be: $[a/x, f(z)/y]$

- Conditions for Unification:
 - Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified. Likes(Alice, x) and Loves(Alice, x) Cannot be unified (Likes $\neq$ Loves)
 - Number of Arguments in both expressions must be identical.
 - It will fail if there are two similar variables present in the same expression.
 - P(x) and P(f(x)) would require $x = f(x)$, which creates an infinite loop

Resolution in FOL

- Resolution: theorem proving technique; builds refutation proofs, i.e., proofs by contradictions.
 - used, if various statements are given, and need to prove a conclusion of those statements.
 - Unification is a key concept in proofs by resolutions.
 - can efficiently operate on the conjunctive normal form or clausal form.
 - Clause (a unit clause): Disjunction of literals (an atomic sentence).
 - Conjunctive Normal Form: conjunction of clauses.
- Steps for Resolution:
 1. Conversion of facts into first-order logic.
 2. Convert FOL statements into CNF
 3. Negate the statement which needs to prove (proof by contradiction)
 4. Draw resolution graph (unification)

Reasoning: mental process of deriving logical conclusion and making predictions from available knowledge, facts, and beliefs; it is

- a way to infer facts from existing data.

- general process of thinking rationally, to find valid conclusions.
1. Deductive reasoning: deducing new information from logically related known information.
 - the argument's conclusion must be true when the premises are true
 - top-down reasoning, and contradictory to inductive reasoning
 - mostly starts from the general premises to the specific conclusion
 2. Inductive reasoning: arrive at a conclusion using limited sets of facts by generalization.
 - starts with the series of specific facts or data and reaches to a general statement or conclusion.
 - also known as cause-effect reasoning or bottom-up reasoning
 3. Abductive reasoning: starts with single or multiple observations to find the most likely explanation or conclusion for the observation.
 - extension of deductive reasoning, but in abductive reasoning, the premises do not guarantee the conclusion.
 - example
 - Implication: Cricket ground is wet if it is raining
 - Axiom: Cricket ground is wet.
 - Conclusion It is raining
 4. Common Sense Reasoning: informal form of reasoning gained through experiences
 - relies on good judgment rather than exact logic and operates on heuristic knowledge and heuristic rules
 - i.e. If I put my hand in a fire, then it will burn
 5. Monotonic Reasoning: once the conclusion is taken, then it will remain the same even if we add some other information to existing information in our knowledge base

- not useful for the real-time systems, as in real time, facts get changed, so we cannot use monotonic reasoning
 - used in conventional reasoning systems, and a logic-based system is monotonic
 - any theorem proving is an example of monotonic reasoning
6. Non-monotonic Reasoning: some conclusions may be invalidated if we add some more information to KB.
- deals with incomplete and uncertain models
 - example:
 - Birds can fly
 - Penguins cannot fly
 - Pitty is a bird
 - from the above we can conclude that Pitty can fly
 - if we add one another sentence into knowledge base "Pitty is a penguin", which concludes "Pitty cannot fly", it invalidates the above conclusion.

Reasoning under uncertainty

- Uncertainty: unsure about predicates
 - $A \rightarrow B$, means if A is true then B is true, but say we don't know that A is true or not so we cannot express this statement, this situation is called uncertainty
 - Causes of uncertainty
 - Information occurred from unreliable sources.
 - Experimental Errors
 - Equipment fault
 - Temperature variation
 - Climate change.

- Probabilistic reasoning: a way of knowledge representation applying the concept of probability to indicate the uncertainty in knowledge; combine probability theory with logic.
 - Ways to solve problems with uncertain knowledge:
 - Bayes' rule: updates the probability of a hypothesis based on new evidence (posterior)
 - example: calculate the event A when event B has already occurred (the probability of A under the conditions of B)
 - when A is given:
 - $P(B/A) = P(A \cap B)/P(A)$
 - conditional probability = joint probability / marginal probability
 - when B is given:
 - $P(A/B) = P(A \cap B)/P(B)$
 - calculate the value of $P(B|A)$ with the knowledge of $P(A|B)$
 - $P(A \cap B) = P(A/B) * P(B)$
 - $P(A \cap B) = P(B/A) * P(A)$
 - $P(A/B) = P(A \cap B)/P(B) = P(B/A) * P(A) / P(B)$
 - posterior = likelihood * prior probability / marginal probability
 - Bayesian Statistics: full inference using Bayes' Rule

Expert System: computer applications developed to solve complex problems in a particular domain, at the level of extra-ordinary human intelligence and expertise

- Capabilities: Advising, Instructing and assisting human in decision making, Demonstrating, Deriving a solution, Diagnosing and Explaining, Interpreting input, Predicting results, Justifying the conclusion, Suggesting alternative options to a problem

- Not capable of: Substituting human decision makers, Producing accurate output for inadequate knowledge base, Refining their own knowledge
- Applications: Design Domain, Medical Domain (deduce cause of disease), Monitoring Systems, Process Control Systems, Knowledge Domain (finding faults), Finance/Commerce (detect fraud)
- Technologies:
 - Expert System Development Environment
 - Tools: editors and debugging tools; have Inbuilt definitions of model, knowledge representation, and inference design
 - Shells: Java Expert System Shell (JESS), Vidwan, Mumbai
- Benefits: Availability, Less Production Cost, Less Error Rate, Speed, Steady response
- Limitations: large systems require high cost and development time; difficult to maintain
- Components:
 - Knowledge bases: contains domain-specific and high-quality knowledge.
 - Components:
 - Factual Knowledge: widely accepted by Engineers in the domain,
 - Heuristic Knowledge: about practice, accurate judgement, evaluation, and guessing.
 - Knowledge Acquisition: formed by readings from various experts, scholars, and the Knowledge Engineers
 - Inference Engine:
 - In knowledge-based ES: the Inference Engine acquires and manipulates the knowledge from the knowledge base to arrive at a particular solution.
 - In rule-based ES, it applies rules repeatedly to the facts obtained from earlier rule application and adds new knowledge into the knowledge base if required.

- Methods:
 - Forward Chaining (data-driven): deduces outcome; "What can happen next?"
 - down-up approach; making a conclusion based on known facts or data;
 - starts from initial state and reaches the goals state
 - Backward Chaining (goal-driven): "Why this happened?"
 - uses DFS, goal is broken into sub-goals to prove the facts true.
 - based on modus ponens inference rule
- User Interface: provides interaction between user and the ES
 - forms of explanations: natural language; verbal narrations; rule numbers
- Development (iterative)
 - Identify Problem Domain: suitable for an expert system? cost-effectiveness of the system?
 - Design the System: ES Technology, identify integration with other systems and databases
 - Develop the Prototype: acquire domain knowledge from expert & represent in IF-ELSE
 - Test and Refine the Prototype: by knowledge engineer and end users
 - Develop and Complete the ES: ensure interaction b/n ES & its environment; document
 - Maintain the System: regular review & update; evolve

Learning Agents

- Learning: changes in a system enabling more efficiency next time.
- computer learns if its performance at tasks in $\underline{T}$, as measured by $\underline{P}$, improves with experience $\underline{E}$

- Components: Learning Element, Performance Element, Critic, Problem Generator
- Types of learning:
 - Supervised learning: we are given a set of (x, y) pairs by a "teacher." (discover the relation)
 - Classification
 - Regression
 - Unsupervised learning: we are only given the x 's (discover outputs by itself)
 - Clustering
 - Association
 - Reinforcement learning: feedback-based (reward for right action, penalty for wrong action)

Decision tree algorithm: classification and prediction tool having a tree like structure,

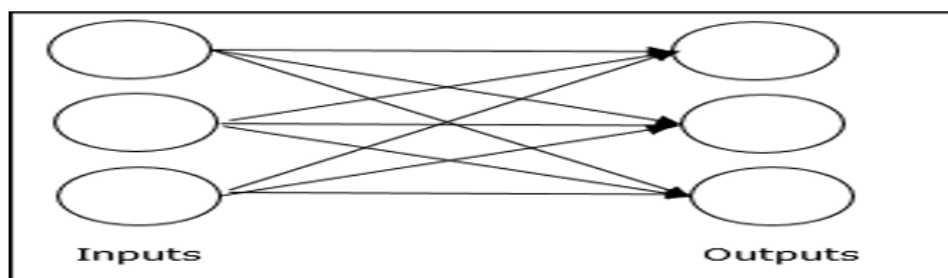
- each internal node denotes a test on an attribute
- each branch represents an outcome of the test
- each leaf node (terminal node) holds a class label
- entropy: randomness in the information being processed; higher = harder to draw conclusions
- information gain: info gained about a random variable from observing another random variable
 - difference between the entropy of parent node and weighted average entropy of child nodes
- Gini impurity: how often a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset.

- Algorithms to build a decision tree:
 - CART (Classification and Regression Trees): Gini impurity as metric.
 - ID3 (Iterative Dichotomiser 3): entropy and information gain as metric.

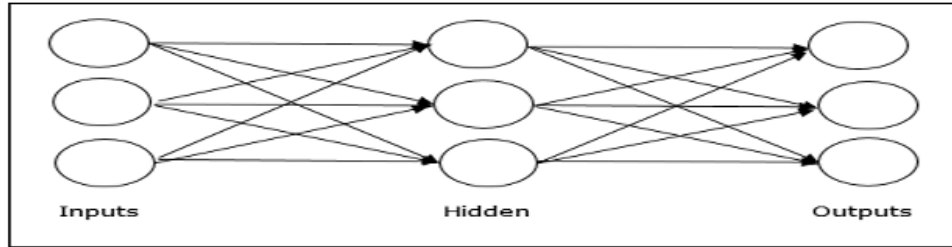
Regression algorithm: identifying and evaluating the relationship between a dependent variable and one or more independent variables (characteristics measured directly; aka predictor or explanatory variables)

Neural Networks: approximate a wide range of complex functions representing multi-dimensional input-output maps

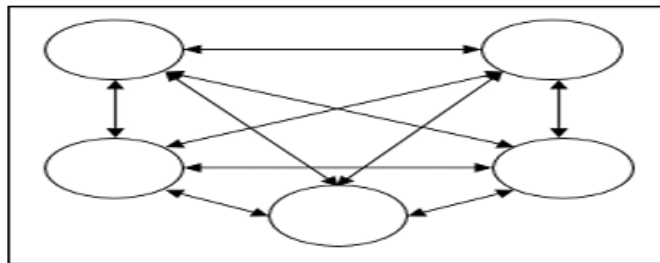
- composed of large number of highly interconnected simple processing elements (neurons)
- "expert" in the category of information it has been given to analyse
- used for projections given new situations of interest and answer "what if" questions
- Advantages: Adaptive learning, Self-Organisation, Real Time Operation (parallel computation), Fault Tolerance via Redundant Information Coding (reduce network without loss of capabilities)
- Network topology:
 - Feedforward Network: non-recurrent, signal can only flow from input to output
 - Single layer feedforward network: only one weighted layer



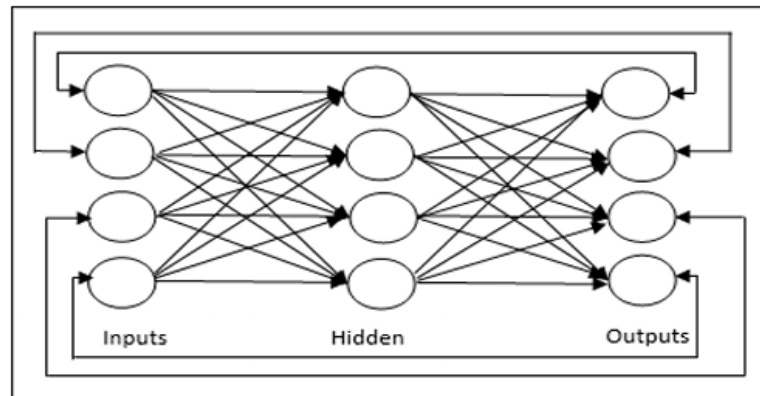
- Multilayer feedforward network: one or more hidden layers b/n input and output layer



- Feedback network: has feedback paths, non-linear dynamic system
 - Fully recurrent network: all nodes are connected to all other nodes, each node works as both input and output



- Jordan network: closed loop network; output will go to the input again



- Activation function: the extra force or effort applied over the input to obtain an exact output
 - Linear Activation
 - Sigmoid Activation
 - Binary Sigmoid (Logistic Function) - binary classification, range: (0, 1)

- Bipolar Sigmoid (Tanh Function) - when negative values are helpful, range: $(-1, 1)$

Communicating, Perceiving, and Acting

- Natural Language Processing (NLP): used by machines to understand, analyze, manipulate, and interpret human's languages
 - Advantages: users ask about any subject, time efficient, efficient documentation
 - Disadvantages: may not show context, unpredictable
 - Components:
 - Natural Language Understanding (NLU): extracts metadata (concepts, entities, emotion, relations, semantic rules) for the machine to understand and analyses human language
 - Natural Language Generation (NLG): translator converting the computerized data into natural language representation (generates language)
 - Phases:
 - Lexical Analysis and Morphological: converts it into meaningful lexemes
 - Syntactic Analysis (Parsing): check grammar, word arrangements and relationships
 - Semantic Analysis: meaning representation
 - Discourse Integration: depends upon the sentences that proceeds it and also invokes the meaning of the sentences that follow it
 - Pragmatic Analysis: discover the intended effect by applying a set of rules that characterize cooperative dialogues
 - Difficulty of NLP: Ambiguity and Uncertainty exist in the language
 - Lexical Ambiguity: two or more possible meanings within a single word
 - i.e. Manya is looking for a match: match = partner or match = game

- Syntactic Ambiguity: two or more possible meanings within the sentence
 - i.e. I saw the girl with the binocular; do i have the binocular? or does the girl have it?
- Referential Ambiguity: when referring to something using a pronoun
 - i.e. Kiran went to Sunita. She said, "I'm hungry."; is it kiran or sunita the hungry one?
- Applications: Machine Translation, Spam detection, Sentiment Analysis, Spelling correction, Speech Recognition, Chatbot, Information extraction

Robotics: domain in AI that deals with the study of creating intelligent and efficient robots.

- Robots: artificial agents acting in real world environment.
 - aimed at manipulating the objects by perceiving, picking, moving, modifying the physical properties of object, destroying it, or to have an effect
 - free manpower from repetitive functions without getting bored, distracted, or exhausted
- Locomotion: mechanism that makes a robot capable of moving in its environment
 - Legged Locomotion: demonstrates walk, jump, trot, hop, climb up or down
 - consumes more power, more motors, difficult to implement (stability issues)
 - Wheeled Locomotion
 - Standard wheel: Rotates around the wheel axle and around the contact
 - Castor wheel: Rotates around the wheel axle and the offset steering joint
 - Swedish 45o and Swedish 90o wheels: Omni-wheel, rotates around the contact point, around the wheel axle, and around the rollers

- Ball or spherical wheel: Omnidirectional wheel, difficult to implement
 - Slip/Skid Locomotion: use tracks as in a tank
- Components of robotics:
 - Power Supply, Actuators, Electric motors (rotational movement), Pneumatic Air Muscles (contract 40%), Muscle Wires (contract 5%), Piezo Motors and Ultrasonic Motors (best for industrial robots), Sensors
- Computer vision: technology of AI with which the robots can see; in domains of safety, security, health, access, and entertainment
 - Computer vision tasks:
 - OCR (Optical Character Reader): software converts scanned documents to editable text.
 - Face Detection: to let a user access the software on correct match.
 - Object Recognition
 - Estimating Position: estimate position of object with respect to camera
 - Application of computer vision: Industries, Military, Medicine, Exploration, Entertainment

Web Programming

World Wide Web (WWW) or the Web: a network of computers all over the world

- Web pages are files stored on computers called Web servers
- Computers reading the Web pages are called Web clients
- Web hosting: the place where all the files of your website live
 - Web host (web hosting service provider): a business providing the technologies and services needed for the website or webpage to be viewed in the Internet
 - Domain name registration: reserving a name on the Internet for a certain period

HTML (HyperText Markup Language) - since 1990 - the most basic building block of the Web.

- not a programming language
- it is a markup language that tells web browsers how to structure the web page
- defines the meaning and structure of web content
- uses CSS to describe appearance/presentation and/or JavaScript for functionality/ behavior
- HTML tag: used for creating an element; delimits the start and end of elements in the markup.
 - HTML Attributes: special words used to control the behavior of an element
 - Meta elements: tags for providing structured metadata about a web page

```
<meta charset="utf-8">
```

- HTML Links (Hyperlinks): allow us to link documents to other documents or resources, links to specific parts of documents

```
<a href="https://www.google.com">Google</a>.
```

- Text Formatting tags: Headings (predefined formats for presentation), Paragraph
- Image inserting tag:
 - : void element requiring at least one attribute to be useful, (src)
 - image map: a list of coordinates relating to a specific image (clickable links)

```
<map name="primary">
  <area shape="circle" coords="75,75,75" href="left.html">
  <area shape="circle" coords="275,75,75" href="right.html">
</map>

```

- HTML Table: for structuring tabular data

```

<table>
  <caption align="center" valign="bottom">table 1.0</caption>
  <tr>
    <th>Column 1</th>
    <th>Column 2</th>
  </tr>
  <tr>
    <td>Cell 1</td>
    <td>Cell 2</td>
  </tr>
  <tr>
    <td>Cell 3</td>
    <td>Cell 4</td>
  </tr>
</table>

```

- Ordered (ol) and Unordered List (ul)

```

<ul type="square">
  <li>book</li>
  <li>marker</li>
  <li>chalk</li>
</ul>
<ol>
  <li>...</li>
  <li>...</li>...
</ol>
: type="number type" {1, i, I, a, A}

<dl>
  <dt>...</dt>
  <dd>...</dd>
  <dt>...</dt>
  <dd>...</dd>
</dl>

```

- HTML Frames: present documents in multiple views, (independent windows or sub-windows)
 - Frame Set: specifies the layout of views in the main user agent window
 - contain a NOFRAMES element to provide alternate content for user agents that do not support frames or are configured not to display frames.
 - attributes: Cols, Rows, Border
 - Internal Frame: define the particular area within an HTML file where another HTML web page can be displayed


```

<!DOCTYPE html>
<html>
<head>
  <title>Frame tag</title>
</head>
<frameset cols="25%,50%,25%">
  <frame src="frame1.html" >
  <frame src="frame2.html">
  <frame src="frame3.html">
</frameset>
</html>

```

- HTML Form and Form Controls: interactive controls for submitting information.
 - attributes: action (backend script), method, target (_blank, _self, _parent), enctype (application/x-www-form-urlencoded - common, mutlipart/form-data - binary data)
- Inserting Multimedia: to add(embed) images or videos
 - <embed>: embed a video, audio and gif files to website, but the browser doesn't know media type ahead of time

```

<embed src = "video" width = "100%" height = "60" >
  <noembed>
    <img src = "image" alt = "Alternative Media" >
  </noembed>
</embed>

```

- <video>: allow to embed a video

```

<video src="video location" controls>
  <p>fallback text </p>
</video>

```

- <audio>: embeds audio

```

<audio controls>
  <source src="audio source" type="type of audio like audio/mp3">
  <source src="audio source" type="type of audio like ">
  <p> if the browser doesn't support this feature it will fallback to this

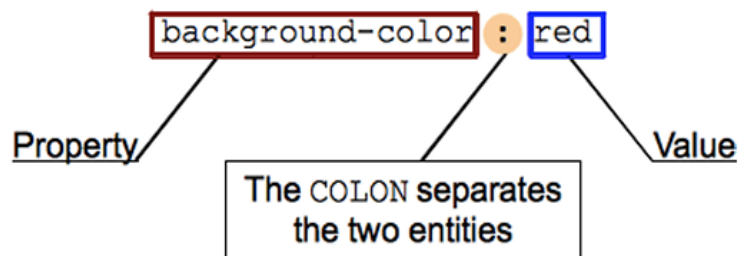
```

- HTML Graphics: visual representations; HTML Canvas (2D or 3D scene - animation, game), HTML SVG (<svg> to embed an SVG fragment)

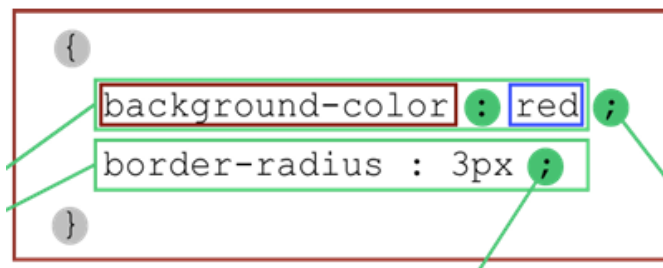
```
<svg width="600"
    height="150">
  <rect width="600"
    height="150"
    fill="green"
    stroke="black"
    stroke-width="6" />
</svg>
```

Cascading Style Sheets (CSS)

- CSS: used to selectively style HTML elements
- building block of CSS syntax: property (identifier) & value (how the feature must be handled by the engine)
- CSS Declarations: sets CSS properties to specific values



- Declaration Block: groups declarations delimited by a brace '{'



- Attaching CSS with HTML:

- Inline stylesheet: specified for an individual element via style attribute

```
<p style="color: red; background:green;">Inline style</p>
```

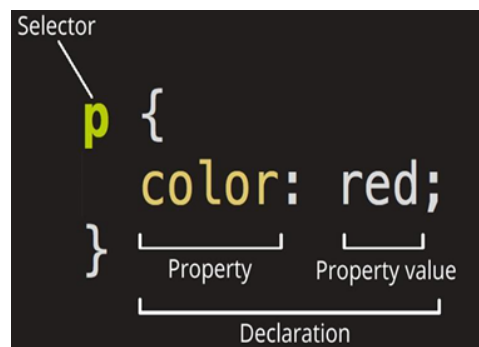
- Embedded stylesheet: used when a single document has a unique style

```
<head>
  <style>
    H1 {
      color:yellow;
      Text-align:left;
    }
    P{
      margin-left:20px;
    }
  </style>
</head>
```

- External style sheets: used when the style is applied to many pages;
<link> tag inside the head section

```
<link rel="stylesheet" href="style.css">
```

- Style Sheet Rules: selector group (HTML element name) and an associated declarations block



- Ruleset: whole structure
- Selectors: Type selector (by tag i.e. img), Class selector (multiple elements with specified class in their class attribute), Id selector (element with specified

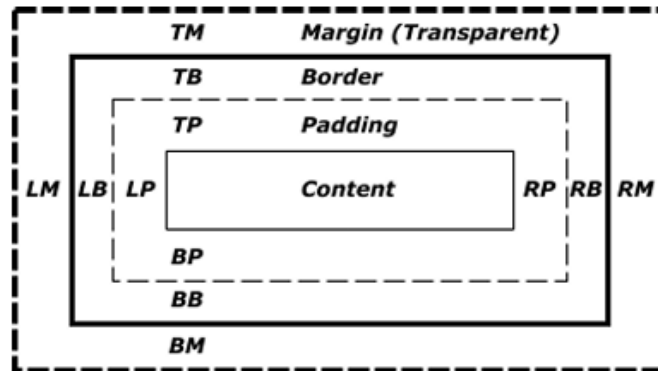
ID in id attribute), Attribute Selector (elements having the specified attribute), Pseudo-class selector (depends on state - i.e. hover)

- Style Inheritance: uses the document tree for inheritance
 - Specificity: how the browser decides which rule applies if multiple rules have different selectors, but could still apply to the same element
 - Cascade: when two rules apply that have equal specificity, the one that comes last in the CSS is the one that will be used

```
p {  
    color: brown;  
}  
p {  
    color: green;  
}
```

- Style Properties
 - Foreground and Background Properties:
 - background shorthand CSS property sets all background style properties at once (color, image, origin and size, or repeat method) - can be image
 - color property sets foreground
 - Font and Text Properties:
 - font CSS shorthand property sets all the different properties (font-family, font-size, font-stretch, font-style, font-variant, font-weight)
 - Text Properties: text-transform(none, uppercase, lowercase, capitalize), text-decoration (none, underline, overline, line-through), text-align(left, right, center, justify)
 - List specific styles: list-style-types(type of bullets), list-style-position(start, inside, outside), list-style-image(custom image for bullet)
 - CSS Box Model: everything in CSS has a box around it (block boxes and inline boxes)
 - Parts of a box

- Content box: where your content is displayed, sized using width and height.
- Padding box: padding sits around the content as white space
- Border box: wraps the content and any padding
- Margin box: outermost layer



- Display CSS property: sets whether an element is treated as a block or inline element
 - block, inline, inline-block, flex, inline-flex, grid, inline-grid, etc
- Table Styling Properties: table-layout(columns), border-collapse(prevent spacing between borders)
- Layout and Positioning Properties: where elements are positioned relative to their default position in normal layout flow, the other elements around them, their parent container, and the main viewport/window
 - Main layouts:
 - normal flow,
 - flexbox(one-dimensional layout - main axis & cross axis),
 - grid(divides page into regions - grid-template-columns # of cols, grid-template-rows # of rows),
 - floats, table layout, multiple-column layout
 - Positioning:

- Static Positioning (position: static;): default, normal position in the document flow
- Relative Positioning (position: relative;): once the positioned element has taken its place in the normal flow, you can then modify its final position, including making it overlap other elements on the page; top, bottom, left, and right are used
- Absolute Positioning (position: absolute;): no longer exists in the normal document flow; isolated UI that doesn't interfere with the layout of other elements
 - we have to set its container to position relative, or else it uses the documents body as relative
- CSS Measuring Units
 - absolute units: cm, mm, in, pt, px
 - relative units: em (parent's font-size), rem (root element's font-size), vw (1% viewport width), vh (1% viewport height)

Client-Side Scripting (JavaScript)

- JavaScript: a lightweight, interpreted or compiled programming language.
 - single threaded, dynamic, supporting object oriented, functional programming styles.
 - we can use JavaScript in coordination with Application programming interfaces (APIs)
 - Internal JavaScript: one way of including JavaScript with script tag in our HTML document.

```
<script>
  console.log("Hello world")
</script>
```

- We can also include JavaScript as an external file.

- defer attribute (Boolean): indicates to a browser that the script is meant to be executed after the document has been parsed, but before firing DOMContentLoaded. (has no effect if src attribute is not present)

```
<script src="script.js" defer></script>
```

- JavaScript Input Output: `var name = prompt("Enter your name: ");`
`alert("Hello " + name);`
 - `console.log` (in the browser console), `alert`(popup), `document.write` (deletes html and writes on it)
- JavaScript Data types: Number (integers and floats), String (single quote, double quote, back tick - to embed variables and expressions into a string), Boolean, Objects (key-value pair), Arrays (regular objects; store indexed values)
- Variable declaration: `let` (block level scope), `var` (hoisted, function level scope), `const`
- Data Type Conversion: `String(value)` - converts to string, `Number(value)` - converts to number, `Boolean(value)`
- Arithmetic operators: `+`, `-`, `*` (multiplication), `/` (division), `%` (remainder), `**` (exponent)
- Logical Operators: `&&` (logical AND), `||` (logical OR), `!` (logical NOT)
- Control Structures (Conditional and Looping Statements):
 - if-else, ternary operators, loops (while, do-while, for)

```
let age = 20;

if (age < 18) {
    alert( 'Under...' );
} else if (age > 18) {
    alert( 'Over' );
} else {
    alert( '18!' );
}

while (condition) {
    // "loop body"
}

do {
    // "loop body"
} while (condition);

for (begin; condition; step) {
    // ... loop body ...
}

let result = condition ? value1 : value2;
```

- Array: store ordered collections; syntax - using new or [listing elements]
 - Methods: Push (append element), Pop (takes an element from the end), Shift (Extracts the first element of the array and returns it), Unshift (add element to the beginning of the array)
 - Functions: building blocks of a program; syntax - function declaration, arrow expression

```
function showCSMessage() {
    alert( 'Hello CS Students!' );
}

        param => expression
    (param1, paramN) => expression
```

- JavaScript DOM (Document object Model): data representation of the objects in the structure and content of the web
 - Accessing HTML elements in JavaScript:
 - getElementById(id), getElementsByClassName(class), querySelectorAll(name), element = document.**querySelector**(selectors)
 - CSS in JavaScript: element.**style**.backgroundColor = "red"
 - Events: a signal that something has happened;
 - element.**addEventListener**(event, handler, [options]);
 - options: once (removes listener), capture (bubbling), passive (won't call preventDefault)
 - element.**addEventListener**("click", handler, [options])
- Handling Exception: try {...} catch {...}
- Form Processing: intercept submit event, form.addEventListener("submit", event ⇒ {...})
- JavaScript BOM (Browser Object Model): to interact with the browser

- JavaScript Window: supported by all browsers, all variables, functions and js objects are members of window
 - window.**document**.getElementById("header"), window.**innerHeight**
 - window.**open()**: open a new window, window.**close()**: close the current window
 - window.**moveTo()**: move current window, window.**resizeTo()**: resize current window
- Window location: represents the location (URL)
 - window.location.**href**: URL of the current page
 - window.location.**hostname**: domain name of the web host
 - window.location.**pathname**: path and filename of the current page
 - window.location.**protocol**: web protocol used (http: or https:)
 - window.location.**assign()**: loads a new document
- JavaScript Cookies: data, stored in small text files, on your computer to remember information about the user
 - saved in name-value pairs
 - when a browser requests a page, cookies of that page are added to the request
 - document.cookie = "username=John Doe"
 - can add an expiry date (in UTC time). default - cookie is deleted when browser is closed
 - document.cookie = "username=John Doe; expires=Thu, 18 Dec 2013 12:00:00 UTC";
 - with a path parameter, you can tell the browser what path the cookie belongs to
 - document.cookie = "username=John Doe; path=/"

Server-Side Scripting (PHP)

- PHP: a server-side scripting language; can be embedded in HTML or used as a standalone binary
 - stands for Hypertext Preprocessor; originally named Personal Home Page Tools,
 - used to generate dynamic output (potentially different each time requested)
 - mostly about connecting Web sites to back-end servers, such as databases.
 - two-way communication: Server to client (assemble pages from back end), Client to server (process entered information)
 - by default PHP documents end with the extension .php
 - syntax

Short-open (SGML-style) tags

```
<?php
//php script
?>
```

Short or short-open tags look like this:

```
<?
//php script
?>
```

- Predefined and User Variables: a variable does not need to be declared before adding a value to it
 - \$txt = "Hello World!"; \$number = 10;
 - \$GLOBALS: reference to every variable in global scope of script.
 - \$_SERVER: array containing information such as headers, paths, and script locations.
 - entries in this array are created by the web server.
 - \$_GET: associative array of variables passed via the HTTP GET method.

- \$_POST: associative array of variables passed via the HTTP POST method.
- PHP Output Statements: for printing to output (browser window); echo and print - can be used with or without parentheses
 - echo "Hey user!"; echo("Welcome back!");
 - multiple arguments by comma: echo "Welcome to ", "our exam!";
 - only with echo and without parenthesis
- Data Types and Variables:
 - PHP is a very loosely typed language: no need to declare variables
 - PHP always converts variables to the type required by their context when accessed
- Arithmetic and Logical Operators: to perform operations on the variables
- Conditional Statements: if - elseif -else, switch, ternary

```

<?php
$t = date("H");

if ($t < "10") {
    echo "Have a good morning!";
} elseif ($t < "20") {
    echo "Have a good day!";
} else {
    echo "Have a good night!";
}
?>

switch (n) {
    case label1:
        code to be executed if n=label1;
        break;
    case label2:
        code to be executed if n=label2;
        break;
    case label3:
        code to be executed if n=label3;
        break;
    ...
    default:
        code to be executed if n is different
}

(condition) ? if TRUE execute this : otherwise execute this;

```

- Loop Statements: while, do-while, for, foreach

```

<?php
    for($y = 0; $y <= 10; $y++){
        echo "The value of Y is: $x <br>";
    }
    while (if the condition is true) {
        // code is executed
    }

    $colors = array("red", "green", "blue", "yellow");

    foreach ($colors as $x) {
        echo "$x <br>";
    }
}

<?php
    $y = 1;
    do {
        echo "The value of Y is: $y <br>";
        $y++;
    } while ($y <= 5);
}
?>

```

- Arrays in PHP: ordered map,
 - map can be treated as an array, list (vector), hash table, dictionary, collection, stack, queue, etc..
 - array syntax: using the array() construct, with [], with [index]

```

<?php
    $array1 = array("Item 1", "Item 2", "Item 3", "Item 4");

    $array2 = array('item1' => "Item 1",
                    'item2' => "Item 2",
                    'item3' => "Item 3",
                    'item4' => "Item 4");

    ?>

    <?php
        $sample[] = "sample 1";
        $sample [] = "sample 2";
        $sample [] = "sample 3";
        $sample [] = "sample 4";

        print_r($sample);
    ?>

```

- PHP Functions: function info(\$name,\$age,\$salary){ echo }; info('Natan', 21, 0);
- Form Processing using PHP: isset (whether the variable in a form has value or not)

```

<?php
if (isset($_POST['submit']))
{
    if (!isset($_POST['firstname']))
    {
        $error = "*" . "Please fill all the required fields";
    }
    else
    {
        $firstname = $_POST['firstname'];
    }
}
?>

```

- PHP File Upload: initially files are uploaded into a temporary directory and then relocated to a target destination
 - Configure The "php.ini" File to allow file uploads
 - form must use method='POST', and enctype='multipart/form-data'
 - extract file from \$_FILES: \$_FILES["fileToUpload"]
- PHP Cookies and Session:
 - PHP session variable: to store information about, or change settings for a user session.
 - holds information about one single user, available to all pages in one application.

```

<?php
session_start();
if(isset($_SESSION['views']))
$_SESSION['views']=$_SESSION['views']+1;
else

$_SESSION['views']=1;
echo "Views=". $_SESSION['views'];

```

Destroying a Session

```

<?php
    unset($_SESSION['views']);
?>

```

completely destroy the session

```

<?php
session_destroy();
?>

```

- cookie is often used to identify a user: setcookie(name, value, expire, path, domain);

- `setcookie("user", "Alex Porter", time()+3600);`
- Database Programming using PHP
 - https://www.w3schools.com/php/php_mysql_connect.asp
 - Creating Database Connection: `$link=mysql_connect($hostname, $user, $password)`
 - Selecting the database: `$db=mysql_select_db("dbname", $link);`
 - Sending Query: `$result = mysql_query("query", $link);`
 - Processing Query:
 - `while($row=mysql_fetch_row($result)){ echo "Name: " . $row[1] }`
 - `while($row=mysql_fetch_object($result)){ echo "Name: " . $row->name ; }`
 - `while($row=mysql_fetch_object($result)){ echo "Name: " . $row{'name'} }`
 - Closing the connection: `mysql_close($link)`
- Input-Output:
 - `fopen & fclose: $file=fopen("hello.txt","r"); fclose($file);`
 - modes: **r** (read only, starts at beginning of file), **r+** (read/write, starts at beginning of file), **w** (write only, open/create and clear file), **w+** (write only, open/create and clear file)
 - `readfile():` reads a file and writes it to the output buffer
 - `echo readfile("webdictionary.txt")`
- PHP Date and Time:
 - `DateTime` class handles dates themselves;
 - `$date = new DateTime('2025-06-10 14:00:00');`
 - `$date = new DateTime();` - now
 - `DateTimeZone` class handles time zones;
 - `$addisAbabaTimezone = new DateTimeZone('Africa/Addis_Ababa');`

- `$date = new DateTime('2017-06-10 14:00:00', $addisAbabaTimezone);`
 - `$date = new DateTime('now', $addisAbabaTimezone);` - now
- DateTimeInterval class handles spans of time between two DateTime instances;
 - `$startPeriod = new DateTime('2025-01-01');`
 - `$intervalPeriod = new DateInterval('P1W');` - Every 1 week
 - `$endPeriod = new DateTime('2025-02-10');`
- DatePeriod: a set of dates and times, recurring at fixed intervals, between a start and an end date
 - `$period = new DatePeriod($startPeriod, $intervalPeriod, $endPeriod);`
 - `foreach ($period as $date) { $date is a datetime object }`
- PHP Mathematical Functions:
 - abs (Absolute value), acos (Arc cosine), acosh (Inverse hyperbolic cosine), asin (Arc sine), asinh (Inverse hyperbolic sine), atan2 (Arc tangent of two variables), atan (Arc tangent), atanh (Inverse hyperbolic tangent), `sin(radian value)`, `cos(M_PI / 4)`, `tan(radian value)`
 - `echo abs(-4.2); // 4.2` (double/float), `echo abs(5); // 5` (integer)
- PHP OOP:
 - Creating Objects: `$apple = new Fruit;` `$apple = new Fruit("apple", "20g");`
 - Accessing Properties and methods:
 - `$object→property_name`
 - `$object→method_name([arg, ... ])`
 - Declaring a Class

```

class Fruit
{
    public $weight = '';

    function setWeight($newWeight)
    {
        $this->weight = $newWeight;
    }
}

```

Advanced JavaScript and XML (AJAX)

- AJAX (Asynchronous JavaScript and XML): web development technique for creating interactive web applications;
 - send and receive information in formats: JSON, XML, HTML, and text files.
 - "asynchronous" nature: communicate with the server without having to refresh the page.
- XMLHttpRequest Object: used to interact with servers; retrieve any type of data, not just XML
 - `var httpRequest = new XMLHttpRequest();`
 - `httpRequest.open('GET', 'http://www.example.org/some.file', true);`
 - `httpRequest.send();`
- Handling Response from Server: write a callback function to handle response before making a request
 - `httpRequest.onreadystatechange = nameOfTheFunction;`

Web Development Frameworks

- Bootstrap: free front end framework for faster and easier web development; collection for creating responsive websites
- Node.js: cross platform runtime environment for creating server-side tools and apps in JS
 - `const server = http.createServer((req, res) => { ... })`

- `server.listen(port, () => { console.log(`Server running on port: ${port}`) })`
- Angular.js: component-based development platform built on TypeScript; collection of well-integrated libraries i.e. routing; a suite of developer tools
 - Install: `npm install -g @angular/cli`
 - Create project: `ng new <name> --routing=false --style=css`
 - Build and serve: `ng serve`
- React.js: front-end JavaScript library for building user interfaces based on UI components
 - Create project: `npx create-react-app <name>`

Operating Systems

Operating system: intermediary program between a computer user and the computer hardware

- Time multiplexing: sharing CPU, printer...
- Resource multiplexing: sharing main memory...
- Resource allocator: manages and allocates resources.
- Control program: controls the execution of user programs and operations of I/O devices.
- Layers:
 - Inner layer, computer hardware
 - Middle layer, operating system
 - Outer layer, different software
- Main functions:
 - Multiprogramming, multiprocessor, Computer resource management
 - Provides a user interface, Runs software utilities and programs
 - Schedule jobs, Enforce security measures

- Provide tools to configure the operating system and hardware
- Administers user actions and accounts, Maintain system integrity, handle failures
- Give users and processes an interface, Provide convenient persistent storage (files)
- Features:
 - Authentication of users: password, passphrase comparison, biometrics, digital authentication (SSL, CA, PKI, Kerberos, DS)
 - Mandatory Access Control (multilevel security): classifying data and users into various security classes
 - Discretionary Access Control: grant privileges to users
 - Protection of memory: user space, paging, segmentations
 - File and I/O device access control: access control matrix
 - Enforcement of sharing resources: preserve integrity, consistency (critical section)
 - Fair service: no starvation and deadlock
 - Inter-process communication & synchronization: shared variable (e.g, using semaphores)
 - Protection of data: encryption, isolation
- Computer System Components: Hardware, Operating system, Applications programs, Users (people, machines, other computers).
- Mainframe OS: processing of many jobs at once, automatically transfers control from one job to another
 - Services:
 - Batch system: processes routine jobs in the absence of interactive user.
 - Multi programmed Batch Systems: jobs are kept in main memory at the same time, and the CPU is multiplexed among them; requires I/O

routines, Memory management, CPU scheduling, Allocation of resources

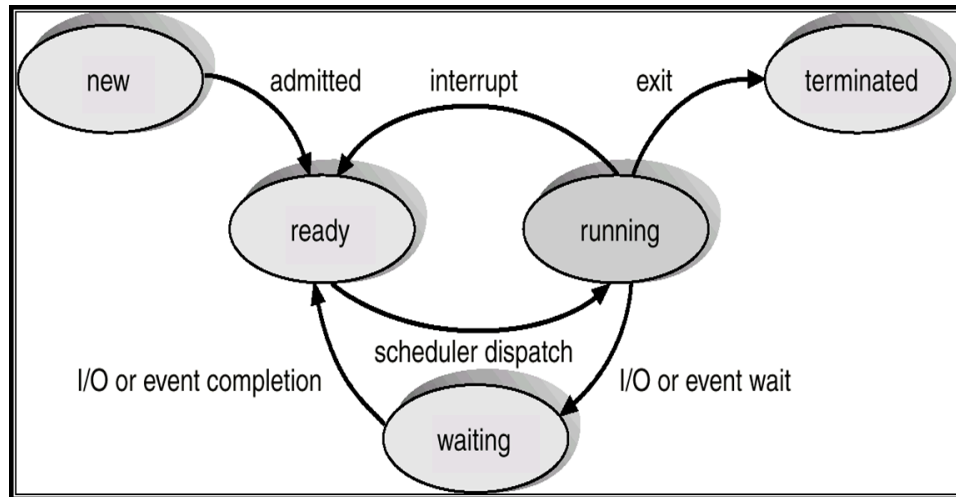
- Transaction processing system: handles large number of small requests
- Time sharing system (interactive computing):
 - CPU is multiplexed among several jobs that are kept in memory and on disk
 - On-line communication between the user and the system
- Desktop Systems (Personal Computers): computer system dedicated to a single user
 - individuals have sole use of computer and do not need advanced CPU utilization
 - may run several different types of operating systems
- Parallel Systems (Multiprocessor OS): connecting multiples CPU's into a single system.
 - Tightly coupled system: processors share bus, memory, clock and peripheral device.
 - communication usually takes place through the shared memory.
 - Advantages: Increased throughput, Economical (share mass storage), Increased reliability (If one fail, the other will take responsibility)
- Distributed Systems: distribute the computation among several physical processors.
 - Loosely coupled system: each processor has a local memory, clock, peripheral devices,...
 - communication through communications lines, (high-speed buses or telephone lines)
 - Advantages: Resources Sharing, Sharing load, Reliability, Communications
 - Requires networking infrastructure; LAN or WAN; Client-server or peer-to-peer

- Real-Time Operating Systems: often a control device in a dedicated application; time is a key parameter
- Embedded Operating Systems: Personal Digital Assistants (PDAs), Cellular telephones, Windows CE (Consumer Electronics), PalmOS, Android OS, ...
 - Issues: Limited memory, Slow processors, Small display screens
- Contemporary OS: prioritizes convenience, efficiency, and the ability to evolve
 - Convenience: User Interface (user friendly GUI), Application Support (platform runs many apps), File Management (directories, folders, and file navigation interfaces), Device integration (plug and play, connect and use devices like printers)
 - Efficiency: Resource Management (optimal performance, manage memory, processor, and storage..), Memory Management (handles memory allocation and deallocation), Multitasking (concurrent execution of multiple apps or processes), I/O Management (allowing devices to communicate with the computer system)
 - Ability to Evolve: Compatibility (allowing users to upgrade their systems without losing compatibility with existing applications), Updates and Patches (vendors frequently release updates and patches to address vulnerabilities), Modularity (allowing components to be updated or replaced independently), Support for New Technologies (incorporate support for emerging technologies i.e. cloud, virtualization, AI, IoT)

Process Management: fundamental task of modern operating systems

- Includes:
 - Creation of processes, Allocation of resources to processes, Managing conflicting demands
 - Protecting resources of each processes from other processes
 - Enabling processes to share and exchange information
 - Enabling synchronization among processes for sequencing & coordination (dependencies)
 - Termination of processes

- Process vs. Program
 - Program: sequence of instructions defined to perform some task; a passive entity
 - Process: a program in execution; instance of a running program; an active entity
 - A processor performs the actions defined by a process
- Types of Processes:
 - Sequential Processes: execute one after the other; at most one process is being executed; traditional single-threaded programs or sequential algorithms
 - **Concurrent Processes**
 - **True Concurrency** (Multiprocessing): two or more processes are executed simultaneously in a multiprocessor environment (real parallelism)
 - **Apparent Concurrency** (Multiprogramming): pseudo-concurrency or simulated concurrency, executed in parallel in a uniprocessor environment by switching from one process to another (pseudo parallelism)
- Process States:
 - New: process just been created but has not yet been admitted to the pool of executable processes by the operating system.
 - information about process is already maintained in memory but the code is not loaded and no space has been allocated for the process.
 - Ready: process not currently executing but is ready as soon as the OS dispatches it.
 - process is in main memory and is available for execution.
 - Running: process is currently being executed.
 - Blocked (Waiting): process is waiting for the completion of some event, such as I/O.



- Process Control Block (task control block) - PCB: how a process is represented in the OS
 - Information:
 - **Process state:** new, ready, running and waiting, halted
 - **Program counter:** address of the next instruction to be executed for this process.
 - **CPU registers:** depends on the computer architecture
 - accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.
 - **CPU-Schedule information:** priority, pointers to scheduling queues, scheduling params
 - **Memory-management information:** value of the base and limit registers, the page tables or the segment tables, (depends on the memory system of the OS)
- Threads: dispatchable unit of work (lightweight process)
 - has independent context, state and stack
 - process is a collection of one or more threads and associated system resources
 - traditional operating systems are single-threaded systems.
 - Types of threads

- User threads: supported above the kernel; implemented by a thread library at the user level.
- Kernel threads: supported directly by the OS; performs thread creation, scheduling, and management in kernel space
- **Multithreading:** a process, executing an application, is divided into threads that can run concurrently.
 - Modern operating systems are multithreaded systems.
 - Benefits:
 - **Responsiveness:** allow a program to continue running even if part of it is blocked or is performing a lengthy operation; increases responsiveness to the user.
 - **Resource sharing:** by default, threads share the memory and the resources of the process to which they belong.
 - **Economy:** allocating memory and resources for process creation is costly; threads share resources of the process to which they belong; it is more economical to create and context switch threads
 - **Utilization of multiprocessor architectures:** each thread may be running in parallel on a different processor; a single-threaded process can only run on one CPU

CPU Scheduling

- CPU Scheduler: selects among ready processes and allocates CPU to one of them
 - CPU scheduling decisions when:
 - Switching from running to waiting state (non-preemptive)
 - Switching from running to ready state (preemptive)
 - Switching from waiting to ready (preemptive)
 - Terminating (non-preemptive)
- CPU Scheduling Criteria

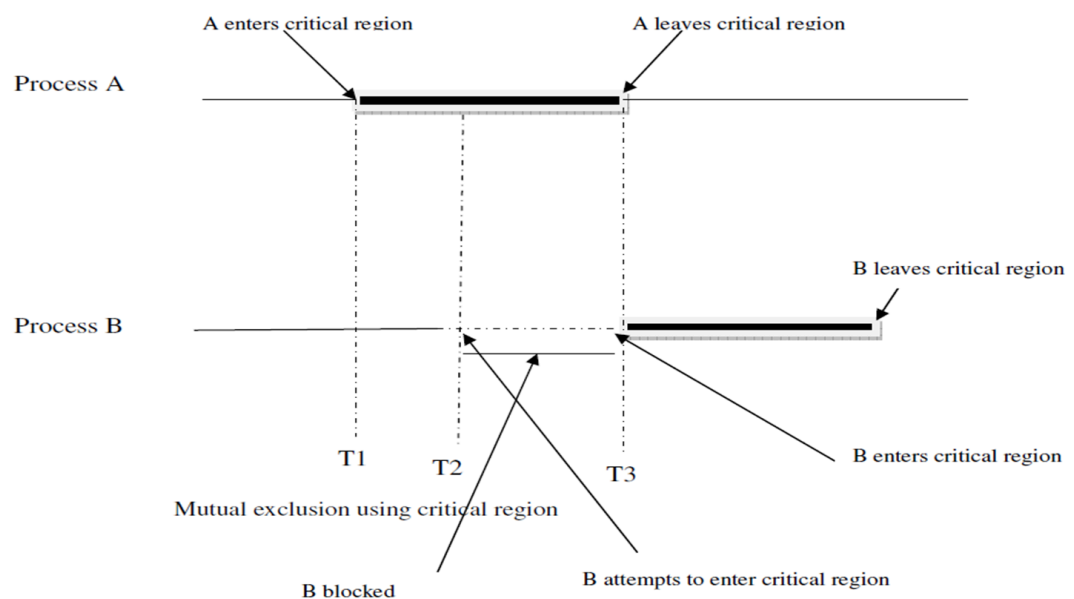
- Maximize throughput: run as many jobs as possible in a given amount of time.
 - by running only short jobs or by running jobs without interruptions.
- Minimize response time: quickly turn around interactive requests.
 - by running only interactive jobs and letting batch jobs wait until the interactive load ceases.
- Minimize turnaround time: move entire jobs in and out of the system quickly.
 - by running all batch jobs first (because batch jobs can be grouped to run more efficiently than interactive jobs).
- Minimize waiting time: move jobs out of the READY queue as quickly as possible.
 - by reducing the number of users allowed on the system.
- Maximize CPU efficiency: keep the CPU busy 100 percent of the time.
 - by running only CPU-bound jobs (and not I/O-bound jobs).
- Ensure fairness for all jobs: give everyone an equal amount of CPU and I/O time.
 - by not giving special treatment to any job, regardless of its processing characteristics or priority.
- Process Scheduling Algorithms
 - Process Scheduler relies on a process scheduling algorithm; based on a specific policy, to allocate the CPU and move jobs through the system
 - Algorithms:
 - First-Come, First-Served (FCFS): non preemptive, uses FIFO queue, no wait queues in strict versions (each job runs to completion)
 - Shortest-Job-First (SJR): associate length of CPU bursts for each process,
 - non-preemptive: can't be preempted until process completes its CPU burst

- preemptive: preempt if a new process arrives with less CPU burst length than the current running process's remaining time
- SJF is optimal: gives minimum average waiting time for a given set of processes
- Priority Scheduling: priority number (integer) is associated with each process
 - SJF is a priority scheduling where priority is the predicted next CPU burst time
 - Problem: Starvation – low priority processes may never execute
 - Solution: Aging – as time progresses increase the priority of the process
- Round Robin (RR): each process gets a small unit of CPU time (*time quantum*)
 - After time has elapsed, process is preempted and added to the end of the ready queue.
 - If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
 - No process waits more than $(n - 1) * q$ time units.
 - Performance: if q is large it becomes a FIFO; if q is small context switching overhead is too high.

Inter Process Communication

- Process need to communicate with other processes.
- Race Condition: situation where several processes access – and manipulate shared data concurrently (main memory, printer...)
 - final value of the shared data depends upon which process finishes last
 - to prevent race conditions, concurrent processes must be synchronized
- The Critical-Section Problem: How do we avoid race conditions?

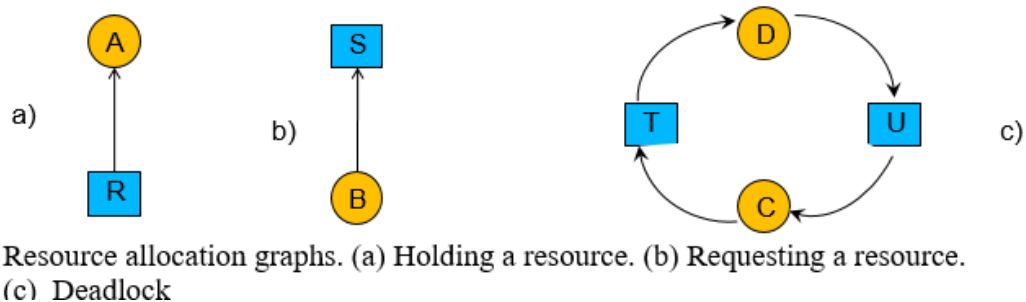
- Critical region: portion of code where a process accesses **shared resources** like memory, files, or devices. If multiple processes enter their critical sections at the same time, it can lead to **race conditions**
- Solution: **Mutual Exclusion**
 - No two processes may be simultaneously inside the critical regions.
 - No assumptions may be made about speeds or the number of CPUs.
 - No process running outside its critical region may block other processes.
 - No process should have to wait forever to enter its critical region.



- Algorithms: Disabling Interrupts, Lock Variables, Strict Alternation, Semaphores, Monitors, Message passing

Deadlock: a process needs exclusive access to several resources; it holds one resource and waits till another resource is released, and this other resource is held by a process waiting for the first resource to be released

- None of the processes can run, none of them can release any resources, and all the processes continue to wait forever
- Conditions of a deadlock (all required):
 - Mutual exclusion condition: resource is assigned to one process at most.
 - Hold and wait condition: processes can request new resources without releasing other resources granted earlier
 - No preemption condition: resources once granted cannot be forcibly taken away, must be explicitly released by the process holding them.
 - Circular wait condition: circular chain of two or more processes, each of which is waiting for a resource held by the next member of the chain
- Deadlock modeling: modeled using directed graphs



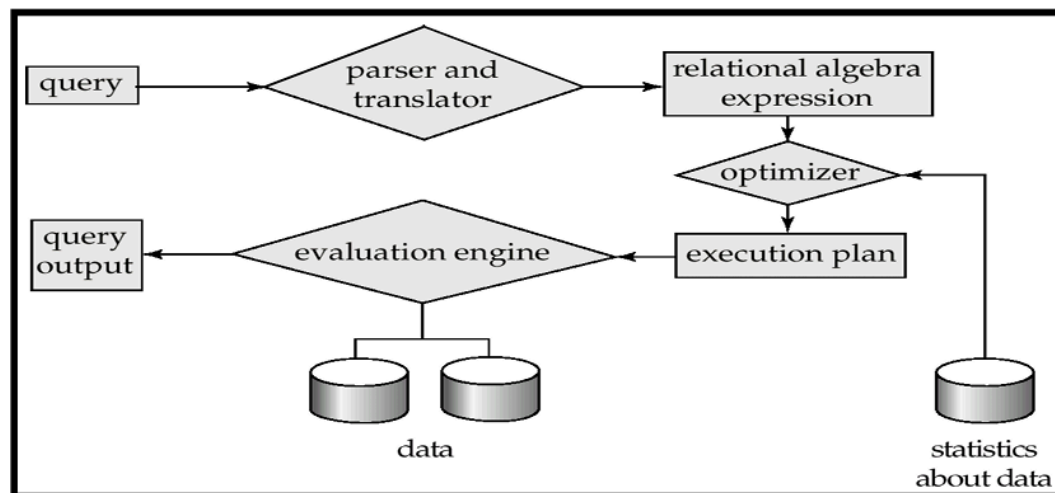
- If granting a particular request might lead to deadlock, the OS can suspend the process without granting the request (don't schedule the process until it is safe)
- Strategies to deal with deadlocks:
 - Just ignore the problem altogether - Ostrich Algorithm
 - Detection and recovery: Let deadlocks occur, detect them, and take action.
 - Detection: with One Resource of Each Type, with Multiple Resources of Each Type

- Recovery: through Preemption, through Rollback, through Killing Processes
- Dynamic avoidance: careful resource allocation; whether granting a resource is safe or not
 - Algorithms: Safe and Unsafe States, The Bankers algorithm
- Prevention: by structurally negating one of the four conditions of a deadlock.

Advanced Database Systems

Query Processing and Optimization

- Query processing: steps required to transform high level SQL query into a correct and efficient strategy for execution and retrieval
- Query Processing Steps:



- Decomposition: transforming a high level query into a relational algebra query; check that the query is syntactically and semantically correct
 - consists of parsing and validation
 - Stages:

- Analysis: lexical and syntactical analysis, based on attributes, data type
 - Query tree will be built:
 - leaf node for base relations
 - non-leaf nodes for relations produced by relational algebra operations
 - root node for the result of the query
 - Activities: analyze query with compiler techniques, verify relations and attributes exist, and verify operations
- Normalization: convert the query into a normalized form. The predicate WHERE will be converted to Conjunctive (A) or Disjunctive ($\vee$) Normal form.
- Semantic Analysis: to reject normalized queries that are not correctly formulated or contradictory. i.e. (Catalog="BS" ^ Catalog= "CS")
- Simplification: detect redundant qualifications, eliminate common sub-expressions, and transform the query to a semantically equivalent effective computed form
- Optimization: find the sequence of steps that produces the answer efficiently
 - Optimizer: chooses an optimal plan for evaluating the request
 - Narrow down intermediate result sets quickly. SELECT and PROJECTION before JOIN
 - Use access structures (indexes)
 - Approaches:
 - Heuristics Approach: uses relationship between the operators and characteristics of relational algebra (Properties of individual operators & Association between operators)

- Transformation rules: <https://www.geeksforgeeks.org/query-optimization-in-relational-algebra/>
 1. Cascade of Selection: $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
 - applying condition θ_1 intersection θ_2 is expensive
 2. Commutativity of Selection Operations: $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$
 - it does not matter whether we apply σ_1 first or σ_2
 3. Cascade of Projection: $\pi_{L_1}(\pi_{L_2}(\dots(\pi_{L_n}(E))\dots)) = \pi_{L_1}(E)$
 - a series of projections is meaningless, outermost projection applies
 4. Commutativity of Selection:
 - $\pi_{L_1}(\sigma_{\theta}(E)) = \sigma_{\theta}(\pi_{L_1}(E))$ - with projection
 - $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$ - with cross product
 - cross product is expensive - applying a selection operation after that scans $m * n$ entries,
 - select only those entries in the cross product which satisfy the Theta condition, without evaluating the entire cross product first
 5. Commutativity of Theta Join and Cross product:
 - $E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1 ; R \times S = S \times R$
 6. Commutativity of Selection with Theta Join
 - $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$
 - where θ_1 contains the attributes of only E1 and θ_2 contains attributes of only E2, apply selections individually
 - $\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$
 - If θ_0 contains attributes from only E1, apply this selection to E1

7. Commutativity of Projection and Theta Join

- $\pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\pi_{L_1}(E_1)) \bowtie_{\theta} (\pi_{L_2}(E_2))$
 - if L_1 is a projection involving columns of only E_1 , and L_2 another projection involves the columns of only E_2 , individually apply the projections on both the tables before joining
- $\pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \pi_{L_1 \cup L_2}((\pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\pi_{L_2 \cup L_3}(E_2)))$
 - introduce L_3 if θ depends on a column not in L_1 and L_2

8. Commutativity of Set Operations:

- $E_1 \cup E_2 = E_2 \cup E_1$
- $E_1 \cap E_2 = E_2 \cap E_1$
- $R - S \neq S - R$

9. Associativity of Theta Join, Cartesian Product, Union, and Intersection

- $(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$
- $(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$
- $(R \bowtie_{C_1} S) \bowtie_{C_2} T \equiv R \bowtie_{C_1} (S \bowtie_{C_2} T)$
- $(R \times S) \times T \equiv R \times (S \times T)$

10. Commuting Selection with Set Operations: reduces the number of comparisons

- $\sigma_P(E_1 - E_2) = \sigma_P(E_1) - \sigma_P(E_2)$
- $\sigma_C(R \cup S) \equiv \sigma_C(R) \cup \sigma_C(S)$
- $\sigma_C(R \cap S) \equiv \sigma_C(R) \cap \sigma_C(S)$

11. Commuting Projection with Union: optimal

- $\pi_L(E_1 \cup E_2) = (\pi_L(E_1)) \cup (\pi_L(E_2))$

- Sequence for Applying Transformation rules:
 1. Rule 1 - cascade selection
 2. Rule 2, Rule 4, Rule 6 - commutativity of selection
 3. Rule 9 - associativity
 4. Rule 3 (cascade projection), Rules 4, 7, 11 - commutativity of projection
- Using Heuristics: generate an initial internal representation (parser), optimize with heuristic rules, generate execution plan
 - main heuristic is to apply first the operations that reduce the size of intermediate results
- Query block: The basic unit that can be translated into the algebraic operators and optimized
 - contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clause if these are part of the block
 - Nested queries within a query are identified as separate query blocks
- Query graph: corresponds to a relational calculus expression; doesn't indicate order of operations to perform
- Cost Estimation Approach to Query Optimization: minimize the cost of processing a query
 - Cost function: I/O cost + CPU processing cost + communication cost + Storage cost
 - Use formulae that estimate costs for a number of options, and select one with lowest cost
 - Estimate and compare the costs of executing a query using different execution strategies and choose the strategy with the lowest cost estimate

- Cost components: <https://www.geeksforgeeks.org/cost-based-optimization/>
 - Access Cost (from secondary storage): Searching, Reading, Writing
 - Memory Cost: buffers for execution of query
 - Storage Cost: cost of storing any intermediate files (intermediary results)
 - Computation Cost (operations): Searching, Sorting, Merging
 - Communication Cost: sending or communicating the query and its results.
- Execution:
 - Query Execution Plans consists of:
 - Relational algebra query tree,
 - Information about the access methods to be used for each relation
 - The methods to be used in computing the relational operators stored in the tree
 - Pipelining:
 - Pipelined evaluation: evaluate several operations simultaneously, passing the results of one operation on to the next
 - pass tuples directly to the join instead of storing results,
 - pass output of join directly to projection
 - Pipelines are executed in 2 ways:
 - Demand driven (lazy evaluation): system repeatedly requests next tuple from top level operation; between calls, operation has to maintain state so it knows what to return next
 - Producer-driven (eager pipelining): operators produce tuples eagerly and pass them up to their parents; buffer maintained

between operators; if buffer is full child waits till it's free and then generates more tuples

- Code generation

Database Security and Authorization

- Database integrity: makes sure that (authorised) users use that information correctly
- Integrity constraints
 - Domain constraints apply to data types
 - Attribute constraints apply to columns
 - Relation constraints apply to rows in a single table
 - Database constraints apply between tables
- Intrarecord integrity (enforcing constraints on contents of fields, etc.)
- Referential Integrity (enforcing the validity of references between records in the database)
- Constraints to protect from inconsistency:
 - Required data: some attributes must always contain a value
 - Attribute domain constraints: a set of values that are legal for an attribute to use
 - Entity integrity: primary key of any entity must be unique and can't be NULL
 - Referential integrity: foreign key value refers to existing occurrence in the parent entity
 - Inserting rules: row should not be inserted in the referencing (child) table unless there already exists a matching entry in the referenced table; sometimes NULL is used in FK
 - Deletion rules: row should not be deleted from the referenced table (parent) if there are matching rows in the referencing table (child); options: Restrict, Nullify, Cascade

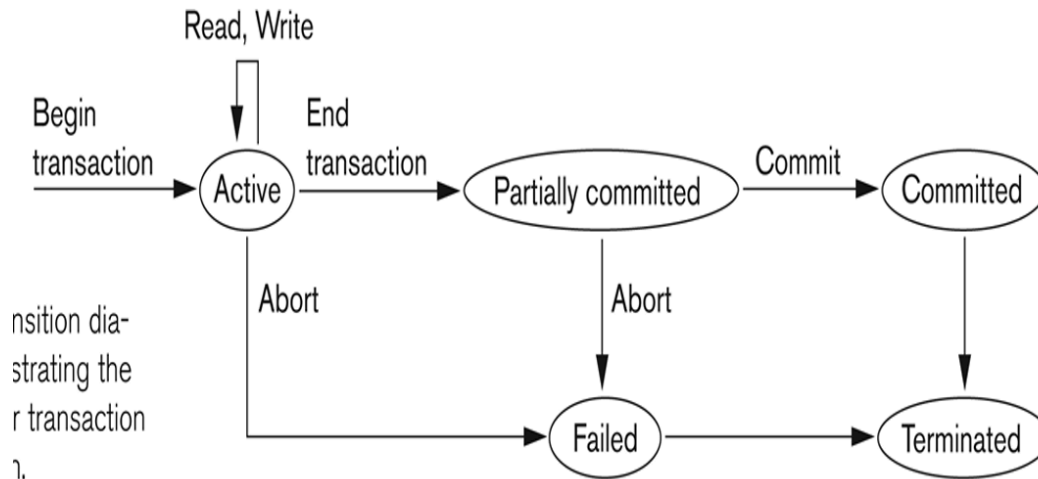
- Enterprise constraints: business rule that may affect the database and the data
- Concurrency control (ensure validity of database updates in a shared multiuser environment)
 - support access by multiple users to the same data, at the same time
 - handle problems (Lost updates, Inconsistent data states, Uncompleted changes to data)
 - Locking:
 - Levels: Database, Table, Block or page, Record, Field
 - Types: Shared (S locks), Exclusive (X locks)
 - Part or all of a transaction can be undone using ROLLBACK
 - Savepoints are places in a transaction that you may ROLLBACK
 - SET TRANSACTION...;
 - SAVEPOINT ALPHA;
 - *SQL STATEMENTS...*
 - IF (CONDITION) THEN ROLLBACK TO SAVEPOINT ALPHA;
 - SAVEPOINT BETA;
 - *SQL STATEMENTS...*
 - IF ...;
 - COMMIT;
- Database security: makes sure that the user is authorized to access information
 - Aspects to consider: Legal issues, Physical security, OS/Network security, Security policies and protocols, Encryption and passwords, DBMS security
 - DBMS Security Support: DBMS can provide some security (user accounts, user permissions)

- Permissions and Privilege: SELECT privilege, INSERT privilege, UPDATE privilege, DELETE privilege
 - owner (creator) of a database has all privileges on all objects in the database & can grant them
 - owner (creator) of an object has all privileges on that object and can pass them
- Authorization Rules: define access permissions on a table by table basis using GRANT & REVOKE SQL commands
- Privileges in SQL:
 - GRANT <privileges> ON <object> TO <users>
 - <privileges>: ALL, SELECT <columns>, INSERT <columns>, DELETE, UPDATE <columns>
 - <users>: list of usernames or PUBLIC
 - <object>: name of a table or view
 - GRANT <privileges> ON <object> TO <users> WITH GRANT OPTION
 - i.e. GRANT SELECT, UPDATE(Salary) ON Employee TO Finance
 - REVOKE <privileges> ON <object> FROM <users>
- View: A subset of the database presented to some set of users
 - CREATE VIEW name AS SELECT field1, field FROM table1, table2 WHERE <where clause>
 - Updates to the base tables change the views and vice-versa
 - For a view to be updatable: Should not use GROUP BY, HAVING, JOIN, DISTINCT, UNION
- Backup and Recovery:
 - Threats to Assets and Functions: Water, Fire, Power Failure, Mechanical breakdown or software failure, Accidental or deliberate destruction of hardware or software (hackers)

- Kinds of Records: Class I: VITAL (irreplaceable), Class II: IMPORTANT (reproducible), Class III: USEFUL (replaceable), Class IV: NONESSENTIAL

Transaction Processing Concepts

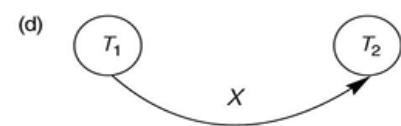
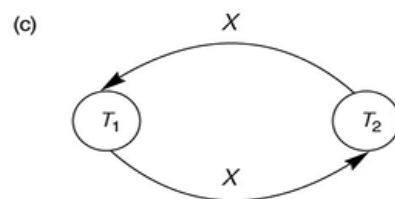
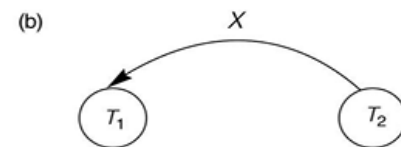
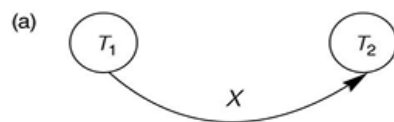
- Transaction: atomic unit of work that is either completed in its entirety or not done at all
 - bounded with Begin and End statements
 - operations:
 - read_item(X) - reads a database item named X into a program variable
 - write_item(X) - writes the value of program variable X into the database item named X
 - system needs to keep track of when transaction starts, terminates, and commits or aborts
- Granularity of data: field, record , or a whole disk block that measure the size of the data item
- DBMS maintains a number of buffers in the main memory holding database disk blocks which contains the database items being processed
 - When buffer is occupied and there is need for more: buffer management policy is used to choose for replacement but if the chosen buffer has been modified, it must be written back to disk before it is used
- Transaction states: Active (beginning of a transaction execution), Partially committed (end of read/write operation), Committed (persistent changes), Failed (aborted during active state or rechecking fails), Terminated (transaction leaving the system)

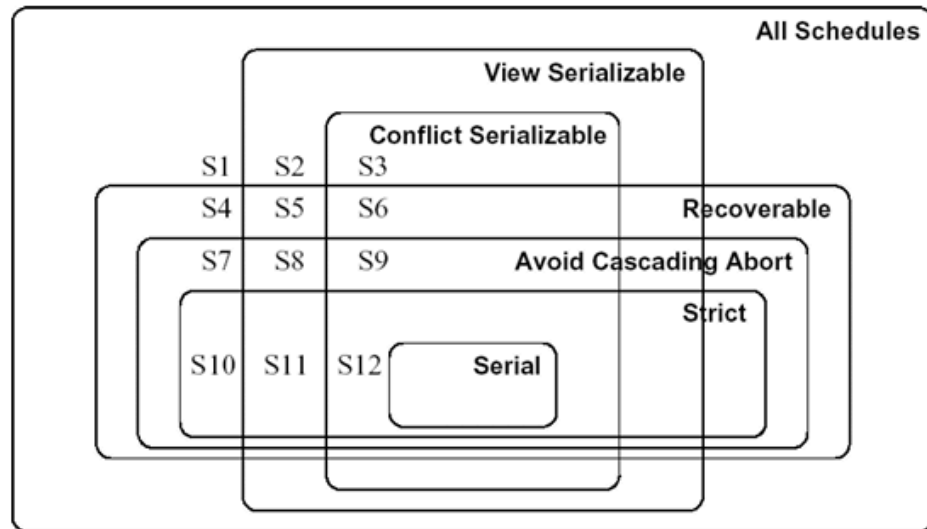


- Types of log record: [start_transaction,T], [write_item,T,X,old_value,new_value], [read_item,T,X], [commit,T], [abort,T]
- Properties of Transactions: Atomicity (performed entirely or not), Consistency preservation, Isolation (updates invisible until commit), Durability or permanency (no loss after commit)
- Transaction Processing: Single-User System (PC), Multiuser System (multiprogramming, i.e. banks), Concurrency (Interleaved - single CPU - round robin, Parallel - multiple CPUs)
- Problems of Concurrent Sharing:
 - Lost Update Problem: interleaved operations overwrite other transaction's update
 - Temporary Update/Dirty Read: transaction fails after updating item, updated item is accessed by another transaction before item is rolled back to original value
 - Incorrect Summary Problem: one transaction calculating an aggregate summary function while others are updating some of these records; aggregate could read some updates and not others
- Schedules and Serializability:
 - Schedule/history: order of execution of operations from the various transactions
 - Sa : r2(X);w2(X);r1(X);w1(X); (schedule = transaction 2 reads X, ...)

- Conflict in schedule: operations belonging to different transaction access the same data item X & there is at least one write_item(X)
- Schedules classified by recoverability:
 - Recoverable: no committed transactions need to be rolled back;
 - for every pair of transactions T_i and T_j
 - T_j reads a data item previously written by T_i
 - T_i must commit before T_j commits.
 - Sc: $r1(X); \underline{w1(X)}; \underline{r2(X)}; r1(Y); w2(x); \underline{c2}; a1$; not recoverable
 Sd: $r1(X); \underline{w1(X)}; \underline{r2(X)}; r1(Y); w2(X); w1(Y); \underline{c1}; c2$; recoverable
 Se: $r1(X); \underline{w1(X)}; \underline{r2(X)}; r1(Y); w2(x); w1(Y); \underline{a1}; a2$; recoverable
 - Cascade-less: every transaction reads only the items that are written by committed transactions
 - Sf: $r1(X); w1(X); r1(Y); \underline{c1}; \underline{r2(X)}; w2(X); w1(Y); c2$;
 - Strict: transaction can neither read or write an item X until the last transaction that wrote X has committed/aborted
- Schedules based on Serializability:
 - Serial schedule: for every transaction T_i participating in the schedule, all the operations of T_i are executed consecutively in the schedule, else non-serial
 - Serializable schedule: effect on any consistent database instance is identical to that of some complete serial schedule over the set of committed transactions in S
- Result equivalent: two schedules produce the same final state of the database
 - Conflict equivalent: order of any two conflicting operations is the same in both schedules
 - S1: $\underline{r1(x)}; \underline{w2(x)}$ & S2: $\underline{w2(x)}; \underline{r1(x)}$ - not conflict equivalent

- Conflict serializable: S is conflict serializable if conflict equivalent to another serializable schedule S_2 ; implies S is serializable
- View equivalent: same set of transactions participates in S and S' , and S and S' include the same operations of those transactions
 - if T_i reads a value A written by T_j in S_1 , it must also read the value of A written by T_j in S_2
 - for each data object A, the transaction that perform the final write on A in S_1 must also perform the final write on A in S_2
- schedule is serializable if and only if the precedence graph has no cycles
 - (a) serial schedule A; (b) serial schedule B; (c) schedule C (not serializable)
 - (d) schedule D (serializable, equivalent to schedule A).





Venn Diagram for Classes of Schedules

- Transaction Support in SQL:
 - A single SQL statement is always considered to be atomic.
 - Transaction characteristics:
 - Access mode: either READ ONLY or READ WRITE (default)
 - Diagnostic size n: number of error conditions that can be held simultaneously in the diagnostic area
 - Isolation level:
 - READ UNCOMMITTED: Dirty reads allowed
 - READ COMMITTED: No dirty reads
 - REPEATABLE READ: No phantom read (multiple reads guarantee same record - no change by other transactions after reading no need to read again)
 - SERIALIZABLE (default): Fully isolated transactions
 - Commit Transaction SQL: COMMIT;

- Rollback Transaction: ROLLBACK; or ROLLBACK TO SAVEPOINT my_first_savepoint;

Concurrency Controlling Techniques

- Transaction Processor is divided into:
 - Concurrency-control manager (scheduler): assure isolation of transactions
 - by maintaining locks on certain pieces of the database
 - locks prevent two transactions from accessing the same piece of data at the same time
 - Logging and recovery manager: responsible for the durability of transactions.
- Concurrency Control Techniques:
 - Locking: conservative (delay transactions in case they conflict with other transactions)
 - Lock: variable associated with a data item, describes it's status of possible operations
 - prevents another transaction from modifying item or even reading it in case of write lock
 - Generally, a transaction must claim a shared (read) or exclusive (write) lock on a data item before read/write
 - Unlocking: removes these permissions from the data item
 - Shared mode: shared lock (X): More than one transaction can apply share lock on X for reading its value but no write lock can be applied on X by any other transaction.
 - Exclusive mode: Write lock (X): Only one write lock on X can exist at any time and no shared lock can be applied by any other transaction on X
 - Lock table: used by lock manager to identify locks by a transaction on a data item and pointer to the next data item locked (linked-list)

- Well formed transaction:
 - must lock the data item before it reads or writes to it;
 - must not lock an already locked data items and it must not try to unlock a free data item
- Lock conversion: Lock upgrade (existing read lock to write lock), Lock downgrade (existing write lock to read lock)
- Using such locks in transaction do not guarantee serializability of schedule on its own
- Two-Phase Locking Techniques: The algorithm - Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction
 - Locking (Growing) Phase - cannot release any locks
 - Unlocking (Shrinking) Phase - cannot acquire new locks
- Deadlock: two or more transaction are each waiting for locks held by the other to be released
 - Solutions:
 - Deadlock prevention protocol:
 - The conservative two-phase locking: transaction locks all data items it refers to before it begins execution
 - Limitation: restricts concurrency
 - Transaction Timestamp(TS(T)): giving each transaction a priority; lower priority transactions can't to wait for higher priority transactions (or vice versa).
 - assign priorities by a timestamp of when transaction starts up
 - oldest transaction has the highest priority
 - if T_i requests a lock held by T_j , strategies:
 - Wait-die: older transaction is allowed to wait on a younger transaction, younger transaction

requesting an item held by an older transaction is aborted

- if T_i is older, it'll wait; if T_j requests what T_i holds, it's aborted
- Wound-wait: younger transaction is allowed to wait on an older one; older transaction requesting an item held by a younger transaction preempts the younger transaction by aborting it
 - if T_i is older, abort T_j ; if T_j requests what T_i holds, it'll wait
- Deadlock Detection and resolution: deadlocks are allowed to happen, scheduler maintains a wait-for-graph for detecting cycle, abort the transactions that have made the least work & have least number of locks and rollback
- Timeouts: if transaction waits for a long time, system assume that may be deadlocked and aborts it; has lower overhead cost and it is simple
- Starvation: a transaction consistently waits or restarted and never gets a chance
 - Solution: FIFO, allow transactions that waited a long time, give higher priority to transactions that have been aborted many times
- Timestamping: conservative:
 - Timestamp: assigned based on time in which the transactions are submitted
 - uses timestamp to serialize the execution of concurrent transactions
 - Monotonically increasing integer indicates the age of an operation or a transaction
 - larger timestamp value indicates a more recent event or operation
 - doesn't use lock, thus deadlock cannot occur

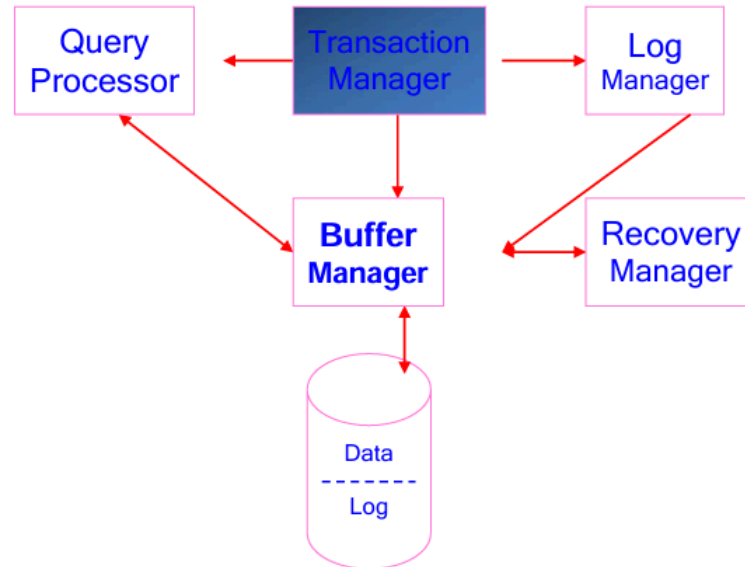
- Conflict check uses:
 - Read_TS(X): Largest TS of transactions that read X.
 - Write_TS(X): Largest TS of transactions that wrote X.
- Algorithms:
 - Basic Timestamp Ordering
 - Write operation $T \rightarrow \text{write_item}(X)$: abort & restart if
 - $\text{read_TS}(X) > \text{TS}(T)$ (a younger transaction read before T wrote)
 - $\text{write_TS}(X) > \text{TS}(T)$ (a younger transaction already wrote)
 - else perform write and update $\text{write_TS}(X) = \text{TS}(T)$
 - Read operation $T \rightarrow \text{read_item}(X)$: abort & restart if:
 - $\text{write_TS}(X) > \text{TS}(T)$ (a younger transaction wrote before T read)
 - else perform read and set $\text{read_TS}(X) = \max(\text{read_TS}(X), \text{TS}(T))$
 - Strict Timestamp Ordering: delaying instead of aborting.
 - Write operation $T \rightarrow \text{write_item}(X)$: If $\text{TS}(T) > \text{read_TS}(X)$: **Delay T** until the reading transaction finishes.
 - Read operation $T \rightarrow \text{read_item}(X)$: If $\text{TS}(T) > \text{write_TS}(X)$: **Delay T** until the writing transaction finishes.
 - Thomas's Write Rule (Improved Basic TS): skip unnecessary aborts for writes
 - Write operation $T \rightarrow \text{write_item}(X)$:
 - If $\text{read_TS}(X) > \text{TS}(T)$: **Abort T**
 - If $\text{write_TS}(X) > \text{TS}(T)$: **Ignore write**, continue (newer write already exists)
 - Else: Perform write and update $\text{write_TS}(X) = \text{TS}(T)$
- Multi-version Concurrency Control: Keeps multiple versions of each data item.
 - Each version X_i has: $\text{read_TS}(X_i)$, $\text{write_TS}(X_i)$

- Read never rejected (guaranteed version always available).
- Ensures **view serializability**, not conflict serializability.
- Requires **more storage**; uses **garbage collection** to clean up
- based on Timestamp Ordering:
 - Write operation $T \rightarrow \text{write_item}(X)$:
 - Let X_i be version with highest $\text{write_TS}(X_i) \leq \text{TS}(T)$
 - If $\text{read_TS}(X_i) > \text{TS}(T)$: **Abort T**
 - Else: Create new X_{i+1} Set $\text{read_TS}(X_{i+1}) = \text{write_TS}(X_{i+1}) = \text{TS}(T)$
 - Read operation $T \rightarrow \text{read_item}(X)$:
 - Find version X_i with highest $\text{write_TS}(X_i) \leq \text{TS}(T)$
 - Return X_i 's value
 - Set $\text{read_TS}(X_i) = \max(\text{read_TS}(X_i), \text{TS}(T))$
- Two-Phase Locking (Certify Lock): uses **two versions**: committed X and tentative X' ; improves concurrency, may cause deadlocks, prevents cascading aborts
 - Allows reading while another transaction holds **write lock**.
 - Steps:
 - X = committed version; T obtains **write lock**, creates X' .
 - Other transactions continue reading X ;
 - T ready to commit $\rightarrow$ obtain **certify lock** on X' .
 - Promote X' to committed version X . Release certify lock.
- Optimistic methods: assume conflict is rare and only check for conflicts at commit; If there's a conflict $\rightarrow$ **abort** and retry.
 - Phases:
 - **Read Phase**: transaction reads committed data; writes go to **local copies**.
 - **Validation Phase** (before commit):

- **Did transaction conflict with others?** If yes → **Abort**; If no → Proceed to write.
- Rules (check in order, If none match → abort T_i):
 1. T_j finished writing before T_i started reading
 2. T_i writes **after** T_j wrote AND T_i read & T_j wrote **disjoint sets**
 3. T_i and T_j read/write **disjoint sets** AND T_j finished reading before T_i
- **Write Phase:** If validated → **Apply changes to DB**; if failed → **Restart transaction**
- Multiple Granularity Locking: controls what level of data a transaction locks: DB vs. a field.
 - Granularity = size of lockable unit
 - A field of a database record
 - A database record
 - A disk block/ page
 - An entire file
 - The entire database
 - Locking Rules (must be followed for producing serializable schedule):
 - Always lock the **root first**
 - To lock a child, the **parent must be locked**:
 - **Shared (S)/Intent Shared (IS)**
 - **Exclusive (X)/Intent Exclusive (IX)/Shared + Intent Exclusive (SIX)**
 - Once you unlock a node, **you can't lock more** → Follows 2PL (Two-Phase Locking)
 - Can unlock a node **only if none of its children are locked**

Database Recovery Techniques

- Purpose of Database Recovery:
 - bring the DB into the last consistent state (existed before failure)
 - preserve transaction properties (Atomicity & Durability)
- Types of Failure:
 - Transaction failure: incorrect input, deadlock.
 - System failure: addressing error, application error, OS fault, RAM failure, etc.
 - Media failure: Disk head crash, power disruption, etc.
- What causes a Transaction to fail:
 - Computer failure (system crash)
 - Transaction or system error: integer overflow, division by zero
 - Exception conditions detected by the transaction: i.e. insufficient account balance
 - Concurrency control enforcement: concurrency control aborted the transaction
 - Disk failure: disk read/write head crash, lost disk block
 - Physical problems and catastrophes
- Transaction Manager: accepts transaction commands from an application
- Transaction processor tasks: Logging (ensure durability), negotiate with buffer manager (buffer → disk), restore the DB to consistent state (Recovery manager)

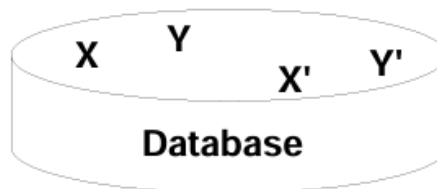


- Recovery Algorithms parts: actions taken on normal execution, actions take on failure
- Storage Structure: Volatile (doesn't survive crashes - main memory, cache), Nonvolatile (survives system crashes - disk, tape, flash), Stable (approximated by maintaining multiple copies)
- Stable-Storage Implementation:
 - multiple copies of each block on separate disks (can be at remote sites),
 - write info. on 1st block → write on 2nd block → successful (prevents inconsistent copies)
 - find inconsistent blocks (bad checksum)
 - Compare the two copies of every disk block - expensive
 - Record in-progress disk writes on non-volatile storage - better approach
- Data Access: Physical blocks (disk), Buffer blocks (main memory)
 - Block movement: input(B) - from disk to memory, output(B) - from memory to disk
 - Each transaction T_i has its private work-area (local copies of all it's data items)
 - T_i 's local copy of a data item X is called x_i .

- $\text{read}(X)$ assigns the value of data item X to the local variable x_i .
 - $\text{write}(X)$ assigns the value of local variable x_i to data item $\{X\}$ in the buffer block.
- $\text{output}(B_X)$ need not immediately follow $\text{write}(X)$. system performs output when deems fit
- Recovery and Atomicity: first output modifications to stable storage without modifying DB
 - Approaches
 - Log based recovery
 - System log: to recover from system failure, system keeps info. on changes to system logs;
 - Strategies: Recovery from catastrophic, Recovery from non-catastrophic failure
 - if transaction hadn't committed at failure, recovery manager has to rollback
 - Transaction Log: data values prior to modification and the new value after modification are required
 - BFIM - BeFore Image & AFIM – AFter Image are stored in a sequential file (appended file) called Transaction log
 - Data Caching: data items to be modified are first stored into database cache by the Cache Manager (CM) and after modification they are flushed (written) to the disk
 - DBMS request for read /write operation: check cache first, else get from disk
 - Write-Ahead Logging:
 - log is necessary for in-place update (immediate or deferred)
 - For Undo: Before AFIM is flushed to DB disk, its BFIM must be written to the log and the log saved on a stable store (log disk).

- For Redo: Before a transaction commits, all its AFIMs must be written to the log
- ways for flushing: No-Steal (can't flush before commit), Steal (can flush before commits - if no space), Force (immediately flushed on commit), No-Force (flush when needed after commit)
 - Combinations:
 - Steal/No-Force (Undo/Redo)
 - Steal/Force (Undo/No-redo)
 - No-Steal/No-Force (No-undo/Redo)
 - No-Steal/Force (No-undo/No-redo)
- Check pointing: sync point between DB & log; reduces recovery time by only requiring redo/undo of transactions after the checkpoint
 - on failure: redo all committed transactions after checkpoint; undo all active transactions at time of crash
- Data Update and Recovery Scheme:
 - **Deferred Update (No Undo/Redo):** Changes are logged and only written at commit, requiring only redo during recovery.
 - **Deferred Update – Single User:** No concurrency issues, changes logged and flushed at commit.
 - **Deferred Update – Concurrent Users:** Uses active and commit tables for recovery, requiring concurrency control and redo of committed transactions only.
 - **Immediate Update - Undo/No-redo:** flush AFIMs under WAL before it commits; recovery manager undoes all transactions during recovery; no transaction is redone; even finished transaction is undone
- Shadowing: two page tables during life of a transaction: current page and shadow page

- pages are the same when transaction starts; update current page; shadow page isn't changed and is used for recovery
- current page table records all updates to database; when transaction completes, current page table becomes shadow page table



X and Y: Shadow copies of data items

X' and Y': Current copies of data items

- ARIES Algorithm: steal/no- force approach,
 - principles: WAL (Write Ahead), Repeating history on redo, Logging changes on undo
 - data update types:
 - **Deferred:** changes written to DB only after commit; redo-only recovery.
 - **Immediate:** changes logged & flushed before DB update; undo/redo needed.
 - **Shadow Update:** Writes new version to separate location; no log needed.
 - **In-place Update:** Overwrites DB directly; log-based
 - recovery information:
 - **Log Records:**
 - LSN (Log Sequence Number) - unique per log, increases monotonically, indicates the disk address of the log record
 - each data page stores the LSN of the latest log record applied

- log written on updating a page (page's LSN set to log's LSN), commit, abort, rollback
 - transaction ID, type, page ID, BFIM...
 - **Transaction Table:** active transactions (ID, status - aborted, committed...) and most recent log record LSN for the transaction.
 - **Dirty Page Table:** dirty pages - page ID and LSN of earliest update to that page
- checkpointing: ARIES uses fuzzy check pointing
 - writes a begin_checkpoint record in the log (mark the start of checkpointing)
 - system gathers the contents of the Transaction Table and Dirty Page Table.
 - writes an end_checkpoint record to the log, appending the captured tables.
 - write LSN of begin_checkpoint to a special file on stable storage (used to locate the latest checkpoint)

Distributed Databases and Client-Server Architectures

- Distributed database (DDB):
 - collection of multiple logically related database distributed over a computer network
 - DDB management system (software): manages a DDB; makes the distribution transparent
 - physical placement of data (files, relations, etc.) is not known to the user (transparency).
 - processes unit of execution (a transaction) in a distributed manner
 - Advantages:

- Distribution & Network Transparency: users access data without knowing its location.
- Replication Transparency: same data can be stored at multiple sites for faster access.
- Fragmentation Transparency: data can be split into fragments for efficient storage and access.
- Increased Reliability & Availability: failure of one node doesn't crash the system.
- Improved Performance: data is kept closer to users, reducing access time.
- Scalability: easy to add new nodes without major reconfiguration.
- Disadvantages:
 - Complexity: difficult to manage due to replication, failures, and networks.
 - Higher Cost: more hardware and skilled staff.
 - Heterogeneous System Issues: machines need coordination.
 - Security & Integrity Risks: harder to control data when it's spread across locations.
- Types
 - Homogeneous: identical setup across sites, same DBMS, little or no local autonomy, can have different OS across sites
 - Heterogeneous: different vendors, managed through a single conceptual schema; degree of local autonomy is minimum - each site follows centralized access policy
- Query Processing in Distributed Databases
 - Issues: Cost of transferring data over the network, Reduce total data transfer size during query execution
- Concurrency Control and Recovery:

- **Multiple Data Copies:** maintain consistent copies during operations and recovery.
- **Site Failures:** system stays available even if some sites fail; recovery mechanisms must handle site restoration.
- **Communication Failures:** failure may create network partition which would affect database availability when sites are still running
- **Distributed Commit:** transaction may be fragmented and they may be executed by a number of sites; requires two-phase or three-phase commit protocols.
- **Distributed Deadlock:** deadlocks can span multiple sites; resolution must be distributed.
- Distributed Concurrency Control Approaches:
 1. **Primary Site Technique:** one site coordinates transactions; simple but creates a single point of failure and bottleneck; can use a backup site for fault tolerance.
 2. **Primary Copy Technique:** each data item has a primary copy at a site; distributes load but requires complex directory management.
 3. **Voting-Based Protocol:** no fixed coordinator; lock granted if a majority of replicas agree includes timeouts to prevent indefinite blocking
- Recovery from Coordinator Failure:
 - **With Backup Site:** Switch to backup, resume processing.
 - **Without Backup:** Abort all active transactions, elect new coordinator.
- Three-Tier Client/Server Architecture:
 - Layers: Client (parses and breaks down queries), Web Server (enforces business rules, manages security), Database Server (executes subqueries, returns data)

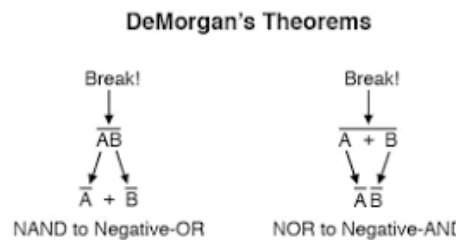
Computer Organization and Assembly Language

Digital Logic Circuits

- Binary: two discrete values, very reliable
- Hardware: Electronic components, electromechanical devices
 - CPU (ALU, Registers, Control), RAM, Input Output Processor
 - Seen from points of: Organization, Design, Architecture
- Software: instructions and data manipulated by hardware
- Computer Architecture (conceptual view): components of computer, user perspective of computer structure and behavior, specifications
- Computer Organization: how computer works, hardware connections, control signals
- Computer Design (implementation): how computer is built, after specs are formulated, determine hardware to be used and parts to connect
- Logic Gates:
 - Functions:
 - Truth table: most elementary specification of Boolean function; describe output values for all combinations of input values (minterms)
 - n input variables $\rightarrow 2^n$ minterms
 - Boolean function: i.e. $F = x + y'z$
 - Karnaugh Map (k-map)
 - Boolean Algebra: algebra with binary variable and logic operations; useful in analysis and synthesis of Digital Logic Circuits
 - Inputs & Outputs: represented by Boolean variables
 - Logic Circuits: represented by logic operations
 - construct logic diagram from Boolean function
 - Basic identities: to simplify and derive equivalent Boolean function

[1] $x + 0 = x$	[2] $x \cdot 0 = 0$
[3] $x + 1 = 1$	[4] $x \cdot 1 = x$
[5] $x + x = x$	[6] $x \cdot x = x$
[7] $x + x' = 1$	[8] $x \cdot x' = 0$
[9] $x + y = y + x$	[10] $xy = yx$
[11] $x + (y + z) = (x + y) + z$	[12] $x(yz) = (xy)z$
[13] $x(y + z) = xy + xz$	[14] $x + yz = (x + y)(x + z)$
[15] $(x + y)' = x'y'$	[16] $(xy)' = x' + y'$
[17] $(x')' = x$	

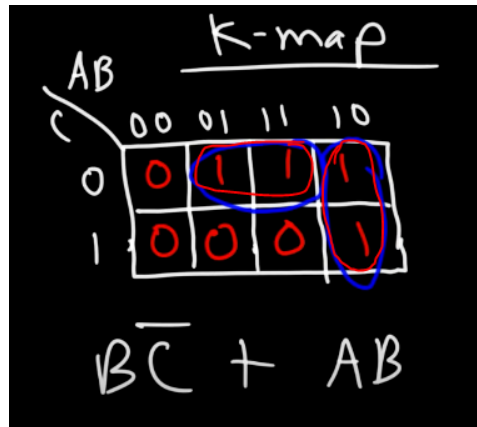
- De Morgan's Theorem:
 - the complement of the result obtained by OR'ing variables is equivalent to AND'ing the complements of the individual variables, and
 - the complement of the result obtained by AND'ing variables is equivalent to OR'ing the complements of the individual variables



- Many different logic diagrams are possible for a given Function
- Exclusive-OR (XOR):
 - inputs = 2: HIGH if only one of the inputs is HIGH
 - inputs > 2: HIGH if odd number of inputs are HIGH
- Exclusive-NOR (XNOR):
 - inputs = 2: either both are HIGH or both are LOW
 - inputs > 2: HIGH if even number of inputs are HIGH
- Karnaugh Map (K-map): visual grid for simplifying Boolean functions
 - **Draw:** label rows/columns in Gray code, fill in 1s from truth table
 - Gray code: binary sequence where only one bit changes between successive values (00, 01, 11, 10)
 - **Simplify:** group adjacent 1s in powers of two (2, 4, ...); write one term per group and combine

- Sum of Products (SOP) - the variables that don't change across the combination are included (combine with AND); different regions are combined with OR; complement variable if it's value is 0;

◦ <https://www.youtube.com/watch?v=RO5aIU6PpSU>



- Product of Sums (POS) - group by 0 and select variables that don't change across the combination and combine them with OR; different regions are combined with AND; complement variable if it's value is 1;

◦ <https://www.youtube.com/watch?v=eznPb3DWOQ0>

		$A + B$			
A	BC	00	01	11	10
	0	0	0	1	0
	1	0	1	1	1

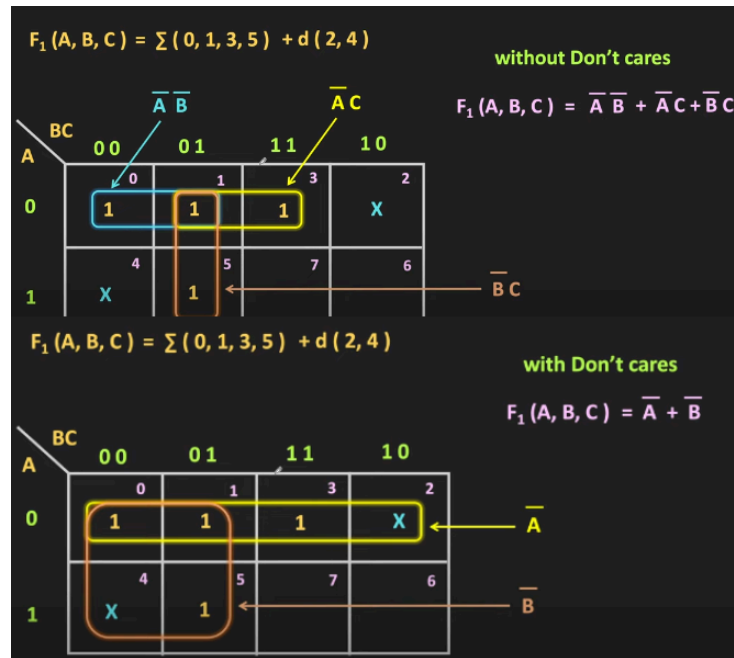
$B + C$

$A + C$

$$Y = (A + B) \cdot (B + C) \cdot (A + C)$$

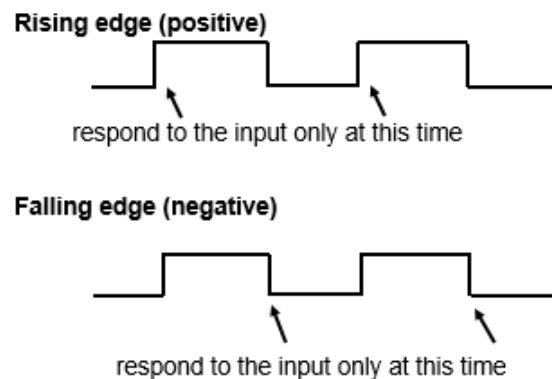
- Don't-care conditions are represented by x's in the corresponding cells (can be 0 or 1) - merge X's too

- i.e. numbers 0-9 are represented from 0000 - 1001, we don't care about 1010 - 1111
- don't cares are given as d(...) in the function.
- <https://www.youtube.com/watch?v=ZayoUTi2tsA>

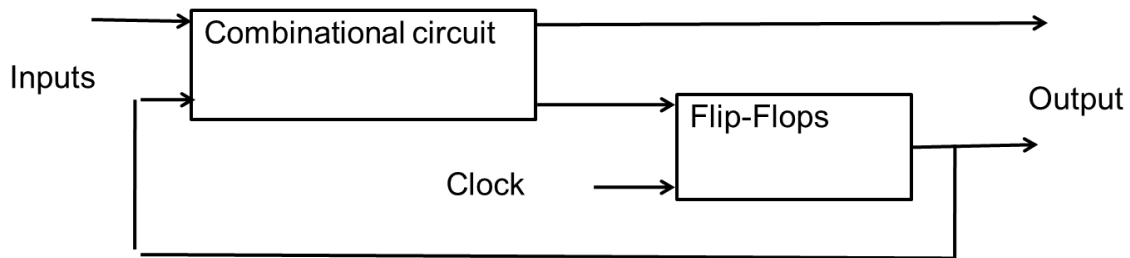


- Combinational logic circuits: outputs at any given time are entirely dependent on the inputs that are present at that time; i.e. half adder, full adder
- Sequential circuits: contain storage elements so that the output depends on previous inputs
 - Synchronous (clocked): most common, all storage elements, achieved through timing device (clock pulse generated), operated when clock pulse is either 0 or 1
 - Asynchronous: no clock to synchronize parts, not widely used
- Flip flops: binary cell that can hold a value of one bit; has two outputs: normal value and complement; maintains state until next clock pulse; operation is specified by a Characteristic Table
 - Clocked Flip flops: to synchronize individual flip flops to clock pulse (has extra c terminal for clock)

- D Flip-flop: has only one input (D) and it simply transfers the input to the output on the clock edge, effectively acting as a 1-bit memory that stores the value of D
- S-R (Set-Reset) flip-flops:
 - **S=0, R=0** → **No change** (holds previous state)
 - **S=0, R=1** → **Reset** (Q = 0)
 - **S=1, R=0** → **Set** (Q = 1)
 - **S=1, R=1** → **Invalid** (undefined state)
- JK Flip-flop: Has two inputs (**J** and **K**) and is a versatile version of the SR flip-flop.
 - **J = 1, K = 0** → **Set** (output becomes 1)
 - **J = 0, K = 1** → **Reset** (output becomes 0)
 - **J = 1, K = 1** → **Toggle** (output inverts on clock edge)
 - **J = 0, K = 0** → **No change**: holds its previous state.
- T Flip-flop:
 - **T = 1** → **Toggle** (output switches between 0 and 1 on clock)
 - **T = 0** → **Hold** (output remains unchanged)
- Edge Triggered Flip flop: state transitions either on rising edge or falling edge of the clock pulse



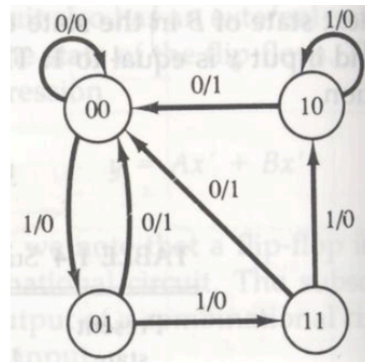
- Sequential Circuit Block Diagram: visual representation of a digital circuit with combinational logic block, flip flops, clock, and input/outputs.



- State table: table that shows the combination for current state and input → next state and output
- State diagram: graphical representation of a sequential circuit showing states as circles and transitions as arrows labeled with input/output values

Present state		Input	Next state		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

State table example



State diagram example

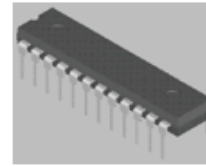
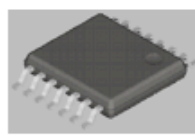
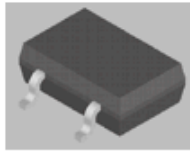
- Sequential Circuit Design Procedure:
 - Specification (desired behavior and requirements) → State Diagram (states and transitions) → State Table (states, inputs, next states, and outputs) → Excitation Table (required flip-flop inputs to achieve state transitions) → Karnaugh Map (simplify) → Circuit Diagram (gates & flip flops)

Integrated Circuits

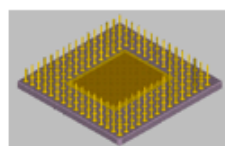
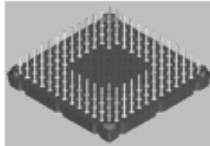
- Integrated Circuits(IC): small silicon semiconductor crystal, called chip;

- various gates are interconnected inside the chip to form the required circuit;
- chip is mounted in a ceramic or plastic container; connections are welded by thin gold wires to external pins;
- each IC has a numeric designation printed on the surface of the package for identification
- Logic family:
 - MOSFET: MOS - high component density, CMOS - low power (4000 series - RCA, MC14500 - Motorola, 5000 - Toshiba, 54C/74C Series, 54HC/74HC Series)
 - BJT: TTL - (54 series - military, 74 series - commercial), ECL
- Package types

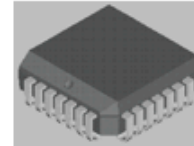
- Small Outline Transistor (SOT)
- Small Outline Package (SOP)
- Dual-In-Line Package (DIP)



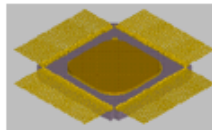
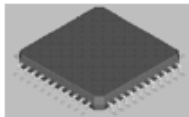
- Plastic/Ceramic Pin Grid Array (PPGA/CPGA)



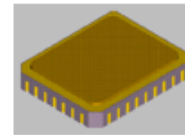
- Plastic Leaded Chip Carrier (PLCC)



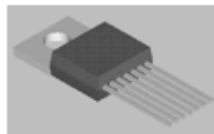
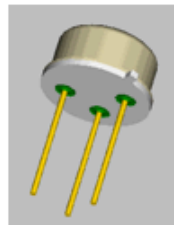
- Plastic Quad Flat Package (PQFP)



- Ceramic Leadless Chip Carrier (LCC)

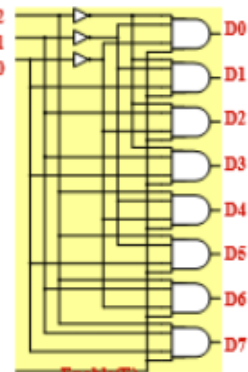


- TO Packages (Transistor single Outline)

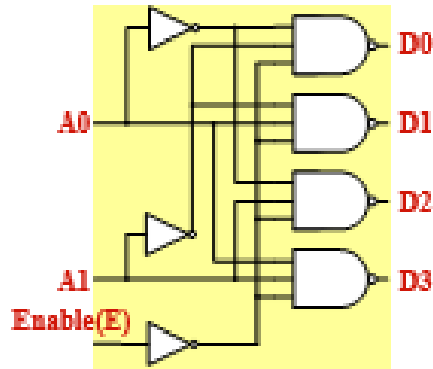


- Decoder: combinational circuit that converts binary from n coded inputs to a maximum of 2^n outputs;
 - n -to- m line decoder ($n \times m$): using AND and NOT gates

Enable	Inputs			Outputs								
E	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0	
0	x	x	x	0	0	0	0	0	0	0	0	A2
1	0	0	0	0	0	0	0	0	0	0	1	A1
1	0	0	1	0	0	0	0	0	0	1	0	A0
1	0	1	0	0	0	0	0	0	1	0	0	
1	0	1	1	0	0	0	0	1	0	0	0	
1	1	0	0	0	0	0	1	0	0	0	0	
1	1	0	1	0	0	1	0	0	0	0	0	
1	1	1	0	0	1	0	0	0	0	0	0	
1	1	1	1	1	0	0	0	0	0	0	0	



- NAND Gate Decoder: uses NAND instead of AND gates



- Encoder: inverse operation of a decoder, 2^n inputs and n outputs.
- Multiplexer (Mux): combinational circuit that receives binary information from one of 2^n lines (has n selection lines) and directs it to a single output line;
- Registers: a group of flip flops with each flip flop storing one bit; n-bit register has a group of n flip-flops;
 - Parallel load: load all bits at once
 - Series load: one bit at a time through a single input,
 - shift registers - shift binary in a direction, 1 serial input, 1 serial output; unidirectional or bidirectional (parallel load)
- Binary counter: a register sequence for counting per clock pulse (adds 1) - n flip flops can count from 0 to $2^n - 1$
 - parallel load - can receive initial binary number and count adding to the initial;
- Memory Unit: collection of storage cells together with a circuit to transfer the information in and out
 - memory stores binary in groups of 'words'
 - a word is processed in one simultaneous operation
 - a byte is a group of eight bits
 - a nibble is a group of four bits

- number of address line: specifies word inside memory; $2^k - 1$ words for k address line
- solid state memory (IC memory):
 - RAM (volatile): operations - read and write (control lines), n data input lines, n data output lines, k address lines
 - ROM (non-volatile) - combinational circuit that performs read operation only; comes with fuses that can be programmed for a specific configuration;
 - micro-programmed control: control unit that utilizes a ROM to store binary control information
 - $m \times n$ ROM: input k address lines ($m = 2^k$) and outputs data lines n
 - Types: UVEPROM(Chip level erase), EEPROM(Byte level erase), Flash ROM(Page or block level erase), OTPROM, Mask ROM

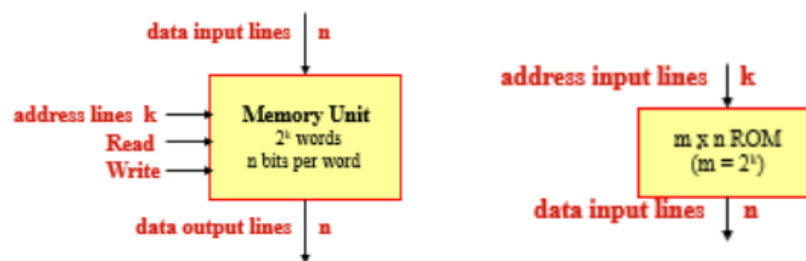


Fig. 2-12 Block diagram of RAM

Data Representation

- Data types: Numbers, Letters, Other discrete symbols
- Number systems: base or radix r system (1 symbol has r digits)
 - Decimal (base 10), Binary (base 2), Octal (base 8), Hexadecimal (base 16)
- Complements: used for simplifying the subtraction operation and for logical manipulation
 - r-1 complement (1's compliment): change 0's to 1, and 1's to 0

- r-complement (2's complement): add 1 to the r-1 complement (e.g., for binary, 2's complement = 1's complement + 1)
- $A - B = A + (2\text{'s complement of } B)$
- Fixed-Point Representation: specify point in binary (fractions)
 - fixed point: binary point always at a fixed position (point not actually present but assumed)
 - `00010110` - if 4 bits for integer, 4 for fraction $\rightarrow$ `0001.0110` = 1.375
 - floating point: mantissa and exponent
 - 1 sign bit | 3 exponent bits | 4 mantissa bits
 - `0 100 1100`
 - **Sign (S):** `0` $\rightarrow$ positive number
 - **Exponent (E):** `100` (binary) = 4 (decimal)
 - **Mantissa (M):** `1100`
 - Exponent bias (allows positive and negative exponents) - for k=3 bit exponents: $2^{k-1} - 1 = 2^{3-1} - 1 = 2^2 - 1 = 4 - 1 = 3$
 - Actual exponent: decimal exponent - exponent bias = $4 - 3 = 1$ (if e was `010` it would be $\underline{2} - 3 = -1$)
 - Normalized mantissa = 1 + fraction bits
 - Fraction bits = $0.1100_2 = 1 \times \frac{1}{2} + 1 \times \frac{1}{4} + 0 + 0 = 0.75$
 - Mantissa = $1 + 0.75 = 1.75$
 - Value = $(-1)^S \times \text{Mantissa} \times 2^{\text{Actual exponent}} = 1 \times 1.75 \times 2^1 = 3.5$
 - Overflow
 - Occurs when adding two numbers with the same sign (both positive or both negative)
 - For unsigned numbers: overflow detected by carry out from the MSB (most significant bit)

- For signed numbers:
 - MSB is the sign bit
 - Sign bit treated as part of the number
 - Carry out from MSB does not indicate overflow
- Overflow Detection
 - Check carry into and carry out of the sign bit position
 - If carries differ (XOR = 1), overflow occurs
- Decimal Fixed-Point Representation
 - Each decimal digit uses 4 flip-flops (F/Fs)
 - Example: 4385 in BCD = 0100 0011 1000 0101 (16 F/Fs total)
 - Uses more storage and complex circuits compared to binary representation
- Gray Code
 - Only one bit changes between successive values
 - Used in ADCs and rotary encoders to avoid transition errors
- Other Decimal Codes
 - Excess-3: Add binary 3 to BCD; unweighted, self-complementing
 - 2421 Code: Weighted code; weights are 2, 4, 2, 1
 - Digit 7 1101: $(1 \times 2) + (1 \times 4) + (0 \times 2) + (1 \times 1) = 7$
 - Digit 77 = Digit 7 Digit 7 = 1101 1101
- Alphanumeric Codes
 - ASCII: Includes characters and control/control flow symbols (e.g., CR, LF, ACK)
 - EBCDIC: Used mainly in IBM systems
- Error Detection Codes
 - Parity Bit: Added to ensure total 1's are even or odd

▪ **Even Parity:** Make total 1's **even**

- Input: 1100 (2 ones – already even) → Parity bit: 0 → 11000
- Input: 1011 (3 ones – odd) → Parity bit: 1 → 10111

▪ **Odd Parity:** Make total 1's **odd**

- Input: 1100 (2 ones – even) → Parity bit: 1 → 11001
- Input: 1011 (3 ones – already odd) → Parity bit: 0 → 10110

- Generator/Checker: Adds/checks parity at sender/receiver
- Detects single-bit errors only; can't locate or fix error